Contents

第五册《AI 时代下的运筹学》 **4**
本册目录 . . . 4
1. 数学工具/预备知识 . . . 4
2. 完整推导 . . . 5
3. 直觉理解 . . . 7
4. 常见误区 . . . 8
5. 代码实现 . . . 8
6. 小结 . . . 9
7. 思考题 . . . 9
1. 数学工具/预备知识 . . . 10
2. 完整推导 . . . 11
3. 直觉理解 . . . 13
4. 常见误区 . . . 13
5. 代码实现 . . . 13
6. 小结 . . . 14
7. 思考题 . . . 14
1. 数学工具/预备知识 . . . 14
2. 完整推导 . . . 15
3. 直觉理解 . . . 17
4. 常见误区 . . . 17
5. 代码实现 . . . 17
6. 小结 . . . 18
7. 思考题 . . . 18
1. 数学工具/预备知识 . . . 19
2. 完整推导 . . . 19
3. 直觉理解 . . . 21
4. 常见误区 . . . 21
5. 代码实现 . . . 21
6. 小结 . . . 22
7. 思考题 . . . 23
1. 数学工具/预备知识 . . . 23
2. 完整推导 . . . 23
3. 直觉理解 . . . 25
4. 常见误区 . . . 25
5. 代码实现 . . . 26
6. 小结 . . . 26
7. 思考题 . . . 27
1. 数学工具/预备知识 . . . 27
2. 完整推导 . . . 27
3. 直觉理解 . . . 29
4. 常见误区 . . . 29
5. 代码实现 . . . 29
6. 小结 . . . 31
7. 思考题 . . . 31
1. 数学工具/预备知识 . . . 31
2. 完整推导 . . . 32
3. 直觉理解 . . . 34
4. 常见误区 . . . 34
5. 代码实现 . . . 34
6. 小结 . . . 35
7. 思考题 . . . 35
1. 数学工具/预备知识 . . . 35
2. 完整推导 . . . 36

3. 直觉理解 37
4. 常见误区 38
5. 代码实现 38
6. 小结 39
7. 思考题 39
1. 数学工具/预备知识 39
2. 完整推导 40
3. 直觉理解 41
4. 常见误区 41
5. 代码实现 41
6. 小结 42
7. 思考题 42
1. 数学工具/预备知识 43
2. 完整推导 43
3. 直觉理解 44
4. 常见误区 44
5. 代码实现 44
6. 小结 45
7. 思考题 45
1. 数学工具/预备知识 46
2. 完整推导 46
3. 直觉理解 47
4. 常见误区 47
5. 代码实现 47
6. 小结 48
7. 思考题 48
1. 数学工具/预备知识 49
2. 完整推导 49
3. 直觉理解 51
4. 常见误区 51
5. 代码实现 51
6. 小结 51
7. 思考题 52
1. 数学工具/预备知识 52
2. 完整推导 53
3. 直觉理解 54
4. 常见误区 54
5. 代码实现 54
6. 小结 55
7. 思考题 55
1. 数学工具/预备知识 56
2. 完整推导 57
3. 直觉理解 58
4. 常见误区 58
5. 代码实现 58
6. 小结 59
7. 思考题 59
1. 数学工具/预备知识 59
2. 完整推导 60
3. 直觉理解 61
4. 常见误区 62
5. 代码实现 62
6. 小结 62
7. 思考题 63
1. 数学工具/预备知识 63

2. 完整推导 64
3. 直觉理解 65
4. 常见误区 65
5. 代码实现 65
6. 小结 66
7. 思考题 66
1. 数学工具/预备知识 67
2. 完整推导 67
3. 直觉理解 68
4. 常见误区 69
5. 代码实现 69
6. 小结 70
7. 思考题 70
1. 数学工具/预备知识 70
2. 完整推导 71
3. 直觉理解 72
4. 常见误区 73
5. 代码实现 73
6. 小结 73
7. 思考题 74
1. 数学工具/预备知识 74
2. 完整推导 75
3. 直觉理解 76
4. 常见误区 76
5. 代码实现 76
6. 小结 77
7. 思考题 78
1. 数学工具/预备知识 78
2. 完整推导 78
3. 直觉理解 79
4. 常见误区 79
5. 代码实现 80
6. 小结 80
7. 思考题 80
1. 数学工具/预备知识 81
2. 完整推导 81
3. 直觉理解 82
4. 常见误区 83
5. 代码实现 83
6. 小结 83
7. 思考题 83
1. 数学工具/预备知识 84
2. 完整推导 84
4. 常见误区 86
5. 代码实现 86
6. 小结 87
7. 思考题 87
1. 数学工具/预备知识 88
2. 完整推导 88
3. 直觉理解 89
4. 常见误区 89
5. 代码实现 89
6. 小结 90
7. 思考题 91
1. 数学工具/预备知识 91

2. 完整推导 92
3. 直觉理解 93
4. 常见误区 93
5. 代码实现 93
6. 小结 94
7. 思考题 94

第五册《AI 时代下的运筹学》

从线性规划、对偶、排队、库存，到可微优化、RL 求解组合优化、LLM 辅助建模，补全从智能到决策的最后一环。

共 24 讲·合订版

本册目录

1. 第 1 讲 线性规划与几何直觉
2. 第 2 讲 对偶理论
3. 第 3 讲 整数规划与分支定界
4. 第 4 讲 离散性的代价
5. 第 5 讲 图与网络流
6. 第 6 讲 动态规划
7. 第 7 讲 排队论
8. 第 8 讲 库存模型：报童问题与 (s,S) 策略
9. 第 9 讲 组合优化：从 TSP 到割平面与分支定界
10. 第 10 讲 启发式的两个家族
11. 第 11 讲 在线算法：在信息不完整时做决策
12. 第 12 讲 鲁棒优化：对不确定性做最坏打算
13. 第 13 讲 学习增广算法：用数据替代人工规则
14. 第 14 讲 强化学习求解组合优化：从构造到改进
15. 第 15 讲 端到端可微优化：把 LP 嵌入神经网络
16. 第 16 讲 神经网络的形式化验证：用运筹方法证明安全性
17. 第 17 讲 LLM 辅助建模：从自然语言到优化模型
18. 第 18 讲 需求预测与库存联动：从预测误差到决策损失
19. 第 19 讲 供应链网络优化：从单仓到多级网络
20. 第 20 讲 定价与收益管理：动态价格的最优结构
21. 第 21 讲 用运筹框架读论文
22. 第 22 讲 找创新点的方法论：从缺口到论文
23. 第 23 讲 实验诊断工具箱：从跑通到跑准
24. 第 24 讲 论文写作的运筹范式：从算法到科学贡献

第 1 讲 线性规划与几何直觉

本讲问题：一家面包房一天要烤面包和蛋糕两种产品。每炉面包耗面粉 2kg、人工 1 小时，利润 30 元；每炉蛋糕耗面粉 1kg、人工 2 小时，利润 50 元。当天面粉库存 200kg、人工 120 小时。老板该各烤多少炉，才能把利润做到最大？这是一个再朴素不过的分配问题，但它的数学结构——线性约束、线性目标——正是运筹学一切模型的原子。

1. 数学工具/预备知识

线性规划（Linear Programming，下文统一简称 **LP**）研究的是：在一组**线性等式/不等式约束**下，优化一个**线性目标函数**。它不是”高级数学”，而是把”用有限资源换最大收益”这件事翻译成代数。下面把本讲要用到的工具一次铺清。

1.1 向量、矩阵与线性函数

把 n 个决策变量排成一个列向量 $x = (x_1, \dots, x_n)^\top \in \mathbb{R}^n$。一个”线性地依赖于 x“的量，写成向量内积：

$$f(x) = c^\top x = \sum_{j=1}^{n} c_j x_j = c_1 x_1 + c_2 x_2 + \cdots + c_n x_n.$$

c 也是一个向量，它的每个分量告诉我们”第 j 个变量每增加 1 单位，目标值变多少”。在面包房问题里，$c = (30, 50)^\top$ 就是”每炉面包赚 30、每炉蛋糕赚 50”。

一组 m 个线性约束，用矩阵形式写就是 $Ax \le b$：A 是 $m \times n$ 矩阵，第 i 行 $a_i^\top$ 代表”第 i 种资源的单耗”，b_i 是”第 i 种资源的总量”。

1.2 半空间、超平面与多面体

- **超平面**：$\{x \mid a^\top x = b\}$，是 $\mathbb{R}^n$ 中一个 $(n-1)$ 维的”平的墙”。二维里它是一条直线，三维里它是一个平面。
- **半空间**：$\{x \mid a^\top x \le b\}$，一个不等式切出来的”墙的一侧”。
- **多面体（polyhedron）**：有限个半空间的交集。LP 的可行域 $\{x \mid Ax \le b,\ x \ge 0\}$ 就是一个多面体。

为什么强调”平”？因为只要约束是线性的，可行域的每一面墙都是直的，没有凹凸。这是后面一切几何结论的前提。

1.3 凸集与凸组合

一个集合 S 叫**凸集**，如果任取两点 $y, z \in S$，整条连线段 $\{\lambda y + (1-\lambda) z \mid 0 \le \lambda \le 1\}$ 都落在 S 里。

半空间是凸的：若 $a^\top y \le b$ 且 $a^\top z \le b$，则 $a^\top(\lambda y + (1-\lambda)z) = \lambda a^\top y + (1-\lambda) a^\top z \le b$。而”凸集的交集仍是凸集”，所以可行域（多个半空间的交）是凸集。

这一点至关重要：**可行域没有洞、没有凹陷**，你从可行域内任一点走到另一点，中途不会掉出去。

1.4 顶点（极点）

多面体的**顶点**（vertex，也叫极点）是那种”不能写成集合内另外两个点的凸组合”的点。二维里它就是凸多边形的”角”。

记住本讲最核心的一句话：**如果一个 LP 有最优解，那么一定存在一个最优解是可行多面体的某个顶点**。本讲第 2.4 节会严格证明它。

2. 完整推导

2.1 从业务语言到数学语言

把面包房问题形式化。设烤面包 x_1 炉、蛋糕 x_2 炉。

- 目标：最大化利润 $\max\ 30x_1 + 50x_2$。
- 面粉约束：$2x_1 + x_2 \le 200$。
- 人工约束：$x_1 + 2x_2 \le 120$。
- 非负：$x_1, x_2 \ge 0$。

写成紧凑形式：

$$\begin{aligned} \max \quad & c^\top x \\ \text{s.t.} \quad & Ax \le b, \\ & x \ge 0, \end{aligned}$$

其中 $x = (x_1, x_2)^\top$，$c = (30, 50)^\top$，$A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$，$b = (200, 120)^\top$。

逐行读这个 A：第一行 $(2,1)$ 是”烤一炉面包吃 2kg 面粉、烤一炉蛋糕吃 1kg 面粉”，乘上 x 就是总面粉消耗，不能超过 200kg。第二行同理是人工。

2.2 可行域的几何

二维情形可以画图。四条约束：

1. $2x_1 + x_2 = 200$，截距 $(100,0)$、$(0,200)$；
2. $x_1 + 2x_2 = 120$，截距 $(120,0)$、$(0,60)$；
3. $x_1 = 0$（纵轴）；
4. $x_2 = 0$（横轴）。

可行域是这四个半空间交出的凸多边形，顶点依次为：

- $O = (0,0)$；
- $A = (100,0)$：面粉约束与 $x_2 = 0$ 的交——令 $x_2 = 0$，由 $2x_1 = 200$ 得 $x_1 = 100$；
- B：两条资源约束的交点（下一小节详算）；
- $C = (0,60)$：人工约束与 $x_1 = 0$ 的交——令 $x_1 = 0$，由 $2x_2 = 120$ 得 $x_2 = 60$。

为什么顶点一定是两条约束线的交点？因为”角”就是两面墙碰头的地方。二维多面体的每个顶点，恰好由两条（线性无关的）约束方程联立解出。推广到 n 维：一个顶点要由 n 个线性无关的紧约束确定。

2.3 求顶点 B 的具体计算

解联立方程 $\begin{cases} 2x_1 + x_2 = 200 \\ x_1 + 2x_2 = 120 \end{cases}$。

第一步：第一式乘 2，得 $4x_1 + 2x_2 = 400$。第二步：减去第二式，$4x_1 + 2x_2 - (x_1 + 2x_2) = 400 - 120$，即 $3x_1 = 280$，所以 $x_1 = 280/3 \approx 93.33$。第三步：代回第一式，$2 \times (280/3) + x_2 = 200$，即 $560/3 + x_2 = 600/3$，所以 $x_2 = 40/3 \approx 13.33$。

因此 $B = (280/3,\ 40/3)$。检验：代回第二式 $x_1 + 2x_2 = 280/3 + 80/3 = 360/3 = 120$，恰好等于右端，验证通过。注意 $x_2 = 40/3$ 不是 $80/3$——后者是常见的算术笔误，回代能立刻揪出来。

2.4 为什么最优解在顶点上（核心证明）

目标函数 $z = 30x_1 + 50x_2$ 是一族平行直线（等值线），斜率为 $-30/50 = -3/5$。等值线的法向量正是 $c = (30,50)$，它指向”利润增大最快”的方向。我们要把这族等值线沿着 c 方向尽可能往上推，直到它”刚好还碰到”可行域——再推就整个落到可行域外面了。

“刚好还碰到”意味着等值线最后接触到的必然是某个顶点（或者某条与等值线平行的边——这时边上所有点都是最优解，但两个端点仍是顶点）。

下面给出严格证明。设 x^* 是一个最优解，但它不是顶点。按顶点定义，x^* 可以写成两个可行点 y, z 的凸组合：

$$x^* = \lambda y + (1-\lambda) z, \qquad 0 < \lambda < 1, \quad y, z \text{ 为可行点且} y \neq z.$$

两边用 $c^\top$ 作用：

$$c^\top x^* = \lambda c^\top y + (1-\lambda) c^\top z.$$

分两种情况：

- 若 $c^\top y \neq c^\top z$，则 $\lambda c^\top y + (1-\lambda)c^\top z$ 一定严格小于 $\max\{c^\top y, c^\top z\}$（加权平均不可能超过最大值）。于是 y, z 中至少有一个的目标值比 x^* 更大，与 x^* 最优矛盾。
- 若 $c^\top y = c^\top z = c^\top x^*$，则 y, z 同样是最优解。但 y, z 都比 x^* “更靠外”（因为 x^* 是它们的中点）。沿着线段 y 到 z 的方向继续走，由于多面体有界，必然能走到这个方向上的某个边界顶点，而目标值一路不变，所以那个顶点也是最优解。

两种情况都推出”存在一个顶点最优解”。证毕。这就是单纯形法的几何根基：既然最优解躲在某个角上，那就从一个角出发，一步一步跳到相邻的、目标不更差的角，直到跳不动。

2.5 代入验证四个顶点的利润

逐点计算：

- $O=(0,0)$：$z=0$ 元。
- $A=(100,0)$：$z=30\times100=3000$ 元。
- $B=(280/3,40/3)$：$z=30\times(280/3)+50\times(40/3)=2800+2000/3=10400/3\approx3466.67$ 元。
- $C=(0,60)$：$z=50\times60=3000$ 元。

最大在 B 点：烤 $280/3\approx93.3$ 炉面包、$40/3\approx13.3$ 炉蛋糕，利润约 3466.67 元。注意现实里炉数是整数——这就是第 3 讲整数规划要处理的事。

2.6 标准形与松弛变量

为了让算法统一，LP 常写成等式标准形：

$$\begin{aligned}\min\quad & c^\top x\\ \text{s.t.}\quad & Ax=b,\\ & x\ge0.\end{aligned}$$

不等式 $Ax\le b$ 通过引入**松弛变量** $s\ge0$ 变成 $Ax+s=b$。s_i 就是”第 i 种资源还剩多少没用完”。面包房问题加松弛后变成三维等式约束，可行域从二维多边形变成三维多面体，顶点与原来一一对应。

单纯形法就是在这个标准形下：先找到一个顶点（一个”基可行解”），再检验所有相邻顶点哪个能让目标变好，跳过去，直到没有任何相邻顶点更优——此时当前顶点就是最优。

2.7 一个最小的单纯形演算（可选）

为了把”从一个角跳到相邻角”落到纸面，给面包房问题加松弛变量 $s_1,s_2\ge0$：

$$\max 30x_1+50x_2,\quad 2x_1+x_2+s_1=200,\quad x_1+2x_2+s_2=120,\quad x_1,x_2,s_1,s_2\ge0.$$

从顶点 $O=(0,0)$ 出发，此时 $s_1=200,s_2=120$。检验”让哪个非基变量进基能改善利润”：x_1 每单位贡献 30，x_2 每单位贡献 50，故选 x_2 进基。x_2 最多能涨到多少？由第一式 $x_2=200-2x_1-s_1\le200$，第二式 $2x_2=120-s_2-x_1\Rightarrow x_2\le60$。取紧的那个 $\min(200,60)=60$，所以 $x_2=60$，此时 s_2 降到 0、出基——这一步从 O 跳到了 $C=(0,60)$。

在 C 点利润 3000。再检验：x_1 进基。把第二式解出 $x_2=60-0.5x_1-0.5s_2$，代入第一式得 $s_1=200-2x_1-(60-0.5x_1-0.5s_2)=140-1.5x_1+0.5s_2$。$x_1$ 上限由 $s_1\ge0\Rightarrow x_1\le140/1.5\approx93.33$ 与 $x_2\ge0\Rightarrow x_1\le120$ 取小，即 93.33。于是 $x_1=280/3$、s_1 出基，跳到 B。此时再检验所有非基变量 (s_1,s_2) 的边际贡献都已 ≤0，无法再改善——B 即最优。这正是 2.5 节算到的点。

2.8 进阶：LP 到底有多难

一个自然的问题：顶点总数可能非常多（m 个约束里挑 n 个紧约束，组合数可达 $\binom{m}{n}$），单纯形法在最坏情况下要走指数多步。历史上 Klee–Minty 构造了一个”被压扁的立方体”坏例子，让单纯形法走遍所有顶点。

但另一条路是 1979 年 Khachiyan 的**椭球法**，首次证明 LP 存在多项式时间算法；1984 年 Karmarkar 的**内点法**（interior-point method）则在大规模问题上实用、高效。今天的商用求解器（Gurobi、CPLEX、COPT）对 LP 普遍同时内置单纯形与内点法，按问题规模自动选择。这解释了为什么几千几万行约束的 LP 在笔记本上几秒就能解。

3. 直觉理解

直觉：把可行域想象成一间凸形屋子，墙壁都是约束。目标函数是一盏探照灯，你要站在屋里、沿着灯光最强的方向一直走，直到撞上墙——你最后停下的位置一定是屋子的某个墙角。为什么不是墙中

间？因为屋子是凸的，墙中间的位置总能沿着墙的某一侧再往前挪一步，而挪一步不会破坏任何约束。这就是”顶点最优”的几何画面。

更进一步，资源约束 $2x_1 + x_2 \leq 200$ 像一道墙，墙的位置由”面粉库存 200”决定。如果库存增加，墙就往外推，屋子变大，最优利润只会涨不会跌——这就是下讲对偶理论要算的”影子价格”：每多 1kg 面粉，利润能涨多少。

与 AI/大模型的联系

LP 是大模型推理算力分配的底层模型：GPU 显存、KV-cache 容量是约束，不同请求的重要性是目标系数，调度器就是在解一个 LP——“在显存约束下，哪些请求优先处理、批量多大，才能吞吐最大”。大模型推理服务（vLLM、TensorRT-LLM）的 continuous batching 本质上是在每一步动态解一个小 LP。理解 LP 的几何直觉，是理解 AI 系统资源调度的前提。

4. 常见误区

误区一：约束越多越好、可行域越大越好。约束越多，可行域越小，最优值越差（对最大化问题）。每加一条约束，都是在削掉一块可能的最优解区域。建模时要警惕”为了显得严谨”而塞进一堆冗余约束——它们既增加求解时间，又可能把真正的最优解切掉。

误区二：最优解一定贴着坐标轴（某种产品不生产）。面包房例子里最优解 B 两种产品都生产，是两条资源约束同时起作用的交点。判断哪些约束”紧”（active，等号成立）比判断哪些变量为 0 更重要。

误区三：把算术错误当求解器错误。本例中 $x_2 = 40/3$ 容易算成 $80/3$，代入约束验证能立刻发现。工程上每次求出解，都要回代约束检查松弛——最优点处被耗尽的资源松弛为 0，没用完的资源松弛为正。

5. 代码实现

下面用 `scipy.optimize.linprog` 解面包房问题。`linprog` 是 SciPy 自带的 LP 接口，底层默认调用 HiGHS 求解器（成熟的开源单纯形/内点实现）。注意它**只做最小化**，所以利润系数要取负。

```
import numpy as np
from scipy.optimize import linprog

# 目标：minimize -c^T x   （等价于 maximize c^T x，因为 linprog 只支持最小化）
c = [-30, -50]          # 利润系数取负：最小化 (-30x1 -50x2) = 最大化 (30x1+50x2)

# 不等式约束 A_ub @ x <= b_ub
A_ub = [[2, 1],         # 面粉：2x1 + x2 <= 200
        [1, 2]]         # 人工：x1 + 2x2 <= 120
b_ub = [200, 120]

# 变量边界：x1, x2 >= 0（(0, None) 表示下限 0、无上限）
bounds = [(0, None), (0, None)]

# method="highs" 使用 HiGHS 求解器，速度和数值稳定性都好
res = linprog(c, A_ub=A_ub, b_ub=b_ub, bounds=bounds, method="highs")

print(" 求解状态:", res.message)
print(" 最优解 x1(面包炉数) =", round(res.x[0], 3))
print(" 最优解 x2(蛋糕炉数) =", round(res.x[1], 3))
print(" 最大利润 =", round(-res.fun, 2), " 元")   # res.fun 是最小化目标，取负还原利润

# 验证两条约束是否" 紧"（松弛是否为 0）
slack_flour = 200 - (2*res.x[0] + res.x[1])
slack_labor = 120 - (res.x[0] + 2*res.x[1])
print(" 面粉剩余松弛:", round(slack_flour, 4))
print(" 人工剩余松弛:", round(slack_labor, 4))
```

运行结果应当给出 $x_1 \approx 93.33$、$x_2 \approx 13.33$、最大利润约 3466.67 元，且两条松弛都接近 0——说明在最优点面粉和人工**全部耗尽**，这正是两条约束同时为紧的几何事实。

再画一张图，把可行域、顶点、目标等值线可视化，直观印证”最优在顶点”。

```
import numpy as np
import matplotlib
matplotlib.use("Agg")   # 无显示环境下用 Agg 后端，避免弹窗报错
import matplotlib.pyplot as plt

# 可行域顶点（按顺序连起来，最后回到起点闭合）
vertices = np.array([[0,0],[100,0],[280/3,40/3],[0,60],[0,0]])

fig, ax = plt.subplots(figsize=(7,6))
ax.fill(vertices[:,0], vertices[:,1], alpha=0.25, color="steelblue", label=" 可行域")
ax.plot(vertices[:,0], vertices[:,1], "o-", color="steelblue")

# 画两条约束线
x = np.linspace(0, 120, 100)
ax.plot(x, 200-2*x, "--", color="orange", label="2x1+x2=200 (面粉)")
ax.plot(x, (120-x)/2, "--", color="green", label="x1+2x2=120 (人工)")

# 画目标等值线（过最优点，z=10400/3）
z = 10400/3
ax.plot(x, (z-30*x)/50, "-", color="red", label="30x1+50x2= 最优")

ax.scatter([280/3],[40/3], color="red", s=80, zorder=5)
ax.annotate(" 最优顶点 B", (280/3,40/3), textcoords="offset points", xytext=(10,-15))

ax.set_xlabel("x1 面包炉数"); ax.set_ylabel("x2 蛋糕炉数")
ax.set_xlim(-5,120); ax.set_ylim(-5,120)
ax.legend(); ax.set_title(" 线性规划的可行域与最优顶点")
plt.savefig("lp_geometry.png", dpi=120, bbox_inches="tight")
print(" 已保存 lp_geometry.png")
```

6. 小结

1. 线性规划 = 线性目标 + 线性约束，可行域是凸多面体。
2. 若 LP 有最优解，则一定存在一个顶点最优解；这是单纯形法的几何根基。
3. 最优点处”紧约束”（等号成立）决定了哪些资源被耗尽，也决定了下讲的对偶价格非零。
4. 任何业务分配问题，第一步都是把”做什么、做多少、花多少、赚多少”翻译成 c, A, b, x 四件套。

局限与延伸

- LP 的威力建立在”线性”假设上。一旦目标或约束出现乘积、分段、取整（如”要么整批生产、要么不生产”），就退化成整数规划或非凸问题，难度陡增——见第 3、4 讲。
- LP 本身在大规模下仍是多项式可解，但”建模对不对”远比”解不解得出”重要：垃圾进、垃圾出。
- 延伸阅读：经典教材 Dantzig & Thapa《Linear Programming》；内点法历史可读 Nesterov–Nemirovski；商用求解器文档见 Gurobi、COPT（中国数学规划求解器）的官方手册。

7. 思考题

基础档

1. 如果蛋糕利润从 50 涨到 80（即 $c = (30, 80)$），最优解会移动到哪个顶点？请先手算四个顶点的利润判断，再用代码验证。
2. 面粉库存从 200 增加到 210（其他不变），最大利润增加多少？这个数值就是”面粉的影子价格”，下讲会严格推导它。

进阶档

3. 若新增一条约束 $x_1 \le 80$（烤箱一次只能烤 80 炉面包），可行域变成几边形？最优解会变到哪个顶点？手算验证，再用代码确认。
4. 证明：若某 LP 的最优等值线恰好与可行域的一条边平行，则这条边上的所有点都是最优解。这种情况在什么条件下发生（提示：目标系数向量与边的法向量成比例）？

开放档

5. 现实中烤炉数必须是整数。如果要求 x_1, x_2 为整数，把 LP 最优解四舍五入（$93, 13$）一定可行且最优吗？试说明为什么”取整”不是可靠办法，并猜想应该用什么方法（预告第 3 讲分支定界）。
6. 在大模型推理调度里，哪些”约束”是真正线性的、哪些其实是离散或非线性的？直接套 LP 会在什么地方失真？

第 2 讲 对偶理论

> **本讲问题**：上一讲面包房老板在最优点把面粉和人工全部耗尽了。现在他问：如果我再多进 1kg 面粉，利润到底能多赚多少？再雇 1 小时人工呢？这个”每单位资源的边际价值”，不是拍脑袋估出来的，而是由一个和原问题长得几乎对称的数学问题自动算出来的——它叫**对偶问题**（Dual Problem）。

1. 数学工具/预备知识

对偶理论是拉格朗日乘子法在约束优化里的系统化版本。先把工具补全。

1.1 拉格朗日乘子的直觉

回顾一个熟悉的场景：在约束 $g(x) \le b$ 下最大化 $f(x)$。拉格朗日的办法是把约束”吸收”进目标：

$$L(x, y) = f(x) + y\,(b - g(x)), \qquad y \ge 0.$$

y 是乘子，俗称**影子价格**（shadow price）。它的角色是对”违反约束”开出罚单：如果你想用超过 b 的资源，相当于要按价格 y 补交费用。在最优点，y 恰好衡量”资源每放宽一单位，目标能改善多少”。

1.2 从拉格朗日到对偶

对原问题 $\max\{c^\top x : Ax \le b,\ x \ge 0\}$，把 m 条约束 $Ax \le b$ 全部乘上非负乘子 $y \ge 0$ 后并入目标：

$$L(x, y) = c^\top x + y^\top (b - Ax) = y^\top b + (c^\top - y^\top A)x.$$

注意对任意**原可行** x，因为 $y \ge 0$ 且 $Ax \le b$，有 $y^\top(b - Ax) \ge 0$，所以

$$c^\top x \le c^\top x + y^\top (b - Ax) = L(x, y).$$

也就是说 $L(x, y)$ 是原目标 $c^\top x$ 的一个上界。要找”最紧的上界”，就让 L 对 x 的系数为零（否则 $x \to \infty$ 会让 L 无界）：要求 $c^\top - y^\top A \le 0$，即 $A^\top y \ge c$。此时 $L(x, y) = y^\top b$。于是”找最紧上界”这件事，恰好是

$$\min_{y \ge 0,\ A^\top y \ge c} y^\top b,$$

这就是对偶问题。原问题在”把资源变着花样赚钱”，对偶问题在”给资源定价使收购成本最小”。

1.3 三个关键定理（先列后证）

- **弱对偶**：任意可行原 x、任意可行对偶 y，都有 $c^\top x \le y^\top b$。
- **强对偶**：若原问题有有限最优解，则对偶也有有限最优解，且二者目标值相等 $c^\top x^* = b^\top y^*$。
- **互补松弛**：$y_i^*(b_i - A_i x^*) = 0$——资源若有剩余，其对偶价格必为 0。

2. 完整推导

2.1 对偶问题的构造

原问题（Primal）：

$$(P) \quad \max_{x} \; c^\top x \quad \text{s.t.} \quad Ax \le b,\; x \ge 0.$$

其中 $A \in \mathbb{R}^{m\times n}$，$m$ 条资源约束、n 个决策变量。对偶问题（Dual）：

$$(D) \quad \min_{y} \; b^\top y \quad \text{s.t.} \quad A^\top y \ge c,\; y \ge 0.$$

构造口诀：

- 原问题是 max、约束是 $\le$，对偶就是 min、变量 $y \ge 0$；
- 原问题每个资源约束 i 对应一个对偶变量 y_i（资源 i 的价格）；
- 原问题每个变量 j 对应一条对偶约束 $A_{\cdot j}^\top y \ge c_j$（生产产品 j 消耗的资源按对偶价格折算，不能低于它的利润 c_j，否则房东宁可不生产）。

2.2 弱对偶定理的推导

任取原可行 x（$Ax \le b,\; x \ge 0$）和对偶可行 y（$A^\top y \ge c,\; y \ge 0$）。

因为 $y \ge 0$ 且 $Ax \le b$，两边乘 $y^\top$ 不改变不等号：

$$y^\top A x \le y^\top b.$$

又因为 $A^\top y \ge c$，两边乘 $x \ge 0$：

$$(A^\top y)^\top x = y^\top A x \ge c^\top x.$$

联立得：

$$c^\top x \le y^\top A x \le y^\top b.$$

这就是弱对偶：**原问题任何可行解的目标值，都不超过对偶问题任何可行解的目标值**。换句话说，对偶目标 $y^\top b$ 是原目标的一个上界。

举个数字例子：随便取 $y = (1, 1)$，检查对偶可行性：$2 \times 1 + 1 = 3 \ge 30$? 不满足。取 $y = (10, 10)$：$20 + 10 = 30 \ge 30$，但 $10 + 20 = 30 \ge 50$? 不满足。再取 $y = (15, 20)$：$30 + 20 = 50 \ge 30$，$15 + 40 = 55 \ge 50$，这是对偶可行解。对偶目标 $= 200 \times 15 + 120 \times 20 = 3000 + 2400 = 5400$。弱对偶说原目标 ≤ 5400——是个上界，但很松。真正的最优对偶解会把这个上界压到 3466.67。

2.3 强对偶定理

若原问题有有限最优解 x^*，则对偶问题也有有限最优解 y^*，且

$$c^\top x^* = b^\top y^*.$$

直观上：弱对偶给了上界，强对偶说这个上界**是紧的**——两边在最优点贴上了。证明思路：单纯形法走到最优顶点 x^* 时，把约束分块为”紧约束对应的最优基 B “与非基部分，取 $y^{*\top} = c_B^\top B^{-1}$（$c_B$ 是基变量对应的目标系数）。可直接验证它满足 $A^\top y^* \ge c$，且目标值 $b^\top y^* = c_B^\top B^{-1} b = c_B^\top x_B^* = c^\top x^*$。这一步同时证明了对偶可行与强对偶相等。

2.4 互补松弛

最优点处有:

$$y_i^*(b_i - A_i x^*) = 0, \quad i = 1, \dots, m.$$

含义:

- 若资源 i 有剩余（$b_i - A_i x^* > 0$），则 $y_i^* = 0$——这种资源富余，再多进 1 单位也不增加利润;
- 若 $y_i^* > 0$，则对应约束必为紧（$A_i x^* = b_i$）——这种资源稀缺，边际价值为正。

对称地，对原变量也有互补松弛：若 $x_j^* > 0$，则对偶约束必取等号 $A_{\cdot j}^\top y^* = c_j$。

2.5 面包房问题的对偶

原问题:

$$\max\ 30x_1 + 50x_2 \quad \text{s.t.} \quad \begin{cases} 2x_1 + x_2 \le 200 \\ x_1 + 2x_2 \le 120 \\ x_1, x_2 \ge 0 \end{cases}$$

对偶变量 y_1 = 面粉价格（元/kg），y_2 = 人工价格（元/小时）。对偶问题:

$$\min\ 200y_1 + 120y_2 \quad \text{s.t.} \quad \begin{cases} 2y_1 + y_2 \ge 30 \\ y_1 + 2y_2 \ge 50 \\ y_1, y_2 \ge 0 \end{cases}$$

2.6 求解对偶、读出影子价格

在原最优点 $x^* = (280/3, 40/3)$，两条约束都紧（$2x_1 + x_2 = 200$、$x_1 + 2x_2 = 120$），且两个变量都为正。由互补松弛，对偶约束也必取等号:

$$\begin{cases} 2y_1 + y_2 = 30 \\ y_1 + 2y_2 = 50 \end{cases}$$

第一步：第一式乘 2 减第二式：$4y_1 + 2y_2 - (y_1 + 2y_2) = 60 - 50$，即 $3y_1 = 10$，$y_1 = 10/3 \approx 3.33$。
第二步：代回第一式：$20/3 + y_2 = 30$，$y_2 = 70/3 \approx 23.33$。

验证对偶目标：$200 \times 10/3 + 120 \times 70/3 = 2000/3 + 8400/3 = 10400/3 \approx 3466.67$。原目标：$30 \times 280/3 + 50 \times 40/3 = 2800 + 2000/3 = 10400/3 \approx 3466.67$。两边相等——强对偶验证通过。

2.7 影子价格的经济解释与有效范围

$y_1^* = 10/3 \approx 3.33$ 元/kg：面粉每多进 1kg，最大利润增加约 3.33 元。$y_2^* = 70/3 \approx 23.33$ 元/小时：人工每多雇 1 小时，最大利润增加约 23.33 元。

但这个”边际价值”只在当前最优点附近、且两条约束仍紧的范围内成立。可以手算面粉的有效范围：保持最优基不变，约束右端 b_1 从 200 变到多少时最优顶点仍由这两条约束交出？由 $x_1 = (2b_1 - 120)/3$、$x_2 = (240 - b_1)/3 \ge 0$ 得 $b_1 \le 240$；由 $x_1 \ge 0$ 得 $b_1 \ge 60$。所以面粉在 $[60, 240]$ 区间内，影子价格 10/3 都成立；超出这个范围，最优顶点切换，价格跳变。这就是为什么采购谈判只能”小幅加价进货”，不能指望靠无限加面粉线性增收。

与 AI/大模型的联系

对偶理论在大模型推理里有一个直接对应：**KV-cache 显存是约束，不同请求的 token 长度是决策变量**。对偶价格告诉你”多 1GB 显存能多服务多少请求”——这就是推理引擎做 batch size 决策时的边际价值。vLLM

的 PagedAttention 本质上是在动态解一个 LP：显存约束下最大化吞吐，对偶价格告诉调度器”哪种请求最占显存、该优先调度谁”。

3. 直觉理解

直觉：把原问题想象成”自己开店生产赚钱”，对偶问题想象成”一个房东来收购你的全部资源”。房东要开价 y_i 买走你所有的面粉和人工，他的目标是让总收购成本 $200y_1 + 120y_2$ 最小。但他必须让你**自愿把资源卖给他、而不是自己生产**——也就是说，按他的价格，你自己生产任何一种产品都赚不到比把资源卖掉更多的钱。这就是对偶约束 $2y_1 + y_2 \geq 30$、$y_1 + 2y_2 \geq 50$ 的来历：房东把价压到刚好让你”自己干不如卖给他”，这时他的最小成本恰好等于你的最大利润。强对偶说的就是：**你自己干赚的钱，和房东把你资源全包走花的钱，一分不差**。

4. 常见误区

误区一：把影子价格当成”资源的市场价格”。影子价格是**在当前最优生产计划下、资源的边际内部价值**，它随最优解变化，不等于批发市场上 1kg 面粉真的卖多少钱。实际采购决策要对比市场价和影子价格——市场价低于影子价格就值得进货，反之不该进。

误区二：以为对偶问题只是”数学游戏”。当原问题有 1000 个变量、只有 10 条约束时，解对偶（只有 10 个变量、1000 条约束）往往比解原问题快得多。这在大规模 LP 求解中是常规操作。

误区三：忽略对偶形式随原约束符号变化。本讲的构造建立在”原 max 配 $\leq$ 约束、变量 ≥ 0 “上。若原约束是 $\geq$ 或等式，对应对偶变量的符号和非负性都要跟着变（等式约束对应自由变量）。直接套口诀不看符号是初学者翻车的重灾区。

5. 代码实现

用 `scipy` 同时解原问题和对偶问题，验证强对偶并读出影子价格。

```
import numpy as np
from scipy.optimize import linprog

# 原问题: max 30x1+50x2  s.t. 2x1+x2<=200, x1+2x2<=120, x>=0
c_primal = [-30, -50]
A_ub = [[2,1],[1,2]]
b_ub = [200,120]

res_p = linprog(c_primal, A_ub=A_ub, b_ub=b_ub, bounds=[(0,None),(0,None)], method="highs")
print(" 原问题最优解 x =", np.round(res_p.x,4))
print(" 原问题最大利润 =", round(-res_p.fun,4))

# 对偶问题: min 200y1+120y2  s.t. 2y1+y2>=30, y1+2y2>=50, y>=0
# linprog 做 min, 把 >= 约束两边乘 -1 变成 <= 再喂给 A_ub
c_dual = [200, 120]
A_ub_dual = [[-2,-1],[-1,-2]]
b_ub_dual = [-30,-50]

res_d = linprog(c_dual, A_ub=A_ub_dual, b_ub=b_ub_dual, bounds=[(0,None),(0,None)],
 ↪  method="highs")
print(" 对偶问题最优解 y =", np.round(res_d.x,4), " (期望 y1=10/3, y2=70/3)")
print(" 对偶问题最小成本 =", round(res_d.fun,4))

print(" 强对偶检验: 原利润 =", round(-res_p.fun,4),
      " 对偶成本 =", round(res_d.fun,4))

# res_p.ineqlin.marginals 直接给出对偶变量（影子价格）
# 注意 HiGHS 对 <= 约束返回的边际在最小化问题下符号需取反
print("scipy 读出的不等式边际 (面粉, 人工) =", np.round(res_p.ineqlin.marginals,4))
```

再做数值实验：把面粉库存从 199 微调到 202，看利润变化是否等于 $y_1^* = 10/3 \approx 3.33$。

```
import numpy as np
from scipy.optimize import linprog

for b_flour in [199, 200, 201, 202]:
    r = linprog([-30,-50], A_ub=[[2,1],[1,2]], b_ub=[b_flour,120],
                bounds=[(0,None),(0,None)], method="highs")
    print(f" 面粉 ={b_flour}, 最大利润 ={-r.fun:.4f}")
```

预期：面粉每涨 1kg，利润涨约 3.33 元（199→200→201→202 的利润差都应接近 10/3），验证影子价格。

6. 小结

1. 对偶问题是原问题的”资源定价镜像”：原 max 约束 $\leq$ 对应对偶 min 变量 ≥ 0。
2. 弱对偶给出上界，强对偶说最优值两边相等。
3. 互补松弛：松弛资源的对偶价格必为 0，紧资源的对偶价格即影子价格。
4. 影子价格 = 当前最优计划下资源的边际内部价值，且只在最优基不变的有效范围内成立。

局限与延伸

- 强对偶依赖凸性（线性规划天然凸）。对非凸/整数规划，拉格朗日对偶通常只给出一个**对偶间隙**（dual gap），即原最优值与对偶最优值之差，这个间隙正是整数规划难的根源——见第 4 讲。
- 影子价格是”局部”的：右端一旦出了有效范围，价格跳变，不能外推。
- 延伸阅读：对偶理论奠基之作 Gale–Kuhn–Tucker（KKT 条件）；求解器灵敏度报告（sensitivity analysis）见 Gurobi/COPT 手册；拉格朗日松弛在整数规划中的应用读 Wolsey《Integer Programming》。

7. 思考题

基础档

1. 如果人工约束从 $x_1 + 2x_2 \leq 120$ 放宽到 ≤ 130，最大利润增加多少？用 $y_2^* = 70/3$ 估算，再用代码精确算，并确认它仍在人工的有效范围内。
2. 若面包利润从 30 涨到 40，对偶约束变成 $2y_1 + y_2 \geq 40$，对偶最优解会不会变？为什么？手算新的对偶解。

进阶档

3. 构造一个原问题，使其最优解处面粉约束是松弛的（$2x_1 + x_2 < 200$），并验证对应对偶变量 $y_1 = 0$。提示：让蛋糕利润足够高，最优解只烤蛋糕。
4. 求面粉影子价格 10/3 的有效区间（提示：保持两条约束仍紧且 $x \geq 0$）。若面粉库存涨到 250，影子价格变成多少？为什么？

开放档

5. 原问题有 5000 个变量、3 条约束，对偶只有 3 个变量。请解释为什么求解器此时会优先解对偶，并在什么规模阈值下这种”解对偶”的策略最划算。
6. 对整数规划应用拉格朗日松弛得到的对偶最优值，为什么通常小于等于（而不是等于）整数最优值？这个间隙从哪来？

第 3 讲 整数规划与分支定界

> **本讲问题**：面包房老板看完第 1 讲的答案皱了眉——“你让我烤 93.33 炉面包？炉是一炉一炉进的，怎么烤三分之一炉？”现实里，产品件数、人数、车辆数、班次都是整数。把连续变量强行四舍五入到整数，经常会撞到约束外面，或者离最优解差十万八千里。整数规划就是处理这种”离散决策”的工具，而分支定界是它的主力算法。

1. 数学工具/预备知识

- **整数规划（Integer Programming, IP）**：LP 基础上要求部分或全部变量取整数值。

- **纯整数规划**：所有变量都整数；**混合整数规划（Mixed-Integer Programming, MIP）**：部分变量整数、部分连续。
- **0-1 变量**：只取 0 或 1 的整数变量，是表达”做不做、选不选”的原子。
- **LP 松弛（LP relaxation）**：把整数约束 $x_j \in \mathbb{Z}$ 放宽为 $x_j \geq 0$ 连续，得到的 LP。
- **分支定界（Branch and Bound, B&B）**：一种隐式枚举法，把可行解空间树状切分，并用松弛的界剪枝。

核心观察：**LP 松弛的最优值是整数规划最优值的界**（对最大化问题，LP 松弛给出上界）。理由很简单：整数可行解集是连续可行解集的子集，子集上的最大值不可能超过全集上的最大值。这个”上界”是 B&B 一切剪枝的根据。

为什么整数规划难

LP 是多项式时间可解的，但一旦要求取整，问题就变成 NP-hard。直观原因：连续可行域是一个光滑凸多面体，最优解总在某个顶点；而整数最优解是这个多面体里”稀疏格点”中的最高点，两者之间没有简单的梯度关系。这个”离散性的代价”会在第 4 讲专门量化。

2. 完整推导

2.1 整数规划建模

面包房问题加整数约束：

$$\begin{aligned} \max \quad & 30x_1 + 50x_2 \\ \text{s.t.} \quad & 2x_1 + x_2 \leq 200, \\ & x_1 + 2x_2 \leq 120, \\ & x_1, x_2 \in \mathbb{Z}_{\geq 0}. \end{aligned}$$

LP 松弛的最优解是 $x^* = (280/3, 40/3) \approx (93.33, 13.33)$，利润 $10400/3 \approx 3466.67$。

四舍五入到 $(93, 13)$：检查约束 $2 \times 93 + 13 = 199 \leq 200$，$93 + 2 \times 13 = 119 \leq 120$，可行，利润 $30 \times 93 + 50 \times 13 = 2790 + 650 = 3440$。

但 $(93, 13)$ 真的最优吗？试 $(92, 14)$：$2 \times 92 + 14 = 198 \leq 200$，$92 + 2 \times 14 = 120 \leq 120$，可行，利润 $2760 + 700 = 3460$，比 $(93, 13)$ 高。再试 $(90, 15)$：$180 + 15 = 195$，$90 + 30 = 120$，利润 $2700 + 750 = 3450$。可见 $(92, 14)$ 利润 3460 是个候选，比 LP 松弛值 3466.67 低 6.67——这个差距就是”离散性的代价”。

2.2 分支定界的核心思想

暴力枚举所有整数点在变量多的时候不可能。B&B 的做法：

1. **松弛**：解当前子问题的 LP 松弛，得到上界 U（最大化问题）。
2. **剪枝（fathoming）**：
   - 若松弛最优解本身是整数解，记为候选可行解，更新下界 L；
   - 若松弛最优值 $\leq L$，这个子树里不可能有更好解，剪掉；
   - 若松弛无可行解，剪掉。
3. **分支（branching）**：选一个分数变量 $x_j^* = \bar{x}_j$（非整数），把当前问题切成两个子问题：
   - 左子问题：$x_j \leq \lfloor \bar{x}_j \rfloor$；
   - 右子问题：$x_j \geq \lceil \bar{x}_j \rceil$。

   这两条新约束合起来覆盖所有整数可行解（任何整数要么 $\leq \lfloor \bar{x}_j \rfloor$，要么 $\geq \lceil \bar{x}_j \rceil$），同时把分数解 $\bar{x}_j$ 排除在外——注意 $\lfloor \bar{x}_j \rfloor$ 与 $\lceil \bar{x}_j \rceil$ 之间没有整数，所以不会漏掉任何整数解。

2.3 在面包房问题上跑一遍

根节点：LP 松弛解 $x = (93.33, 13.33)$，上界 $U = 3466.67$。选 $x_1 = 93.33$ 分支。

左子问题 L1：加约束 $x_1 \le 93$。在 $x_1 = 93$ 边界上，代入 $2\times 93 + x_2 \le 200 \Rightarrow x_2 \le 14$，$93 + 2x_2 \le 120 \Rightarrow x_2 \le 13.5$，所以 $x_2 \le 13.5$。最优 $x_2 = 13.5$，利润 $30\times 93 + 50\times 13.5 = 2790 + 675 = 3465$。$x_2 = 13.5$ 还是分数，上界 $U_{L1} = 3465$。

右子问题 R1：加约束 $x_1 \ge 94$。代入 $2\times 94 + x_2 \le 200 \Rightarrow x_2 \le 12$，$94 + 2x_2 \le 120 \Rightarrow x_2 \le 13$，所以 $x_2 \le 12$。最优 $x_2 = 12$，利润 $30\times 94 + 50\times 12 = 2820 + 600 = 3420$。这个解 $(94, 12)$ 是整数！记为当前最好候选 $L = 3420$。

现在 $L = 3420$，回头看 L1 的上界 3465 > 3420，不能剪，继续分支 L1。

L1 分支：$x_2 = 13.5$，切为 $x_2 \le 13$ 与 $x_2 \ge 14$。

- **L1a**：$x_1 \le 93$，$x_2 \le 13$。最优 $x_1 = 93, x_2 = 13$，利润 3440，整数可行，更新 $L = 3440$。
- **L1b**：$x_1 \le 93$，$x_2 \ge 14$。代入 $2x_1 + 14 \le 200 \Rightarrow x_1 \le 93$，$x_1 + 28 \le 120 \Rightarrow x_1 \le 92$。最优 $x_1 = 92, x_2 = 14$，利润 $2760 + 700 = 3460$，整数可行，更新 $L = 3460$。

现在所有活节点的上界：R1 已探明（3420），L1a 已探明（3440），L1b 已探明（3460）。没有活节点了，最优解就是 $(92, 14)$，利润 3460。

2.4 为什么 B&B 是对的

- **完整性**：每次分支都把分数变量的所有可能整数取值分成两块，两块合起来覆盖全部整数解；递归下去直到每个叶子节点要么变量全整数、要么无可行解。所以不会漏掉任何整数可行解。
- **最优性**：剪枝只在上界 $\le L$ 时发生，而 L 是已知整数可行解的最好值。被剪掉的子树上界不超过 L，即使其中有整数解，也不会比 L 更好。

2.5 分支策略与割平面

B&B 的效率高度依赖两件事：

1. **选哪个分数变量分支**。常用规则是”最大不可行性”——挑离最近整数最远的变量（如 13.5 比 93.33 离整数更远），或”伪费用”——估计该变量分支后目标能下降多少。选得好，树能小一个数量级。
2. **加割平面（cutting plane）**。在松弛多面体上切一刀、把分数解”割掉”但不切掉任何整数解，能让上界更快逼近真实整数最优。这种”B&B + 割平面”的混合叫**分支定割（Branch and Cut）**，是现代 MIP 求解器的核心。

理论上 B&B 最坏仍是指数级，但工程上靠”紧松弛、好分支、好初始可行解”三板斧，能解上万个 0-1 变量的 MIP。

2.6 0-1 变量的三个常用建模技巧

整数规划的威力，很大程度来自 0-1 变量能表达”逻辑”。下面三个是建模日常。

技巧一：固定成本（要么不做，要做就掏一笔固定钱）。设连续变量 $x \ge 0$ 产量，0-1 变量 $y \in \{0, 1\}$ 表示”启不启动这条产线”。用

$$x \le My$$

绑定：$y = 0$ 时强制 $x = 0$；$y = 1$ 时 x 可在 $[0, M]$ 内任意。目标里减去固定成本 fy。M 要取 x 的一个上界（如最大产能），但别取过大，否则数值不稳。

技巧二：二选一约束（要么满足 A、要么满足 B）。两个互斥约束 $g(x) \le 0$ 与 $h(x) \le 0$，用 0-1 变量 y 与大 M：

$$g(x) \le My, \qquad h(x) \le M(1 - y).$$

$y = 0$ 时第一式放松（$g \le M$ 几乎无效）、第二式收紧（$h \le 0$）；$y = 1$ 反之。这就实现了”两个约束只生效一个”。

技巧三：目标取最大/最小。$\max_i\{a_i x\}$ 这种非线性，用一个辅助变量 t 和 0-1 变量族转化为线性约束，交给 MIP 求解器。这是”把非线性逻辑装进整数框架”的标准手段。

这些技巧共同的代价：每引入一个 0-1 变量，搜索树就可能翻一倍。所以建模哲学是”能用连续就用连续，必须离散才上 0-1”。

与 AI/大模型的联系

整数规划是大模型推理部署的底层问题：选哪些层量化为 8-bit、哪些保持 16-bit，这是一个 0-1 决策变量的 MIP——目标是精度损失最小、约束是显存和延迟。B&B 思想也用于神经架构搜索（NAS）：逐层分支，剪枝掉明显不优的候选结构。理解 MIP 的离散性代价，是理解 AI 系统”为什么不能无限压缩模型”的关键。

3. 直觉理解

直觉：把整数可行解想象成一片地上的稀疏树桩（整数格点）。LP 松弛是整片松软土地——你随便走都能找到一个最高点，但那个最高点往往在两个树桩之间的空地上。B&B 就是用一把刀把土地切成小块，每切一块就问：“这块地里最高的那点，比我已经找到的最高树桩还高吗?”如果不比它高，整块地连同里面所有树桩都不用看了——这就是剪枝。切到最后，剩下的小块里刚好只装得下一个树桩，你就找到了答案。

4. 常见误区

误区一：把 LP 松弛解四舍五入当整数最优解。本例中四舍五入到 $(93, 13)$ 利润 3440，比真正最优 $(92, 14)$ 的 3460 少 20 元。在大规模问题里，四舍五入甚至可能违反约束（比如四舍五入后 $2x_1 + x_2 = 201 > 200$），根本不可行。

误区二：认为”整数规划就是 LP 加个约束，求解器顺便就解了”。变量数 100 的 LP 毫秒解完，对应的 MIP 可能要几分钟甚至算不出来。建模时要克制：只有真正必须取整的变量才设为整数，能连续就连续。

误区三：以为 0-1 变量是免费的。每引入一个 0-1 变量表达”要么全做、要么不做”，就可能让搜索树多一倍深度。用大 M 法建模时，M 取得过大还会导致数值不稳定。

5. 代码实现

用 `scipy.optimize.milp`（SciPy ≥ 1.9）直接解面包房整数规划，并手写一个迷你 B&B 验证。

```
import numpy as np
from scipy.optimize import linprog, milp, LinearConstraint, Bounds

# 用 milp 求解（milp 只做最小化）
c = np.array([-30.0, -50.0])          # 利润取负
A = np.array([[2,1],[1,2]])
b = np.array([200.0, 120.0])
constraints = LinearConstraint(A, -np.inf, b)   # A x <= b
bounds = Bounds([0,0],[np.inf,np.inf])
integrality = np.array([1,1])          # 1 表示该变量必须取整数

res = milp(c, constraints=constraints, bounds=bounds, integrality=integrality)
print(" 状态:", res.message)
print(" 整数最优解 x =", res.x)
print(" 最大利润 =", -res.fun)
```

预期输出 $x = (92, 14)$，利润 3460。

下面手写一个迷你 B&B，帮助理解算法（仅用于教学，规模一大就该交给求解器）。

```
import numpy as np
from scipy.optimize import linprog

def solve_lp(c, A_ub, b_ub, extra_rows=None, extra_rhs=None):
    """ 解一个带额外约束的 LP，返回 (status, x, obj)。obj 已还原为最大化利润。"""
    A = list(A_ub); b = list(b_ub)
```

```
    if extra_rows is not None:
        A.extend(extra_rows); b.extend(extra_rhs)
    r = linprog(c, A_ub=A, b_ub=b, bounds=[(0,None),(0,None)], method="highs")
    if not r.success:
        return "infeasible", None, -np.inf
    return "opt", r.x, -r.fun

def branch_and_bound(c, A_ub, b_ub):
    best_x, best_obj = None, -np.inf
    queue = [( [], [] )]                # 待分支节点: (额外约束行, 额外右端)
    while queue:
        rows, rhs = queue.pop()
        status, x, obj = solve_lp(c, A_ub, b_ub, rows, rhs)
        if status == "infeasible" or obj <= best_obj:
            continue                    # 剪枝: 无可行解 或 上界不超过当前最好
        if np.all(np.abs(x - np.round(x)) < 1e-6):
            if obj > best_obj:          # 找到整数可行解
                best_x, best_obj = x, obj
            continue
        # 选离最近整数最远的分数变量分支
        j = int(np.argmax(np.abs(x - np.round(x))))
        floor_v, ceil_v = np.floor(x[j]), np.ceil(x[j])
        row_l = [0,0]; row_l[j] = 1     # 左: x[j] <= floor_v
        queue.append((rows+[row_l], rhs+[floor_v]))
        row_r = [0,0]; row_r[j] = -1    # 右: x[j] >= ceil_v -> -x[j] <= -ceil_v
        queue.append((rows+[row_r], rhs+[-ceil_v]))
    return best_x, best_obj

x, obj = branch_and_bound([-30,-50], [[2,1],[1,2]], [200,120])
print(" 手写 B&B 结果: x =", x, " 利润 =", obj)
```

6. 小结

1. 整数规划 = LP + 变量取整要求，是 NP-hard 问题。
2. LP 松弛给出整数最优值的界（最大化问题的上界）。
3. 分支定界通过”切分可行域 + 松弛剪枝”隐式枚举所有整数解。
4. 建模时克制使用整数变量，必要时才上 MIP 求解器。

局限与延伸

- MIP 在变量上千、约束上万时仍可能超时；现代求解器靠割平面、启发式初解、并行分支联合攻坚，但最坏复杂度没有本质改善。
- 延伸阅读：Wolsey《Integer Programming》；分支定割原始论文 Land–Doig (1960)、Dakin (1965)；商用求解器 Gurobi、COPT、HiGHS 的 MIP 文档。

7. 思考题

基础档

1. 如果面包利润从 30 改成 35，整数最优解还是 $(92, 14)$ 吗？用代码验证。
2. 在 B&B 中，如果一开始就用启发式找到 $(92, 14)$（即初始下界 $L = 3460$），哪些分支可以直接剪掉？

进阶档

3. 为什么”选哪个变量分支”会影响搜索树大小？试分别从 x_1 和 x_2 开始分支，比较节点数。
4. 引入一个 0-1 变量 y 表达”若烤面包则必须租烤箱（固定成本 500 元）“。写出约束（提示：用大 M：$x_1 \le My$，目标里减去 $500y$），并讨论 M 取 200 和取 1000 对求解速度的影响。

开放档

5. 神经架构搜索（NAS）把”选哪层用哪种算子”建成 MIP。随着网络层数加深，0-1 变量线性增长，搜索树指数膨胀。你会怎么用 B&B 之外的办法（如权重共享、强化学习）缓解?
6. 为什么说”LP 松弛越紧，B&B 越快”? 从剪枝条件 上界 $\leq L$ 出发解释：若松弛上界一开始就等于整数最优值，会发生什么?

第 4 讲 离散性的代价

> **本讲问题**：第 3 讲里，LP 松弛给出的最大利润是 3466.67 元，而真正整数最优解是 3460 元——差了 6.67 元。这个差距看起来不大，但它是整数规划最本质的现象：**连续世界的最优解，常常落在整数世界之外**。这一讲我们要回答：这个差距由什么决定? 能不能不靠暴力分支，直接在 LP 松弛上”切一刀”，把分数解挤出去?

1. 数学工具/预备知识

- **LP 松弛（LP relaxation）**：把整数约束放宽为连续，得到的 LP。它的可行域是一个包含所有整数可行点的凸多面体。
- **整数间隙（integrality gap）**：$\dfrac{z_{IP}-z_{LP}}{z_{IP}}$（最大化问题，$z_{LP}\geq z_{IP}$），衡量松弛和真实 IP 的差距。
- **有效不等式（valid inequality）**：对所有整数可行解都成立、但对某些 LP 松弛极点不成立的线性不等式。
- **割平面（cutting plane）**：一条有效不等式，作用是把当前分数最优解”切掉”、同时不切掉任何整数可行解。
- **完全幺模矩阵（Totally Unimodular, TU）**：一个整数矩阵 A，其每个方子矩阵的行列式都属于 $\{-1,0,1\}$。

核心思想：**与其在分数解上分支，不如加一条约束把分数解所在的那块区域整个切掉**。切完再解 LP，如果还是分数解，就再切。

为什么凸多面体的极点会是分数

把整数可行点（一堆稀疏格点）的凸包记为 P_{IP}。LP 松弛的可行域 P_{LP} 通常比 P_{IP} 大。LP 最优解取在 P_{LP} 的某个极点上，而这个极点不一定是整数格点——它可能落在 P_{LP} 内、P_{IP} 外的一块”虚胖”区域。整数间隙，就是这块虚胖区域给目标值带来的虚高。

2. 完整推导

2.1 整数间隙有多大

先看面包房问题：

$$z_{LP}=10400/3\approx 3466.67,\quad z_{IP}=3460.$$

间隙 $\frac{3466.67-3460}{3460}\approx 0.19\%$，非常小。这是因为这个例子的约束矩阵 $A=\begin{pmatrix}2&1\\1&2\end{pmatrix}$ 的行列式 $4-1=3$，并非 TU，但右端结构恰好让最优顶点离整数不远。

再看一个”间隙很大”的经典例子——固定费用设施选址。LP 松弛会让每个仓”开一半”，目标值比真实整数解好得多。理论上某些问题的整数间隙可以是常数倍甚至无界的。间隙大小，是判断一个 MIP”好不好解”的第一指标。

2.2 完全幺模性：什么时候间隙为 0

定理：若 A 是 $m\times n$ 整数矩阵，且每个方子行列式取值在 $\{-1,0,1\}$，则对任意整数右端 b，$Ax\leq b$，$x\geq 0$ 的极点都是整数点。此时 LP 松弛 = 整数规划，间隙为 0。

为什么? 顶点由 n 个线性无关约束联立解出，$x=B^{-1}b$。当 B 是 TU 的整数方阵时，$\det B\in\{-1,1\}$，由克莱姆法则，$B^{-1}b$ 的每个分量都是两个整数的比值，而分母是 ± 1，所以 x 必为整数。

运输问题、指派问题、最小费用流的约束矩阵都有一个共同特征：**每列恰好两个非零元，一个 $+1$、一个 -1**。这种矩阵是 TU 的。所以这些问题虽然”形式上要求整数”，但直接解 LP 就得到整数解——这就是为什么第 5 讲网络流能高效求解。

2.3 Gomory 割平面：一个通用切法

考虑一个 LP 最优单纯形表。对某个基变量 x_i，它在表中的方程是：

$$x_i + \sum_j \bar{a}_{ij} x_j = \bar{b}_i.$$

假设 $\bar{b}_i$ 不是整数，记 $\bar{b}_i = \lfloor \bar{b}_i \rfloor + f_i$，$0 < f_i < 1$。对每个非基变量系数 $\bar{a}_{ij}$，记其小数部分 $f_{ij} = \bar{a}_{ij} - \lfloor \bar{a}_{ij} \rfloor \in [0, 1)$。

对任何整数可行解，x_i 和所有 x_j 都是整数，所以

$$\sum_j \bar{a}_{ij} x_j = \bar{b}_i - x_i.$$

左边和右边都是整数，它们的小数部分必须相等：

$$\left\{ \sum_j \bar{a}_{ij} x_j \right\} = f_i.$$

注意到对非负 x_j，把 $\bar{a}_{ij} x_j$ 拆成整数部分加小数部分 $\bar{a}_{ij} x_j = \lfloor \bar{a}_{ij} \rfloor x_j + f_{ij} x_j$，前者是整数、不影响小数部分，所以 $\{\bar{a}_{ij} x_j\} \le f_{ij} x_j$。于是：

$$f_i = \left\{ \sum_j \bar{a}_{ij} x_j \right\} \le \sum_j f_{ij} x_j.$$

即得到 **Gomory 分数割**：

$$\sum_j f_{ij} x_j \ge f_i.$$

它对所有整数可行解成立，但把当前 LP 最优解切掉了——因为在那个点上所有非基 $x_j = 0$，左边 $= 0 < f_i$，不满足这条新约束。

2.4 在面包房问题上试一刀

面包房 LP 最优解 $x = (280/3, 40/3)$。用几何方式找一条有效不等式更直观：把两条资源约束相加，$2x_1 + x_2 \le 200$ 与 $x_1 + 2x_2 \le 120$ 相加得 $3x_1 + 3x_2 \le 320$，即 $x_1 + x_2 \le 320/3 \approx 106.67$。因为 $x_1 + x_2$ 是整数，向下取整得

$$x_1 + x_2 \le 106.$$

这就是一条有效不等式！它对所有整数可行解成立，但当前 LP 最优点 $x_1 + x_2 = 320/3 \approx 106.67 > 106$，被切掉了。

加这条割后重新解 LP：新约束 $x_1 + x_2 \le 106$ 与 $x_1 + 2x_2 \le 120$ 联立，相减得 $x_2 = 14$，代回得 $x_1 = 92$——正好是整数最优解 $(92, 14)$！一刀就切到了。这演示了割平面的威力：切得准，一下就把分数解挤到整数点。

2.5 割平面法 vs 分支定界

维度	分支定界	割平面法
操作	切分可行域为子问题	在原问题上加有效不等式
树结构	搜索树	单层 LP 迭代
优点	通用性强，工程成熟	紧化松弛，间隙小
缺点	树可能爆炸	割太多导致 LP 病态
现代做法	分支定割（B&C）：在 B&B 每个节点上同时加割	

2.6 整数凸包与 facet

把所有整数可行点的凸包记为 P_{IP}。它本身也是一个多面体，LP 松弛的可行域 P_{LP} 包含它。理想的”最紧松弛”就是 P_{IP} 本身——在它上面解 LP 直接得到整数最优。

P_{IP} 的每一条面（facet）都是一个不能被其他面推出的有效不等式。割平面理论的目标，就是找到这些 facet 的近似。对有些问题（如匹配问题），P_{IP} 的 facet 被完整刻画（Edmonds 的带花树），所以能高效求解；对大多数问题（如一般 0-1），P_{IP} 的 facet 指数多，只能边解边找。

2.7 为什么不能”一直切到整数”

理论上 Gomory 割平面法可以有限步收敛到整数最优，但实际中割平面数量爆炸、数值稳定性差。现代求解器把割平面作为 B&B 的辅助：在每个节点上生成少量”有用的割”（cover cut、flow cover、lift-and-project 等），紧化该节点的松弛，再继续分支。

与 AI/大模型的联系

离散性代价在 AI 里无处不在：模型压缩里”每层选不选 4-bit 量化”是 0-1 决策，LP 松弛给一个理论上界，但实际压缩比和精度损失之间的 gap 就是”整数间隙”。理解间隙大小，能帮你判断”理论最优压缩比”和”实际能做到多少”之间的差距。

3. 直觉理解

直觉：LP 松弛像一个宽松的大笼子，整数解是笼子里的几只鸟。B&B 是把笼子切成一小块一小块，每块里找鸟；割平面是往笼子里插栅栏，把鸟可能飞不到的角落封起来。插一刀没用就再插一刀，笼子越收越紧，最后鸟被困在一个小笼子里，一眼就看到了。好的割平面就是”插得准”——一刀下去正好把分数最优解那块空地封死，但不碰任何一只鸟。

4. 常见误区

误区一：以为加割平面总能把问题”变容易”。割太多会让 LP 矩阵变得很大、条件数变差，反而求解变慢。工程上要控制割的数量和质量。

误区二：把”LP 松弛解是分数”当成”问题很难”的同义词。很多问题（运输、指派、最短路径）虽然变量名义上是整数，但因为约束矩阵完全幺模，LP 解自然是整数，根本不需要 IP。看到整数约束先检查矩阵结构，再决定要不要上 MIP 求解器。

误区三：以为有效不等式”随便加”。一条错的割（切掉了某个整数可行解）会让最终解偏离真实最优。割必须先证明”对所有整数可行解成立”，才能加。

5. 代码实现

数值验证整数间隙，并演示加一条割后 LP 最优解跳到整数点。

```
import numpy as np
from scipy.optimize import linprog, milp, LinearConstraint, Bounds
```

```python
def solve_lp(c, A_ub, b_ub):
    r = linprog(c, A_ub=A_ub, b_ub=b_ub, bounds=[(0,None),(0,None)], method="highs")
    return r.x, -r.fun

c = [-30,-50]
A = [[2,1],[1,2]]
b = [200,120]

x0, z0 = solve_lp(c, A, b)
print(f" 初始 LP: x={np.round(x0,4)}, 利润 ={z0:.4f}")

# 加割: x1 + x2 <= 106
A_cut = A + [[1,1]]
b_cut = b + [106]
x1, z1 = solve_lp(c, A_cut, b_cut)
print(f" 加割后 LP: x={np.round(x1,4)}, 利润 ={z1:.4f}")

# 对比 IP 最优值
r = milp(np.array(c,dtype=float),
         constraints=LinearConstraint(np.array(A,float), -np.inf, np.array(b,float)),
         bounds=Bounds([0,0],[np.inf,np.inf]),
         integrality=np.array([1,1]))
print(f"IP 最优: x={r.x}, 利润 ={-r.fun:.4f}")
print(f" 初始整数间隙 = {z0 - (-r.fun):.4f}")
print(f" 加割后间隙   = {z1 - (-r.fun):.4f}")
```

预期：初始 LP $x \approx (93.33, 13.33)$，利润 3466.67；加割后 $x = (92, 14)$，利润 3460，间隙从 6.67 降到 0。

再做一个实验：枚举不同右端 b_1，画出整数间隙变化曲线。

```python
import numpy as np
from scipy.optimize import linprog, milp, LinearConstraint, Bounds
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

gaps = []
for b1 in range(180, 221, 2):
    r_lp = linprog([-30,-50], A_ub=[[2,1],[1,2]], b_ub=[b1,120],
                   bounds=[(0,None),(0,None)], method="highs")
    r_ip = milp(np.array([-30,-50.]),
    ↪   constraints=LinearConstraint(np.array([[2,1],[1,2.]]),-np.inf,np.array([b1,120.])),
                bounds=Bounds([0,0],[np.inf,np.inf]), integrality=[1,1])
    gap = (-r_lp.fun) - (-r_ip.fun)
    gaps.append((b1, gap))

gaps = np.array(gaps)
plt.plot(gaps[:,0], gaps[:,1], "o-")
plt.xlabel(" 面粉库存 b1"); plt.ylabel(" 整数间隙 z_LP - z_IP")
plt.title(" 整数间隙随资源右端变化")
plt.grid(True); plt.savefig("gap.png", dpi=120)
print(" 已保存 gap.png; 间隙最大值约", round(gaps[:,1].max(),3))
```

6. 小结

1. 整数间隙 = LP 松弛与 IP 最优值的差，是离散性代价的度量。
2. 完全幺模矩阵的问题间隙为 0，可直接解 LP。
3. 割平面是有效不等式，切掉分数解但不切整数可行解。

4. 现代 MIP 求解器 = 分支定界 + 割平面（分支定割）。

局限与延伸

- 整数间隙是问题固有属性，无法靠算法消除；它决定了 MIP 在最坏情况下的难度下界。
- 延伸阅读：Gomory 原始论文 (1960)；Schrijver《Combinatorial Optimization》关于 TU 与割平面的章节；Wolsey 对 flow cover cut 的推导。

7. 思考题

基础档

1. 除了 $x_1 + x_2 \le 106$，还能找一条有效不等式把面包房 LP 分数解切掉但不切 $(92, 14)$ 吗?
2. 为什么运输问题的约束矩阵是完全幺模的? 写出它的矩阵形式，观察每行每列的 $+1/-1$ 结构。

进阶档

3. 构造一个”错误的割”例子：加一条把某个整数可行解切掉的约束，求解器会给出什么结果? 为什么这很危险?
4. 证明：若 A 是 TU 的，对任意整数 b，多面体 $\{x : Ax \le b,\ x \ge 0\}$ 的极点都是整数。从克莱姆法则出发解释。

开放档

5. 在神经网络剪枝里，“每层保留多少通道”是整数决策。你会如何用割平面思想，先解一个 LP 松弛、再逐步加有效不等式逼近?
6. 当整数间隙很大时，单纯加割平面也救不了——此时该换什么算法范式?（提示：启发式、近似算法、学习增广，见第 9、10、13 讲。）

第 5 讲 图与网络流

> **本讲问题**：一个物流公司有三个仓库、四个门店，仓库各有若干吨货，门店各要若干吨货，每条路线有单位运费。怎么调货总成本最低? 这就是运输问题。它看似是个 LP，实则藏着一个深刻的数学结构——最大流与最小割的极值对偶，这一讲把它挖出来。

1. 数学工具/预备知识

- **图** $G = (V, E)$：顶点集 V、边集 E。有向边 $(u, v) \in E$ 带容量 $c_{uv} \ge 0$。
- **流** f：每条边上的非负数 f_{uv}，满足两件事：容量约束 $0 \le f_{uv} \le c_{uv}$；中间点流量守恒 $\sum_u f_{ui} = \sum_v f_{iv}$（进多少出多少）。
- **割（cut）**：把顶点分成 S, T 两部分，$s \in S$，$t \in T$。割的容量 $c(S, T) = \sum_{u \in S, v \in T} c_{uv}$。
- **剩余网络（residual graph）**：对每条已用容量 f_{uv} 的边，它还能再推 $c_{uv} - f_{uv}$ 的正向流，也能”撤销”f_{uv} 的反向流。
- **增广路（augmenting path）**：剩余网络里从 s 到 t 的一条路径。
- **最大流-最小割定理**：从 s 到 t 的最大流值 = 最小 s-t 割容量。

流量守恒的意义

中间点既不生产也不消耗货物，所以净流入为 0。只有源点 s 净流出、汇点 t 净流入。流值 $v(f)$ 定义为 s 的净流出 $\sum_v f_{sv} - \sum_v f_{vs}$。这个量等于穿过任何 s-t 割的净流量——因为中间点不积存，所有流出 s 的货最终都得穿割到 t。

2. 完整推导

2.1 运输问题的 LP 形式

设 m 个供应点 $i = 1, \dots, m$，供应量 s_i；n 个需求点 $j = 1, \dots, n$，需求量 d_j。单位运费 c_{ij}。决策变量 x_{ij} = 从 i 运到 j 的量。

$$
\begin{aligned}
\min \quad & \sum_{i,j} c_{ij} x_{ij} \\
\text{s.t.} \quad & \sum_{j} x_{ij} = s_i, \quad \forall i \\
& \sum_{i} x_{ij} = d_j, \quad \forall j \\
& x_{ij} \geq 0.
\end{aligned}
$$

约束矩阵每列恰好两个非零元（一个 $+1$ 对应供应点、一个 -1 对应需求点），这是网络流矩阵的特征——它是完全幺模的（见第 4 讲），所以 LP 解自动是整数，不需要 MIP。

2.2 最大流问题

给定源点 s、汇点 t，每条边容量 c_{uv}，求从 s 净流出的最大总量 $v(f)$。

$$
\begin{aligned}
\max \quad & v \\
\text{s.t.} \quad & 0 \leq f_{uv} \leq c_{uv}, \ \forall (u,v) \in E \\
& \sum_{u} f_{ui} - \sum_{v} f_{iv} = 0, \ \forall i \notin \{s,t\} \\
& \sum_{v} f_{sv} = v.
\end{aligned}
$$

2.3 增广路算法：一步步推流

从零流开始。每一步在剩余网络里找一条 $s \to t$ 的路径，沿这条路径把流尽量推到瓶颈容量。推完后更新剩余网络（正向剩余减少、反向剩余增加）。重复，直到剩余网络里再也找不到 $s \to t$ 的路径。

举个小例子：网络 $s \to a$ 容量 3，$s \to b$ 容量 2，$a \to t$ 容量 2，$b \to t$ 容量 3，$a \to b$ 容量 1。

- 第一步找路 $s \to a \to t$，瓶颈 $\min(3,2) = 2$，推 2。现在 $a \to t$ 满了。
- 第二步找路 $s \to b \to t$，瓶颈 $\min(2,3) = 2$，推 2。现在 $s \to b$ 满了。
- 第三步：$s \to a$ 还剩 1，$a \to b$ 剩 1，$b \to t$ 还剩 1。找路 $s \to a \to b \to t$，瓶颈 $\min(1,1,1) = 1$，推 1。
- 总流 $= 2 + 2 + 1 = 5$。剩余网络里 s 再也到不了 t，算法停。

2.4 最大流-最小割定理证明

第一步：流值不超过任何割容量。对任意割 (S,T)，源流出的全部流量必须穿过这条割（中间点守恒，不会凭空消失）。所以

$$
v(f) = \sum_{u \in S, v \in T} f_{uv} - \sum_{u \in T, v \in S} f_{uv} \leq \sum_{u \in S, v \in T} f_{uv} \leq \sum_{u \in S, v \in T} c_{uv} = c(S,T).
$$

这给出弱对偶：最大流值 $\leq$ 最小割容量。

第二步：存在一个流达到最小割容量。当增广路算法停时，记 S 为剩余网络里从 s 能到达的顶点集，$T = V \ \ S$。此时：

- 每条 $S \to T$ 的原边必已满：否则剩余网络里它还有正向容量，S 就能到 T 内的点，矛盾；
- 每条 $T \to S$ 的原边必为空：否则剩余网络里它有反向容量，S 能沿反向边”撤销”流量、走到 T 内的点，矛盾。

于是穿过割的净流量 = $\sum_{S \to T} f_{uv} - 0 = \sum_{S \to T} c_{uv} = c(S,T)$，即

$$v(f) = c(S, T).$$

结合第一步的不等式，这个流就是最大流，这个割就是最小割。这正是对偶理论在图上的化身：原问题”最大流”与对偶问题”最小割”在最优值上贴上。

2.5 最短路径：Dijkstra 手算一步

把边权 w_{uv} 当成距离，求 s 到所有点的最短距离 $d[v]$。核心松弛：

$$d[v] \leftarrow \min(d[v],\ d[u] + w_{uv}).$$

Dijkstra 要求权非负：每次选 d 最小的未定点 u，松弛它的出边。时间 $O(m + n \log n)$（二叉堆）。若有负权边，用 Bellman–Ford，时间 $O(nm)$，同时能检测负环。

举个手算小例：$s \to a$ 权 2，$s \to b$ 权 5，$a \to b$ 权 1，$a \to t$ 权 10，$b \to t$ 权 1。

- 初始 $d[s] = 0$，其余 ∞。选 s，松弛出边：$d[a] = 2$，$d[b] = 5$。
- 从未定点里选最小的 a（$d = 2$），松弛：$d[b] = \min(5, 2 + 1) = 3$，$d[t] = \min(\infty, 2 + 10) = 12$。
- 选 b（$d = 3$），松弛：$d[t] = \min(12, 3 + 1) = 4$。
- 选 t（$d = 4$）。结束。$s \to t$ 最短是 4，走 $s \to a \to b \to t$，而不是看似直达的 $a \to t$。

为什么 Dijkstra 能”选了就定死”？因为权非负，一旦 u 是当前未定点里 d 最小的，任何其他未定点绕到 u 再出发只会更远，所以 $d[u]$ 不可能再被更新——这是非负权保证的贪心正确性。

2.6 最小费用流

在最大流基础上加单位费用 w_{uv}，要求在指定流量 v 下总费用 $\sum w_{uv} f_{uv}$ 最小。求解法叫**连续最短路（successive shortest augmenting path）**：每次在剩余网络（含反向边负费用）里找 $s \to t$ 的最短费用路径，沿它尽量推流，直到流量达到 v。运输问题就是它的特例：加一个超级源 s、超级汇 t，$s \to i$ 容量 s_i 费用 0，$i \to j$ 容量 ∞ 费用 c_{ij}，$j \to t$ 容量 d_j 费用 0。

与 AI/大模型的联系

网络流是大模型分布式训练的通信底座：数据并行时，每张 GPU 的梯度要 AllReduce，这本质上是一个最小费用流问题——把梯度从各 worker 汇聚到 PS，带宽约束下最小化通信时间。Ring AllReduce、Tree AllReduce 都是网络流算法的工程实现。理解最大流-最小割，是理解”为什么 8 卡 A100 的有效带宽不是 8× 单卡带宽”的关键。

3. 直觉理解

直觉：把网络想象成一组水管，s 是水泵、t 是出水口。每条水管有粗细（容量）。最大流 = 水泵能泵出多少水；最小割 = 你要切水管把 s 和 t 断开，最少要切多粗的一组水管。最大流-最小割定理说这两个数相等——因为水流穿过去多少，你就要切多粗的管子才能拦住它。对偶理论在这里有了物理形象：原问题是”放水”，对偶问题是”拦水”。

4. 常见误区

误区一：把运输问题当普通 LP 用一般求解器解。虽然能解，但网络单纯形法比通用单纯形快几个数量级，因为它利用了图结构。调度、路由、匹配问题都应走网络流算法。

误区二：认为”最大流等于最大边权路径的容量”。错。最大流可以同时走多条路径，总和才是流值。

误区三：忽略剩余网络里的反向边。反向边不是多余的——它允许”撤销”之前的错误推流，这正是 Ford–Fulkerson 能找到最优的关键。没有反向边，算法会过早卡死在次优解。

5. 代码实现

用 scipy 解一个小型运输问题，并用 networkx 验证最大流-最小割。

```
import numpy as np
from scipy.optimize import linprog

# 3 个仓库，4 个门店
s = [20, 30, 25]                  # 供应
d = [10, 25, 20, 20]              # 需求（总和 75 = 供应总和）
c = np.array([
    [2, 3, 1, 4],
    [5, 4, 2, 3],
    [3, 2, 5, 2]
], dtype=float)                    # 单位运费

# 变量 x[i,j] 共 12 个，展平为 x[0..11]
m, n = 3, 4
c_flat = c.flatten()
A_eq = []; b_eq = []
# 供应约束：对每个仓库 i，sum_j x_ij = s_i
for i in range(m):
    row = [0.0]*(m*n)
    for j in range(n): row[i*n+j] = 1.0
    A_eq.append(row); b_eq.append(s[i])
# 需求约束：对每个门店 j，sum_i x_ij = d_j
for j in range(n):
    row = [0.0]*(m*n)
    for i in range(m): row[i*n+j] = 1.0
    A_eq.append(row); b_eq.append(d[j])

r = linprog(c_flat, A_eq=A_eq, b_eq=b_eq, bounds=[(0,None)]*(m*n), method="highs")
x = r.x.reshape(m,n)
print(" 调运方案（行 = 仓库，列 = 门店）:\n", np.round(x,2))
print(" 最小总运费 =", round(r.fun,2))
```

最大流-最小割验证（用 networkx）：

```
import networkx as nx

G = nx.DiGraph()
# 网络：s->a 3, s->b 2, a->t 2, b->t 3, a->b 1（对应 2.3 节手工例子）
edges = [("s","a",3),("s","b",2),("a","t",2),("b","t",3),("a","b",1)]
for u,v,cap in edges: G.add_edge(u,v,capacity=cap)

max_flow_val, flow_dict = nx.maximum_flow(G, "s", "t")
cut_value, partition = nx.minimum_cut(G, "s", "t")
print(" 最大流 =", max_flow_val, " 最小割 =", cut_value)
print(" 最小割的顶点划分 S =", partition[0], " T =", partition[1])
```

预期最大流 = 最小割 = 5（与 2.3 节手工推流一致），且划分应为 $S = \{s\}$ 或类似紧割。

6. 小结

1. 运输、指派、路由都是网络流 LP，约束矩阵完全幺模，解自动整数。
2. 最大流 = 最小割，是对偶理论在图上的化身。
3. 增广路算法靠剩余网络（含反向边）一步步推流，直到找不到 $s \to t$ 路。
4. 最短路径是单目标线性代价的极值问题，Dijkstra 求非负权；最小费用流把”流量”和”代价”捏在一起。

局限与延伸

- 网络流算法依赖”线性 + 可分式”代价；一旦出现固定成本、整数批量，就退回 MIP（见第 3 讲）。
- 延伸阅读：Ahuja–Magnanti–Orlin《Network Flows》是网络流百科；Ford–Fulkerson 原始论文 (1956)；networkx 文档中的 `nx.max_flow_min_cost`。

7. 思考题

基础档

1. 在 2.3 节的网络里，如果把 $a \to b$ 的容量改成 3，最大流变成多少？手算增广路，再用代码验证。
2. 为什么最短路径不能用”最大流”求解？两者目标有何不同？

进阶档

3. 把指派问题（n 个人分配 n 个任务，每人一个、每任务一人、成本 c_{ij}）写成最小费用流：画出超级源、超级汇和边容量。
4. 证明运输问题约束矩阵是完全幺模的（提示：每列恰好一个 $+1$、一个 -1）。

开放档

5. 分布式训练的 AllReduce 为什么不能简单等价成一次最大流？通信还有哪些约束（带宽、延迟、拓扑）使它比教科书网络流复杂？
6. 当边上的容量是随机时变的（如 GPU 间带宽抖动），静态最大流的解会在哪失效？该转向什么模型（提示：随机网络、鲁棒优化，见第 12 讲）？

第 6 讲 动态规划

> **本讲问题**：你要从北京自驾到深圳，途经 5 个城市群，每段路程的油耗不同。你不可能枚举所有路径——可能有上百条。但你可以反过来想：“不管我怎么走到最后一个城市，从它到深圳的最短距离是固定的。”这就是动态规划的灵魂：**把大问题拆成重叠的小问题，每个小问题只解一次**。

1. 数学工具/预备知识

- **状态（state）**：描述当前局面的最少信息。状态选得好不好，直接决定 DP 会不会爆。
- **阶段（stage）**：决策的时间步。
- **贝尔曼方程（Bellman equation）**：最优值函数满足的递推 $V(s) = \min_a \{c(s,a) + \gamma V(s')\}$。
- **最优子结构**：全局最优解由局部最优解拼接而成。
- **重叠子问题**：不同路径会访问同一状态，DP 把它存下来复用。
- **折扣因子** $\gamma \in [0,1]$：未来奖励的折损，让无穷期问题收敛。

为什么需要”最优子结构”

DP 能成立的数学前提是：从状态 s 出发的最优总代价，可以写成”第一步代价”加上”下一状态的最优总代价”。如果子问题的最优拼不出全局最优，DP 的递推就是错的。这就是为什么”最长简单路径”不能用 DP——子路径可能要重复经过某个点，破坏了简单路径约束。

2. 完整推导

2.1 最短路问题的 DP 形式

把城市分层：第 0 层北京，第 5 层深圳，中间第 1~4 层各有若干城市。设 $V_k(v)$ = 从第 k 层城市 v 到深圳的最短距离。

边界（最后一层）：$V_5(v) = 0$（已经到深圳，后面不再花钱）。

递推：对第 k 层城市 v，枚举所有能到达的下一层城市 w：

$$V_k(v) = \min_{w \in \text{next}(v)} \{c(v,w) + V_{k+1}(w)\}.$$

计算顺序：从第 4 层往第 0 层倒着算。算 $V_4(v)$ 时只需要 V_5；算 $V_3(v)$ 时只需要 V_4，以此类推。

2.2 为什么这是对的（最优性原理）

假设从 v 出发的最短路径是 $v \to w^* \to \cdots \to$ 深圳。那么 $w^* \to \cdots \to$ 深圳这一段，必须也是从 w^* 到深圳的最短路径——否则把它换成更短的一段，整条路径就更短了，与”最短”矛盾。

这就是贝尔曼最优性原理：**最优策略的子策略也是最优的**。形式化地：

$$V_k(v) = c(v, w^*) + V_{k+1}(w^*) = \min_w \{c(v, w) + V_{k+1}(w)\}.$$

关键在那个”否则矛盾”的反证：如果子段不是最优，拼接出来的整段就不会是最优——这正是递推合法的理由。

2.3 背包问题

n 件物品，第 j 件重量 w_j、价值 v_j，背包容量 W，求最大价值。

状态：$DP[j][w]$ = 只考虑前 j 件物品、总重不超过 w 的最大价值。

递推：对第 j 件物品，只有两种选择——不装它，价值就是 $DP[j-1][w]$；装它（前提 $w \geq w_j$），价值是 $v_j + DP[j-1][w-w_j]$。取大：

$$DP[j][w] = \max\left\{DP[j-1][w], \quad v_j + DP[j-1][w-w_j]\right\}.$$

边界：$DP[0][w] = 0$（没有物品，价值为 0）。

时间 $O(nW)$。这是**伪多项式**算法——W 在输入里只是一个数字，但以二进制表示时其位数是 $\log W$，所以 $O(nW)$ 关于输入位数是指数的。这也是为什么背包是 NP-hard。

2.4 序列对齐：编辑距离

把串 A 变到串 B，允许插入、删除、替换，每种操作代价 1，求最小总代价。设 $dp[i][j]$ = $A[1..i]$ 变到 $B[1..j]$ 的最小代价。最后一步只有三种可能：

$$dp[i][j] = \min \begin{cases} dp[i-1][j] + 1 & \text{删除} A[i] \\ dp[i][j-1] + 1 & \text{插入} B[j] \\ dp[i-1][j-1] + \mathbb{1}(A[i] \neq B[j]) & \text{替换或匹配} \end{cases}$$

边界：$dp[0][j] = j$（删 j 个）、$dp[i][0] = i$（插 i 个）。这就是 Levenshtein 距离，是 DNA 序列对齐、拼写检查、模糊匹配的基础。

2.5 DP 的两种实现方向

- **自顶向下（memoized recursion）**：递归 + 备忘录，只算用到的状态。适合状态空间稀疏的问题。
- **自底向上（table fill）**：按阶段顺序填表，适合状态空间稠密的情况，工程上更快（无递归开销）。

2.6 维度灾难

DP 的时间 = 状态数 × 每个状态的转移数。状态维度每加一维，状态数指数增长。这就是”维度灾难”，也是为什么强化学习要用函数近似替代精确 DP——当状态空间是连续高维时，根本存不下整张表。贝尔曼方程在 RL 里变成 $Q(s,a) \leftarrow r + \gamma \max_{a'} Q(s', a')$，其中 Q 用神经网络逼近，而不是查表。

与 AI/大模型的联系

贝尔曼方程是强化学习的理论根基：Q-learning 里 $Q(s,a) \leftarrow r + \gamma \max_{a'} Q(s',a')$ 就是贝尔曼方程的样本估计。大模型的 reasoning 链也有 DP 影子——每一步推理是一个状态，下一步选择是一个转移，最终答案是”最优路径”。理解 DP 的最优子结构，是理解 RL 为什么能收敛、以及为什么大模型 reasoning 有时会”绕远路”（不满足最优子结构）的前提。

3. 直觉理解

直觉：DP 像剥洋葱——从最外层（终点）开始，一层一层往里剥。每剥一层，你只需要知道”下一层每个状态的最优值”，不需要知道它是怎么算出来的。这把一个指数级搜索的问题，压成了一个状态数 × 转移数的多项式问题。本质上，DP 是在”利用重叠子问题”——如果两个不同路径走到了同一个城市，那从这个城市往后的最短路线是同一条，没必要算两遍。

4. 常见误区

误区一：把 DP 当成”递归 + 缓存”的技巧。 DP 的真正前提是**最优子结构**——如果子问题的最优解不能拼出全局最优，递归缓存再聪明也没用。比如最长简单路径就不能用普通 DP。

误区二：状态设计冗余。状态里塞了无关信息，状态数爆炸；状态信息不够，又拼不出最优。设计状态要恰好”够决定未来”——这是 DP 最考验功力的地方。

误区三：以为 DP 只能解最短路/背包类问题。任何有明确阶段、状态、转移的决策问题都能套——库存控制、期权定价、序列决策、价值迭代，背后都是贝尔曼方程。

5. 代码实现

实现 0-1 背包（含回溯选物）和编辑距离。

```
import numpy as np

def knapsack(weights, values, W):
    """0-1 背包：返回最大价值与选中的物品索引。"""
    n = len(weights)
    dp = np.zeros((n+1, W+1), dtype=int)
    for j in range(1, n+1):
        for w in range(W+1):
            dp[j][w] = dp[j-1][w]                          # 默认不装第 j 件
            if w >= weights[j-1]:
                dp[j][w] = max(dp[j][w],
                               values[j-1] + dp[j-1][w-weights[j-1]])
    # 回溯：从右下角往回走，判断每件物品是否被选
    chosen, w = [], W
    for j in range(n, 0, -1):
        if dp[j][w] != dp[j-1][w]:                         # 价值变了 = 选了第 j 件
            chosen.append(j-1)
            w -= weights[j-1]
    return dp[n][W], sorted(chosen)

# 物品：重量 [2,3,4,5]，价值 [3,4,5,6]，容量 5
# 枚举：选 {0,1} 重 5 价值 7；选 {3} 重 5 价值 6；选 {0} 重 2 价值 3
val, items = knapsack([2,3,4,5], [3,4,5,6], 5)
print(f" 背包最大价值 = {val}，选中物品索引 = {items}")


def edit_distance(a, b):
    """Levenshtein 编辑距离。"""
    n, m = len(a), len(b)
    dp = np.zeros((n+1, m+1), dtype=int)
```

```
    for i in range(n+1): dp[i][0] = i        # a 删光
    for j in range(m+1): dp[0][j] = j        # 插入 b
    for i in range(1, n+1):
        for j in range(1, m+1):
            cost = 0 if a[i-1]==b[j-1] else 1
            dp[i][j] = min(dp[i-1][j]+1,             # 删
                           dp[i][j-1]+1,             # 插
                           dp[i-1][j-1]+cost)        # 替/匹配
    return dp[n][m]

print("kitten -> sitting 编辑距离 =", edit_distance("kitten","sitting"))
```

预期：背包最大价值 7（选物品 0、1，重量 2+3=5）；编辑距离 kitten→sitting = 3。

延伸算例：网格世界的价值迭代

为了让贝尔曼方程从”最短路”推广到”带折扣的序贯决策”，看一个 3×3 网格。右下角是终点（价值 0），每走一步代价 1，折扣 $\gamma = 1$。价值迭代就是反复把

$$V(s) \leftarrow \min_a \{c(s,a) + V(s')\}$$

整张表扫一遍，直到收敛。下面代码实现它，并打印收敛后的价值表。

```
import numpy as np

# 3x3 网格，目标格 (2,2)，障碍 (1,1)
N = 3
goal = (2, 2)
blocked = {(1, 1)}
V = np.zeros((N, N))
delta = 1.0
it = 0
while delta > 1e-6 and it < 1000:
    delta = 0.0
    V_new = np.full((N, N), np.inf)
    for i in range(N):
        for j in range(N):
            if (i, j) == goal:
                V_new[i, j] = 0.0; continue
            if (i, j) in blocked:
                V_new[i, j] = np.inf; continue
            best = np.inf
            for di, dj in [(1,0),(-1,0),(0,1),(0,-1)]:
                ni, nj = i+di, j+dj
                if 0 <= ni < N and 0 <= nj < N and (ni,nj) not in blocked:
                    best = min(best, 1 + V[ni, nj])   # 一步代价 1 + 下一状态价值
            V_new[i, j] = best
            delta = max(delta, abs(best - V[i, j]))
    V = V_new
    it += 1

print(f" 价值迭代收敛于第 {it} 轮")
# 障碍格用 'X' 标记，其余打印到终点的最短步数
V_show = np.where(np.isfinite(V), V.astype(int), -1)
for row in V_show:
    print("  ".join(" X " if v < 0 else f"{v:2d}" for v in row))
```

输出应给出每个格子到右下角终点的最短步数（绕开障碍），这正是贝尔曼方程 $V(s) = \min_a\{1 + V(s')\}$ 在网格上的数值解。把它换成”奖励 $+1$、折扣 $\gamma = 0.9$ “，就是 Q-learning 要逼近的那张表。

6. 小结

1. 贝尔曼方程 $V(s) = \min_a \{c(s,a) + \gamma V(s')\}$ 是 DP 的核心。
2. DP 有效需要两个条件：最优子结构 + 重叠子问题。
3. 时间 = 状态数 × 转移数，状态维度过大时退化为指数（维度灾难）。
4. DP 是 RL 的理论根基——RL 就是在状态太大时用函数逼近 V。

局限与延伸

- 精确 DP 只在状态数可控时可行；连续高维问题必须靠 RL/函数逼近，而逼近又引入偏差与收敛性问题。
- 延伸阅读：Bellman《Dynamic Programming》(1957) 原始著作；Russell–Norvig 关于决策的章节；Sutton–Barto《Reinforcement Learning》讲贝尔曼方程如何驱动 Q-learning。

7. 思考题

基础档

1. 写出”最长公共子序列（LCS）“的 DP 递推 $dp[i][j]$。它和编辑距离有什么关系?
2. 为什么 0-1 背包是”伪多项式”的? 把容量 W 从 5 加倍到 10，计算时间大约增加多少?

进阶档

3. 构造一个不满足最优子结构的问题（提示：最长简单路径），说明为什么 DP 不适用。
4. 把最短路 DP 推广到”每走一步有 $\gamma = 0.9$ 折扣”。写出递推，并说明为什么 $\gamma < 1$ 能让无穷路径问题收敛。

开放档

5. 大模型的思维链（CoT）逐步推理，哪一步对应 DP 的”状态”、哪一步对应”转移”? 为什么 CoT 常常找不到全局最优（不满足最优子结构)?
6. 当状态空间太大无法填表时，用神经网络逼近 $V(s)$ 会引入哪些误差? 这些误差如何破坏贝尔曼方程的严格性?

第 7 讲 排队论

> **本讲问题**：医院急诊窗口平均每小时来 10 个病人，医生看一个病人平均 5 分钟。你直觉上会说”10 个病人 × 5 分钟 = 50 分钟 < 60 分钟，医生闲 10 分钟”。但现实里急诊室永远在排队——为什么? 因为病人到达是随机的、看病时长也是随机的，随机性会被排队系统放大。这一讲用 M/M/1 模型算清楚这个账。

1. 数学工具/预备知识

排队论（Queuing Theory）研究”随机到来的顾客”和”随机时长的服务”组成的等待系统。先把三个随机工具讲清。

1.1 泊松过程与指数分布

- **到达过程为泊松**：在长度 t 的时间内到达 k 个顾客的概率是 $P(N(t)=k) = e^{-\lambda t}(\lambda t)^k/k!$，速率 λ（单位时间平均到达数)。
- **到达间隔服从指数分布**：相邻两个顾客到达的间隔 T 满足 $P(T > t) = e^{-\lambda t}$。密度 $f(t) = \lambda e^{-\lambda t}$，均值 $1/\lambda$。
- **指数分布的无记忆性**：$P(T > s+t \mid T > s) = P(T > t)$。翻译成人话：已经等了 s 分钟还没人来，“还要再等多久”跟”从头开始等”一模一样。这是指数分布独有的性质，也是排队模型可解的关键。

1.2 Kendall 记号

$A/S/c$：A 到达过程（M = Markovian 泊松)、S 服务时间分布（M = 指数)、c 个并行服务台。所以 **M/M/1** = 泊松到达、指数服务、1 个服务台。

1.3 几个名词

- **利用率** $\rho = \lambda/\mu$：服务台忙的时间比例。μ 是服务率（单位时间能服务几个）。
- **稳态（steady state）**：系统跑久了，各状态的概率分布不再随时间变化。
- **Little 定律**：$L = \lambda W$，系统平均人数 = 到达率 × 平均逗留时间。

2. 完整推导

2.1 M/M/1 的生灭过程与流量平衡

状态 n = 系统里的顾客数（排队 + 正在服务）。状态转移：

- 从 n 到 $n+1$：一个顾客到达，速率 λ；
- 从 n 到 $n-1$：一个顾客服务完成离开，速率 μ（仅当 $n \geq 1$）。

这是一个**生灭过程**（birth-death chain）：状态只在相邻之间跳。稳态分布 π_n 满足”每个截面的流入 = 流出”。考虑把状态 $0, 1, \ldots, n$ 和状态 $n+1, n+2, \ldots$ 切开，穿过这个截面的净流量必须为零，所以

$$\lambda \pi_n = \mu \pi_{n+1}, \quad n \geq 0.$$

（从左到右穿过截面的只有”状态 n 因到达跳到 $n+1$ “，速率 $\lambda\pi_n$；从右到左穿过的只有”状态 $n+1$ 因服务完成跳到 n “，速率 $\mu\pi_{n+1}$。两者相等。）

2.2 解出稳态分布

由 $\pi_{n+1} = \frac{\lambda}{\mu}\pi_n = \rho\pi_n$，递推得 $\pi_n = \rho^n \pi_0$。

归一化要求 $\sum_{n=0}^{\infty} \pi_n = 1$。这是等比级数：

$$\pi_0 \sum_{n=0}^{\infty} \rho^n = \pi_0 \cdot \frac{1}{1-\rho} = 1 \quad \Rightarrow \quad \pi_0 = 1 - \rho.$$

（等比级数 $\sum_{n=0}^{\infty} \rho^n = 1/(1-\rho)$ 只在 $\rho < 1$ 时收敛。）于是

$$\pi_n = (1-\rho)\rho^n.$$

这是几何分布，要求 $\rho < 1$。若 $\rho \geq 1$，排队人数趋于无穷，系统不稳定——这就是为什么利用率不能到 1。

2.3 关键性能指标

平均系统人数 L（含正在服务的）：

$$L = \sum_{n=0}^{\infty} n\pi_n = (1-\rho)\sum_{n=0}^{\infty} n\rho^n.$$

用级数 $\sum_{n=0}^{\infty} n\rho^n = \rho/(1-\rho)^2$（对几何分布求期望），得

$$L = (1-\rho)\frac{\rho}{(1-\rho)^2} = \frac{\rho}{1-\rho}.$$

平均排队人数（不含正在服务）L_q：系统里 n 个人时，排队的是 $\max(n-1, 0)$，所以

$$L_q = \sum_{n=1}^{\infty} (n-1)\pi_n = L - \sum_{n=1}^{\infty} \pi_n = L - \rho = \frac{\rho^2}{1-\rho}.$$

($\sum_{n=1}^{\infty}\pi_n = 1-\pi_0 = \rho$，即服务台忙的概率。)

2.4 Little 定律与等待时间

Little 定律 $L = \lambda W$ 是一个惊人普适的结论：不管到达和服务是什么分布、几个台，稳态下”系统平均人数 = 到达率 × 顾客平均逗留时间”。直觉上：一个顾客平均在系统里待 W 时间，单位时间进来 λ 个，所以系统里”同时存在”的人数就是 λW——就像水池里的水量 = 流入速率 × 水的平均停留时间。它不依赖分布，这是它强大的原因。

由 $L = \lambda W$：

$$W = \frac{L}{\lambda} = \frac{\rho/(1-\rho)}{\lambda} = \frac{\lambda/\mu}{(1-\rho)\lambda} = \frac{1}{\mu(1-\rho)} = \frac{1}{\mu-\lambda}.$$

同理排队等待时间（等服务开始）：

$$W_q = \frac{L_q}{\lambda} = \frac{\rho^2/(1-\rho)}{\lambda} = \frac{\rho}{\mu(1-\rho)} = \frac{\rho}{\mu-\lambda}.$$

2.5 数字代入

医院例子：$\lambda = 10$ 人/小时，$\mu = 12$ 人/小时（5 分钟/人），$\rho = 10/12 = 5/6 \approx 0.833$。

- $L = \frac{0.833}{1-0.833} = 5$ 人（系统里平均 5 个病人）。
- $L_q = \frac{0.833^2}{1-0.833} \approx 4.17$ 人（排队中）。
- $W = \frac{1}{12-10} = 0.5$ 小时 = 30 分钟。
- $W_q = \frac{0.833}{2} = 0.417$ 小时 = 25 分钟。

虽然医生只忙 83%，但平均病人要等 25 分钟！这就是随机性的代价。

2.6 灵敏度分析：$\rho \to 1$ 会怎样

$W_q = \frac{\rho}{\mu(1-\rho)}$。当 ρ 从 0.8 涨到 0.9，W_q 从 $4/\mu$ 涨到 $9/\mu$；从 0.9 涨到 0.99，从 $9/\mu$ 涨到 $99/\mu$。**利用率越接近 1，等待时间非线性爆炸**（分母是 $1-\rho$）。这就是为什么服务系统不能把利用率设计到 99%——看起来”满负荷高效”，实际排队会失控。运营上要留 15%~20% 能力冗余。

2.7 M/M/c 模型

c 个并行服务台。设 $a = \lambda/\mu$（总业务量），$\rho = a/c < 1$。Erlang-C 公式给出”到达顾客必须等待（所有台都忙）“的概率：

$$P_{\text{wait}} = \frac{\dfrac{a^c}{c!}}{1-\rho} \Big/ \left[\sum_{k=0}^{c-1}\frac{a^k}{k!} + \frac{a^c}{c!}\frac{1}{1-\rho}\right].$$

平均排队人数：

$$L_q = \frac{P_{\text{wait}}\rho}{1-\rho}.$$

加服务台能显著降等待——并行服务摊薄了随机性。

与 AI/大模型的联系

排队论是大模型推理服务的容量规划工具：用户请求是”到达”，GPU 是”服务台”，λ 是 QPS，μ 是单卡每秒处理 token 数。大模型服务的 SLA（P99 延迟）设计，本质就是选 c（GPU 卡数）让 W_q 控制在阈值内。为什么大模型服务不能把 GPU 利用率跑到 99%？因为 $\rho \to 1$ 时 P99 延迟爆炸——这就是排队论的直接结论。

3. 直觉理解

直觉：排队系统像一个水库。来水（到达）和出水（服务）都是随机的，有时来水快、有时出水慢。水库水位 = 排队长度。即使长期平均来水 < 出水（$\rho < 1$），短期波动也会让水位涨高。利用率 ρ 越高，水库的”排水余量”越小，一次突发来水就能让水位暴涨。这就是为什么运营设计里要留 15%~20% 的能力冗余——不是浪费，是用来吸收随机波动的。

4. 常见误区

误区一：用”平均到达率 = 平均服务率”判断系统平衡。正确判据是 $\rho = \lambda/\mu < 1$ 且要有足够余量。把 ρ 设计在 0.95 看似高效，实际 W_q 会是 $\rho = 0.7$ 时的近 10 倍。

误区二：以为加服务台是线性收益。从 1 个加到 2 个，等待时间可能从 25 分钟降到 5 分钟（不是对半），因为并行消除了排队的指数尾部。

误区三：把 M/M/1 的公式套到确定性系统。如果到达和服务都是确定的（每 6 分钟来一个、每个 5 分钟做完），根本不会排队。M/M/1 的长队全部来自”随机性”，确定性系统是它的反面。

5. 代码实现

数值计算 M/M/1 与 M/M/c，并画利用率-等待时间曲线。

```
import math
import numpy as np

def mm1(lam, mu):
    """M/M/1: lam 到达率, mu 服务率。返回利用率与各指标。"""
    rho = lam/mu
    L = rho/(1-rho)
    Lq = rho**2/(1-rho)
    W = 1/(mu-lam)          # Little: W = L/lam
    Wq = rho/(mu-lam)
    return dict(rho=rho, L=L, Lq=Lq, W=W, Wq=Wq)

def erlang_c(lam, mu, c):
    """M/M/c: 返回" 必须等待概率" P_wait 与平均排队人数 Lq。"""
    a = lam/mu                          # 总业务量
    rho = a/c
    s = sum((a**k)/math.factorial(k) for k in range(c))
    p_wait = (a**c/math.factorial(c)) / (a**c/math.factorial(c) + (1-rho)*s)
    Lq = p_wait*rho/(1-rho)
    return p_wait, Lq

# 验证医院例子: lam=10, mu=12
print("M/M/1 医院例子:", {k: round(v,3) for k,v in mm1(10,12).items()})

# 不同服务台数 c 下的表现
for c in [1,2,3,4]:
    p_wait, Lq = erlang_c(10, 12, c)
    print(f"c={c}: P_wait={p_wait:.3f}, Lq={Lq:.3f}")

# 画 rho -> Wq 曲线
import matplotlib
```

```
matplotlib.use("Agg")
import matplotlib.pyplot as plt
mu = 12
rhos = np.linspace(0.1, 0.95, 50)
Wqs = [r/(mu-mu*r) for r in rhos]   # Wq = rho/(mu(1-rho))
plt.figure(figsize=(7,5))
plt.plot(rhos, Wqs, "b-")
plt.xlabel(" 利用率 rho = lambda/mu")
plt.ylabel(" 平均排队等待时间 Wq (小时)")
plt.title("M/M/1: 等待时间随利用率非线性爆炸")
plt.grid(True); plt.savefig("mm1.png", dpi=120)
print(" 已保存 mm1.png")
```

6. 小结

1. M/M/1 稳态分布 $\pi_n = (1-\rho)\rho^n$，几何分布，要求 $\rho < 1$。
2. $W_q = \rho/(\mu - \lambda)$，$\rho \to 1$ 时等待时间爆炸。
3. Little 定律 $L = \lambda W$ 普适于任何稳态排队系统。
4. 加并行服务台是降等待最有效的手段（Erlang-C）。

局限与延伸

- M/M/1 假设泊松到达、指数服务、无限等待空间。现实中急诊室有”不耐烦离开”（impatience）、有限床位（有限系统 M/M/c/K）、非指数服务（M/G/1），需要更复杂的模型。
- 延伸阅读：Kleinrock《Queueing Systems》两卷本；Erlang-C 原始公式；大模型 SLA 设计参考 Google/Borg 的集群排队论文。

7. 思考题

基础档

1. 把医生从 1 个加到 2 个，$\lambda = 10$ 不变，W_q 从多少降到多少？用 Erlang-C 算。
2. 如果到达是确定的（每 6 分钟一个）、服务也是确定的（5 分钟），还会排队吗？为什么？

进阶档

3. Little 定律 $L = \lambda W$ 在非稳态（系统尚未跑久）下还成立吗？为什么推导依赖”稳态”？
4. 推导 M/M/1 中”顾客到达时必须等待”的概率。它等于多少？（提示：系统非空的概率。）

开放档

5. 大模型推理里 prefill 和 decode 的服务时间分布形状完全不同（prefill 接近确定、decode 接近指数）。直接套 M/M/1 会在哪失真？该怎么建模？
6. 当请求有不同优先级时，排队规则从 FCFS 变成优先级队列，Little 定律还成立吗？平均等待时间会怎么变？

第 8 讲 库存模型：报童问题与 (s,S) 策略

> **本讲问题**：你在地铁口卖花，每束花进价 10 元、卖 25 元，卖不掉的晚上只能 3 元处理。今天该进多少束？进少了不够卖、眼睁睁看钱飞走；进多了卖不掉、折价亏损。这个”在不确定性下选一个进货量”的问题，就是经典报童模型（newsvendor problem）。它不是凭感觉拍，而是有最优解——一个临界分位数。

1. 数学工具/预备知识

- **需求随机**：D 是随机变量，分布 $F(d) = P(D \le d)$，密度 $f(d)$。
- **订货量** Q：决策变量。
- **单位欠储成本** c_u（**underage**）：少进一单位、少卖一单位的机会损失 = 售价 - 进价。

- **单位超储成本 c_o（overage）**：多进一单位、卖不掉折价的损失 = 进价 - 处理价。
- **EOQ（经济订货量）**：确定性需求下，权衡”订货固定成本”与”持有成本”的最优批量。
- (s, S) **策略**：连续盘点库存，降到 s 就补到 S。

为什么要分”欠储”和”超储”

报童的两难本质是两种错：进少了（underage）错过了本来能赚的钱；进多了（overage）压货亏了本。最优订货量不是”猜需求”，而是让这两种错的边际代价相等。

2. 完整推导

2.1 报童模型的目标

进货 Q，需求 D。

- 若 $D \geq Q$：全部卖光，收入 $Q \cdot b$，无超储。
- 若 $D < Q$：卖出 D 束、赚 $D \cdot b$，剩下 $Q - D$ 束折价、损失 $(Q - D)c_o$。

期望利润：

$$\pi(Q) = b \cdot E[\min(D, Q)] - c_o \cdot E[(Q - D)^+].$$

这里 $(x)^+ = \max(x, 0)$。等价地，考虑期望”机会损失 + 折价损失”：

$$L(Q) = c_u \cdot E[(D - Q)^+] + c_o \cdot E[(Q - D)^+].$$

最小化 $L(Q)$ 与最大化 $\pi(Q)$ 等价（只差一个常数项）。

2.2 求最优 Q^*（临界分位数）

对 Q 求导。第一项 $E[(D - Q)^+] = \int_Q^\infty (d - Q) f(d)\, dd$，由莱布尼茨法则：

$$\frac{d}{dQ} E[(D - Q)^+] = -\int_Q^\infty f(d)\, dd = -(1 - F(Q)).$$

第二项 $E[(Q - D)^+] = \int_0^Q (Q - d) f(d)\, dd$：

$$\frac{d}{dQ} E[(Q - D)^+] = \int_0^Q f(d)\, dd = F(Q).$$

所以

$$\frac{dL}{dQ} = -c_u(1 - F(Q)) + c_o F(Q).$$

令导数为零：

$$-c_u(1 - F(Q^*)) + c_o F(Q^*) = 0 \implies F(Q^*)(c_u + c_o) = c_u \implies F(Q^*) = \frac{c_u}{c_u + c_o}.$$

这就是**临界分位数公式**：最优订货量 Q^* 是需求分布的 $\dfrac{c_u}{c_u + c_o}$ 分位数。

直观推导（不依赖求导）：多进第 Q 束，只有当 $D \geq Q$（概率 $1 - F(Q)$）时它能卖出去、赚 c_u；当 $D < Q$（概率 $F(Q)$）时它压货、亏 c_o。边际期望收益 $c_u(1 - F(Q))$ 与边际期望损失 $c_o F(Q)$ 相等时停止进货，即得同一公式。

2.3 数字代入

花束例子：$c_u = 25 - 10 = 15$ 元（少卖一束少赚 15），$c_o = 10 - 3 = 7$ 元（多进一束亏 7）。

临界分位数 $= 15/(15+7) = 15/22 \approx 0.682$。

若需求服从正态 $N(\mu = 30, \sigma = 10)$，标准正态 0.682 分位数 $z \approx 0.47$，则

$$Q^* = \mu + z\sigma = 30 + 0.47 \times 10 \approx 34.7.$$

进 35 束。

2.4 临界分位数的直觉

- 当 $c_u = c_o$：临界分位数 = 0.5，进均值附近。
- 当 $c_u \gg c_o$（缺货损失远大于折价）：临界分位数 → 1，多进。
- 当 $c_o \gg c_u$（折价损失远大于缺货）：临界分位数 → 0，少进。

这和直觉一致：缺货越痛，越敢多备。

2.5 EOQ：确定性需求下的批量

需求速率恒定 μ（单位时间），每次订货固定成本 K，单位货物单位时间持有成本 h。每批订 Q，则：

- 订货频率 μ/Q，单位时间订货成本 $K\mu/Q$；
- 库存从 Q 线性降到 0，平均库存 $Q/2$，单位时间持有成本 $hQ/2$。

总成本 $C(Q) = \dfrac{K\mu}{Q} + \dfrac{hQ}{2}$。对 Q 求导令零：

$$-\frac{K\mu}{Q^2} + \frac{h}{2} = 0 \;\Rightarrow\; Q^*_{EOQ} = \sqrt{\frac{2K\mu}{h}}.$$

注意这是**平方根公式**：固定成本 K 翻 4 倍，Q^* 只翻 2 倍（因为 $\sqrt{4} = 2$）。这就是为什么”订货越贵，批量只温和增大”。

2.6 (s,S) 连续盘点策略

报童是单期。多期库存里常用 (s, S) 策略：每当库存 I 降到 s 以下，就订货把库存补到 S，订货量 $= S - I$。s 是再订货点，S 是目标库存上限。

为什么是双阈值？因为有**固定订货成本** K：每次订货要花一笔固定费用，不能天天订。一旦决定订，就一次订够推到高位 S（批量用 EOQ）；没到 s 就不订，避免频繁触发固定成本。

在需求 $N(\mu, \sigma)$、提前期 L 下：提前期内需求 $D_L \sim N(L\mu, \sqrt{L}\sigma)$，再订货点 $s = L\mu + z\sqrt{L}\sigma$，$z$ 由服务水平决定（95% 服务水平对应 $z = 1.645$）；$S = s + Q^*_{EOQ}$。

与 AI/大模型的联系

报童模型是大模型推理算力分配的底层：GPU 是”库存”，请求是”需求”。多备 GPU（超储）有闲置成本，少备（缺货）有排队延迟。临界分位数告诉你：延迟损失远大于闲置成本时，应多备 GPU——这就是为什么大模型服务商在高峰期宁愿闲置也不挤爆。vLLM 的 prefill/decode 分离调度，本质是在动态调整”库存”（KV-cache 配额）。

3. 直觉理解

> **直觉**：报童问题的临界分位数是在”缺货的痛”和”压货的痛”之间找平衡。把它想象成天平：左边放缺货成本 c_u、右边放超储成本 c_o。天平往哪边沉，你就往另一边加库存。临界分位数 $c_u/(c_u + c_o)$

就是天平平衡时缺货那一栏的占比。如果缺货痛 15 块、压货痛 7 块，天平偏缺货，所以你要备到”需求有 68% 概率不超过库存”那一点——宁可多压一点货，也别不够卖。

4. 常见误区

误区一：把订货量定为”平均需求”。平均需求对应分位数 0.5，只有在 $c_u = c_o$ 时才最优。大多数生意里缺货损失（丢客户、丢商誉）远大于折价损失，所以订货量应 > 均值。

误区二：以为需求预测准了就能解报童问题。即使均值预测准，方差仍未知——而报童公式用的是分位数，对方差敏感。预测的价值不在给均值，而在给整个分布。

误区三：把 EOQ 当万能。EOQ 假设需求恒定、无缺货、单价固定。真实需求波动、有提前期、有数量折扣，这些都要扩展模型。

5. 代码实现

数值求报童最优解，并用蒙特卡洛验证临界分位数。

```
import numpy as np
from scipy.stats import norm

# 花束例子
cu, co = 15, 7
mu, sigma = 30, 10
critical = cu/(cu+co)
z = norm.ppf(critical)                  # 标准正态临界分位数
Q_star = mu + z*sigma
print(f" 临界分位数 = {critical:.3f}, z = {z:.3f}, Q* = {Q_star:.2f}")

# 蒙特卡洛验证：扫描 Q 算期望利润
np.random.seed(0)
D = np.random.normal(mu, sigma, size=300000)
def expected_profit(Q):
    sell = np.minimum(D, Q)
    leftover = np.maximum(Q-D, 0)
    return np.mean(15*sell - 7*leftover)

Qs = np.linspace(10, 60, 200)
profits = [expected_profit(Q) for Q in Qs]
best = Qs[np.argmax(profits)]
print(f" 蒙特卡洛数值最优 Q = {best:.2f}, 最大期望利润 = {max(profits):.2f}")
# 应与解析值 Q*≈34.7 接近

# EOQ 数值验证
K, h, mu_d = 100, 2, 1000
Q_eoq = np.sqrt(2*K*mu_d/h)
print(f"EOQ 最优批量 = {Q_eoq:.2f}")
```

再做 (s, S) 库存轨迹模拟。

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

def simulate_ss(s, S, daily_mean, days=60):
    np.random.seed(1)
    inv = S
    trail = [inv]
    for _ in range(days):
        if inv <= s:
```

```
            inv = S                          # 库存降到 s 以下，补到 S
        inv -= max(np.random.normal(daily_mean, 2), 0)
        trail.append(inv)
    return trail

trail = simulate_ss(s=25, S=40, daily_mean=5)
plt.figure(figsize=(8,4))
plt.plot(trail, "-o", markersize=3)
plt.axhline(25, color="r", ls="--", label="s=25 再订货点")
plt.axhline(40, color="g", ls="--", label="S=40 目标库存")
plt.xlabel(" 天"); plt.ylabel(" 库存"); plt.title("(s,S) 策略下的库存轨迹")
plt.legend(); plt.grid(True); plt.savefig("ss.png", dpi=120)
print(" 已保存 ss.png")
```

6. 小结

1. 报童问题最优订货量 = 需求分布的 $c_u/(c_u + c_o)$ 分位数。
2. EOQ $= \sqrt{2K\mu/h}$，对固定成本平方根敏感。
3. 多期库存用 (s, S)：库存降到 s 就补到 S，提前期需求决定 s。
4. 预测的价值在分布而非均值。

局限与延伸

- 报童是单期、无补货提前期；多期、有提前期、有产能约束的版本（如 base-stock 策略）见 Silver–Pyke–Peterson《Inventory Management》。
- 延伸阅读：Arrow 等 (1951) 报童原始论文；服务水平与安全库存的进阶讨论见 Nahmias。

7. 思考题

基础档

1. 如果折价处理价从 3 元涨到 8 元，c_o 变多少？最优订货量变多还是变少？
2. 为什么 EOQ 公式 $\sqrt{2K\mu/h}$ 对固定成本 K 是平方根敏感？K 翻 4 倍，Q^* 翻几倍？

进阶档

3. 构造一个需求双峰分布（周末需求高、工作日低），报童公式还适用吗？为什么？
4. 证明 EOQ 的总成本 $C(Q) = K\mu/Q + hQ/2$ 在 Q^* 处取得最小（二阶导数 $2K\mu/Q^3 > 0$）。

开放档

5. 大模型推理里，“预分配 KV-cache”相当于提前进货。如果延迟损失 c_u 远大于闲置成本 c_o，临界分位数接近 1 意味着什么？现实上为什么不能无限接近 1？
6. 当需求分布未知、只能从历史数据估计时，报童的分位数该怎么估？分布误差会如何影响 Q^*？

第 9 讲 组合优化：从 TSP 到割平面与分支定界

> **本讲问题**：一个外卖骑手要在一个城市里送 20 单，怎么规划路线最短？这就是旅行商问题（TSP）的世俗版。20 个城市的全排列是 $20! \approx 2.4 \times 10^{18}$，枚举不可能。但 TSP 不是束手无策——它有 LP 松弛、割平面、分支定界一整套精确求解工具，能解到几千个城市。这一讲看 TSP 的建模与精确求解。

1. 数学工具/预备知识

- **TSP（旅行商问题）**：给定 n 个城市两两距离 c_{ij}，找一条经过每个城市恰好一次、回到起点的最短回路（哈密顿回路）。
- **哈密顿回路**：经过每个顶点恰好一次的环。

- **MTZ 约束（Miller–Tucker–Zemlin）**：用辅助变量 u_i 消除子环。
- **子环消除约束（subtour elimination）**：对任意真子集 S，S 内部的边数不超过 $|S|-1$。
- **Christofides 近似**：三角不等式下给 1.5 倍近似比。

为什么 TSP 难

n 个城市的不同回路数是 $(n-1)!/2$（除以 2 是因为正反方向同一条路）。$n=20$ 时已有约 6×10^{16} 条，枚举不可能。TSP 是 NP-hard：没有已知多项式算法能求最优。但”NP-hard”不等于”算不动”——靠结构（三角不等式、欧氏距离）和割平面，几千城市也能精确求解。

2. 完整推导

2.1 TSP 的 IP 建模

变量 $x_{ij}\in\{0,1\}$：是否从 i 走到 j。目标是最小化总距离：

$$\min\sum_{i,j}c_{ij}x_{ij}.$$

每组约束保证每个城市恰好进一次、出一次：

$$\sum_j x_{ij}=1,\quad \forall i\quad(\text{出度}1),\qquad \sum_i x_{ij}=1,\quad \forall j\quad(\text{入度}1),\qquad x_{ii}=0.$$

问题是：只靠进出度约束，解可能是**几个不相连的小环**（子环），而不是一个大环。例如 6 个城市可能解成 $\{1,3,5\}$ 一个环、$\{2,4,6\}$ 另一个环。所以要加子环消除。

2.2 MTZ 约束

引入辅助变量 u_i（记录访问顺序），对 $i,j\neq1$：

$$u_i-u_j+nx_{ij}\le n-1.$$

读懂它：若 $x_{ij}=1$（真的走了 $i\to j$），则约束变成 $u_i-u_j\le n-2$，等价于 $u_j\ge u_i+1$，即访问顺序严格递增；若 $x_{ij}=0$，左边 $\le n-1$ 自动成立（因为 u 取值在 $[1,n]$），约束失效。这个”递增”关系把所有城市串成一条链，不可能再分裂成独立子环。代价是引入 n 个连续变量、n^2 条约束。

2.3 割平面法（Dantzig–Fulkerson–Johnson）

不用 MTZ，而是显式加子环消除约束。步骤：

1. 解 LP 松弛（只有出度入度约束），解可能是分数的，或整数但有子环。
2. 在解里找一个子环 S（比如 $\{1,3,5\}$ 自成一个环）。
3. 加约束 $\sum_{i,j\in S}x_{ij}\le|S|-1$，禁止 S 内部闭合。
4. 重新解 LP，重复直到解是单一回路。

妙处：大多数可能的子环根本不会出现在最优解附近，所以”发现一个、切一个”比”一开始把所有子环都禁掉”省得多。这正是第 4 讲讲的割平面思想在 TSP 上的具体应用。

2.4 Christofides 1.5 近似

满足三角不等式（$c_{ik}\le c_{ij}+c_{jk}$）时，有多项式近似算法：

1. 求最小生成树 MST（连接所有城市的最短无环图）；
2. 找出 MST 中度数为奇数的顶点集 O（奇度点数必为偶数）；

3. 求 O 上的最小权完美匹配；
4. 把 MST 和匹配合并成一个欧拉图，走欧拉回路、遇到重复城市就”抄近路”跳过，得到 TSP 回路。

近似比 1.5：MST 长度 $\le$ 最优 TSP 长度，最小匹配长度 $\le$ 最优 TSP 长度的一半，合并后总长 $\le 1.5\times$ 最优。这是”没有最优保证时的稳妥下界”。

与 AI/大模型的联系

TSP 是大模型推理请求路由的抽象：n 个 GPU 上有排队中的请求，要按延迟最优分配到各 GPU，这是带容量约束的 TSP 变种（VRP）。理解 TSP 的精确求解边界，能判断”请求路由该用启发式还是上求解器”——几千个请求以内可精确，上万个必须启发式。

3. 直觉理解

直觉：TSP 的 LP 松弛像一个”松散的导游”——它允许你在城市之间随便连边，只要每个城市进出各一次。结果它可能画出几个小圈子（子环），而不是一个完整的环球路线。割平面就是每次看到小圈子就画一条禁令：“你们这一组人不许在内部闭合”。禁令一条条加，导游最后被迫走成一条大环。

4. 常见误区

误区一：以为 TSP 只能用启发式。小规模 TSP（几百城市）用 branch-and-cut 能求到最优，并有严格最优性证明。启发式是规模大到精确解算不动时才用。

误区二：把”贪心最近邻”当 TSP 近似。最近邻经常次优 20%~30%，而 Christofides 有 1.5 保证。

误区三：忽略三角不等式。 Christofides 的 1.5 比依赖三角不等式；不满足时（如飞机快但贵），近似比不成立。

5. 代码实现

先用最近邻 + 2-opt 求解，并与穷举（小规模）对比验证正确性。

```
import numpy as np

def tour_length(tour, C):
    return sum(C[tour[i], tour[(i+1)%len(tour)]] for i in range(len(tour)))

def nearest_neighbor(C, start=0):
    n = len(C); visited = [start]; cur = start
    for _ in range(n-1):
        nxt = min((j for j in range(n) if j not in visited), key=lambda j: C[cur,j])
        visited.append(nxt); cur = nxt
    return visited

def two_opt(tour, C):
    """ 确定性 2-opt：反复扫描所有 (i,j)，直到没有任何反转能改进。"""
    best = tour[:]; best_len = tour_length(best, C)
    n = len(best)
    improved = True
    while improved:
        improved = False
        for i in range(1, n-1):
            for j in range(i+1, n):
                new = best[:i] + best[i:j+1][::-1] + best[j+1:]
                L = tour_length(new, C)
                if L < best_len - 1e-9:
                    best, best_len = new, L
                    improved = True
```

```
    return best, best_len

# 5 个城市（小到可穷举验证）
coords = np.array([[0,0],[1,2],[3,1],[4,3],[2,4]], dtype=float)
C = np.linalg.norm(coords[:,None,:]-coords[None,:,:], axis=-1)

tour = nearest_neighbor(C)
print(" 最近邻路线:", tour, " 长度:", round(tour_length(tour,C),3))
opt, L = two_opt(tour, C)
print("2-opt 改进:", opt, " 长度:", round(L,3))
```

再做穷举验证（$n = 5$，$(n-1)!/2 = 12$ 条回路，完全可枚举）。

```
import numpy as np
from itertools import permutations

# 复用上一段的 C（距离矩阵）和 tour_length
def tour_length(tour, C):
    return sum(C[tour[i], tour[(i+1)%len(tour)]] for i in range(len(tour)))

coords = np.array([[0,0],[1,2],[3,1],[4,3],[2,4]], dtype=float)
C = np.linalg.norm(coords[:,None,:]-coords[None,:,:], axis=-1)

def brute_force_tsp(C):
    n = len(C); best, best_len = None, np.inf
    for perm in permutations(range(1, n)):     # 固定起点 0，枚举其余排列
        tour = (0,) + perm
        L = tour_length(list(tour), C)
        if L < best_len:
            best, best_len = tour, L
    return list(best), best_len

opt_bf, L_bf = brute_force_tsp(C)
print(" 穷举最优:", opt_bf, " 长度:", round(L_bf,3))
```

预期 2-opt 能追到穷举最优（比值 = 1），验证启发式在小例子上的正确性。

6. 小结

1. TSP 是 NP-hard，但欧氏 TSP 可由 branch-and-cut 精确求解到数千城市。
2. 子环消除约束（割平面）是 TSP 精确求解的核心；MTZ 是另一种消环写法。
3. Christofides 在三角不等式下给 1.5 近似。
4. 启发式（2-opt、Lin–Kernighan）用于大实例。

局限与延伸

- TSP 是”理想回路”；现实外卖有时间窗、载重、多车，是 VRP 及其变种，难度更高。
- 延伸阅读：Applegate 等《The Traveling Salesman Problem》；Lin–Kernighan 原始论文；networkx 的 `nx.approximate_tsp`。

7. 思考题

基础档

1. 4 个城市的 TSP 有多少条不同回路？为什么是 $(n-1)!/2$？
2. MTZ 约束里 nx_{ij} 项起什么作用？去掉它会怎样？

进阶档

3. 三角不等式不满足时（飞机比火车快但贵），Christofides 的 1.5 近似比还成立吗？

4. 用割平面思想说明：为什么”发现子环才加约束”比”枚举所有子集加约束”高效?

开放档

5. 外卖 VRP 有时间窗和多车约束。直接套 TSP 求解器会在哪失效？该引入哪些额外决策变量?
6. 学习增广（learning-to-augment，第 13 讲）如何用神经网络预测”该优先切哪个子环”？这相比手工割平面好在哪?

第 10 讲 启发式的两个家族

> **本讲问题**：当 TSP 有 10000 个城市、MIP 求解器跑一夜也到不了最优时，工程上怎么办？答案是启发式（heuristic）——不追求最优，只追求”够好、够快”。启发式分两个家族：**构造式**从空解长出一个完整解；**改进式**从一个解出发反复局部优化。再往上是元启发式（模拟退火、遗传算法），它们是跳出局部最优的通用框架。

1. 数学工具/预备知识

- **构造启发式（constructive heuristic）**：一步一步拼出完整解，如最近邻、贪心、节约算法。
- **改进启发式（improvement heuristic）**：给定一个完整解，在邻域里找更优解，如 2-opt、局部搜索。
- **邻域（neighborhood）**：解 x 的邻居 $N(x)$，通常是做一次小改动能到达的解集合。
- **局部最优**：x^* 不被任何 $x' \in N(x^*)$ 改进。
- **元启发式（metaheuristic）**：跳出局部最优的通用框架——模拟退火（SA）、遗传算法（GA）、禁忌搜索、蚁群。
- **近似比（approximation ratio）**：启发式解与最优解的比值上界（如 Christofides 的 1.5），是有保证的启发式；否则叫”纯启发式”，没有理论保证。

局部最优为什么会卡住

改进启发式本质是”瞎子下山”：只要邻域里有更低的点就走过去。问题是它停在的”盆地”可能只是局部小坑，远处山脉另一边有更深的谷。元启发式的全部意义，就是”允许偶尔上山”，翻过山脊看到更深的谷。

2. 完整推导

2.1 构造式：以最近邻为例

从一个城市出发，每次走最近的未访问城市。时间 $O(n^2)$，但解通常次优 20%~30%——因为”每步贪最近”可能早早把自己逼进死角，最后被迫走一段长路回来。

节约算法（Clarke–Wright，用于 VRP）：从每个城市单独一条路线开始，计算合并两条路线的”节约” $s_{ij} = d_{i0} + d_{j0} - d_{ij}$（$i, j$ 直接相连比各自回仓库省多少），从大到小合并路线。它比最近邻系统，因为它显式比较”合并收益”。

2.2 改进式：以 2-opt 为例

给定一个回路，选两条边 $(i, i+1)$ 和 $(j, j+1)$，把它们换成 (i, j) 和 $(i+1, j+1)$（即把 $i+1$ 到 j 这一段反转）。反复扫描所有 (i, j) 对，只要有反转能缩短回路就执行，直到没有任何反转改进——此时停在局部最优。

2.3 局部最优的问题

局部搜索卡在局部最优。想跳出去，要么换邻域（3-opt、Lin–Kernighan），要么用元启发式允许接受差解。

2.4 模拟退火（Simulated Annealing, SA）

灵感来自金属退火：高温时原子乱跑，慢慢降温，原子逐步落入低能态。

```
T = T0
x = 初始解
while T > T_min:
```

```
x' = 邻域随机扰动(x)
delta = f(x') - f(x)
if delta < 0:                    # 更优，直接接受
    x = x'
else:
    p = exp(-delta / T)          # 更差，以概率 p 接受
    if random() < p: x = x'
T = cooling(T)
```

温度 T 高时几乎什么都接受（乱跳、探索）；温度低时只接受更优解（贪心、收敛）。关键在冷却计划。

2.5 为什么 SA 能收敛到全局最优

接受更差解的概率是 $e^{-\Delta/T}$：

- T 大：$e^{-\Delta/T} \approx 1$，几乎全接受，相当于随机游走；
- T 小：$e^{-\Delta/T} \approx 0$，只接受改进，变成贪心下降。

理论上，如果冷却足够慢（$T_k \sim c/\log k$），SA 以概率 1 收敛到全局最优——但那需要极长时间。工程上用指数降温 $T_{k+1} = \alpha T_k$（$\alpha \approx 0.95$），牺牲收敛保证换速度。

2.6 遗传算法（Genetic Algorithm, GA）

维护一个种群，每个个体是一个解的编码（如 TSP 里城市排列）。

- **选择**：按适应度（目标值）选父母；
- **交叉**：两个父母交换片段产生孩子（如 OX 交叉）；
- **变异**：随机翻转/交换；
- **代际更新**：孩子替换种群。

GA 是并行搜索，靠种群多样性避免局部最优。它适合解空间崎岖、目标不可微的组合问题。

与 AI/大模型的联系

元启发式是大模型架构搜索（NAS）的祖先：早期 NAS 用遗传算法搜层间连接。强化学习做 NAS 是后来的事，但”搜索 + 局部改进”的元启发式框架，是理解现代 AutoML 的起点。值得注意的是，学习增广（第 13 讲）正是用神经网络替代表发式的”邻域/交叉”设计。

3. 直觉理解

直觉：把优化想象成在一片山脉里找最低点。局部搜索是瞎子下山——每次走一步往下走，直到走到一个盆地就停了，但这个盆地可能只是小坑，远处还有更深的谷。模拟退火是允许你偶尔上山——不是因为你傻，而是因为上山可能翻过山脊看到另一边更深的谷。温度高时你大胆翻山，温度低时你只下小坡。遗传算法是派一群瞎子在山脉各处走，互相交换笔记，慢慢往深谷汇聚。

4. 常见误区

误区一：以为元启发式比精确算法”高级”。元启发式没有最优性保证，小规模问题上常输给 branch-and-cut。它的用武之地是大规模、结构复杂、精确求解器算不动的场景。

误区二：把 SA/GA 当即插即用黑盒。邻域设计、冷却计划、交叉算子都要针对问题调参，调不好就退化成随机搜索。

误区三：混淆”近似算法”与”启发式”。近似算法有可证明的近似比（如 Christofides 1.5）；纯启发式只在经验上好用，没有保证。论文里要分清。

5. 代码实现

用 numpy 手写模拟退火解 TSP 小例子，并与最近邻对比。

```python
import numpy as np

def total_dist(tour, C):
    return sum(C[tour[i], tour[(i+1)%len(tour)]] for i in range(len(tour)))

def sa_tsp(C, T0=100.0, T_min=0.01, alpha=0.995, steps_per_T=200):
    n = len(C)
    tour = list(range(n)); np.random.shuffle(tour)   # 随机初始回路
    best = tour[:]; best_len = total_dist(best, C)
    T = T0; cur_len = total_dist(tour, C)
    while T > T_min:
        for _ in range(steps_per_T):
            i, j = sorted(np.random.choice(range(n), 2, replace=False))
            new = tour[:i] + tour[i:j+1][::-1] + tour[j+1:]   # 反转一段（2-opt 邻域）
            new_len = total_dist(new, C)
            delta = new_len - cur_len
            # 更优直接接受；更差以 exp(-delta/T) 概率接受
            if delta < 0 or np.random.rand() < np.exp(-delta/T):
                tour, cur_len = new, new_len
                if cur_len < best_len:
                    best, best_len = tour[:], cur_len
        T *= alpha                          # 指数降温
    return best, best_len

# 生成 20 个随机城市
np.random.seed(0)
coords = np.random.rand(20, 2) * 100
C = np.linalg.norm(coords[:,None,:]-coords[None,:,:], axis=-1)

tour, L = sa_tsp(C)
print("SA 路线长度 =", round(L, 2))

# 对比：纯随机重复 200 次取最好
rand_best = min(total_dist(np.random.permutation(20), C) for _ in range(200))
print(" 纯随机最好 =", round(rand_best, 2), "（SA 应明显优于它）")
```

预期 SA 长度明显小于纯随机最好（SA 利用了邻域与降温，而非瞎猜）。

6. 小结

1. 构造启发式快但次优；改进启发式从解出发局部优化。
2. 局部搜索会卡局部最优，元启发式通过”接受差解”跳出。
3. 模拟退火以概率 $e^{-\Delta/T}$ 接受差解，温度控制探索强度。
4. 近似算法有保证，纯启发式没有；大规模才用元启发式。

局限与延伸

- 元启发式结果对随机种子、邻域、超参敏感，可复现性差；工程上要多次重启取最好。
- 延伸阅读：Aarts–Lenstra《Local Search in Combinatorial Optimization》；模拟退火原始论文 Kirkpatrick (1983)；现代 LKH（Lin–Kernighan–Helmgaon）是 TSP 启发式的巅峰。

7. 思考题

基础档

1. 把 SA 的接受概率改成”只接受更优解”，它退化成什么算法?
2. TSP 邻域改成”随机交换两个城市”，和 2-opt 反转一段，哪个邻域更大、搜索更充分?

进阶档

3. 为什么 SA 的冷却不能太快？太快（$\alpha = 0.9$）会发生什么？用接受概率公式解释。
4. GA 的交叉算子对 TSP（排列编码）有什么特殊要求？为什么不能像实数向量那样随便交换？

开放档

5. 用神经网络预测”该反转哪一段”（学习引导的局部搜索），相比固定的 2-opt 邻域好在哪？可能又有什么风险？
6. 当解空间巨大且目标评估昂贵（如训练一个大模型才算一个适应度），启发式的哪个环节最贵？如何用代理模型（surrogate）降本？

第 11 讲 在线算法：在信息不完整时做决策

> **本讲问题**：你今天要打车还是坐地铁？打车 50 块、地铁 5 块，但打车可能堵成 1 小时。问题是：你出门前不知道堵不堵。等你上车发现堵，已经晚了。这就是**在线问题**——决策时未来信息未知，只能一边看一边走。离线算法假设信息全知道，在线算法要在信息不全的情况下保证”别比知道答案的人差太多”。

1. 数学工具/预备知识

- **在线问题（online problem）**：输入是序列，一个一个来，每来一个要立刻决策，不能反悔。
- **竞争比（competitive ratio）**：算法 A 的代价 C_A 与离线最优代价 C_{OPT} 的比值。我们要找最坏情况下 sup 这个比值尽可能小的算法。
- **出租问题（ski rental，滑雪租买）**：滑雪一次 1 元租，买 10 元；不知道会滑几天。
- **paging 问题**：内存缓存的页面替换策略（LRU）。
- **对手模型（adversary）**：竞争比的最坏输入由”对手”构造——它知道你的算法，故意给你最难的序列。

竞争比的严格定义

对最小化问题，在线算法 A 有竞争比 c，是指存在常数 α 使得对所有输入序列 σ：

$$C_A(\sigma) \le c \cdot C_{OPT}(\sigma) + \alpha.$$

$\alpha = 0$ 是严格竞争比。直觉：不管对手怎么安排未来，你付出的代价不超过”提前知道全部未来的人”的 c 倍。c 越小越好，$c = 1$ 意味着在线与离线一样好（罕见）。

2. 完整推导

2.1 出租问题（滑雪租买）

你不知道会滑雪几次。两个选择：每次租 1 元；直接买 10 元。

如果知道滑 k 次：$k < 10$ 租划算，$k \ge 10$ 买划算。这是离线最优。

在线算法：前 9 次租，第 10 次买。逐情况算代价：

- 实际滑 $k \le 9$ 次：算法代价 k（一直租），离线最优也是 k，竞争比 1。
- 实际滑 $k = 10$ 次：算法租 9 次（9 元）+ 第 10 次买（10 元）= 19 元；离线最优直接买 10 元，竞争比 $19/10 = 1.9$。
- 实际滑 $k > 10$ 次：算法仍是 19 元（买了之后再滑不要钱），离线最优 10 元，竞争比仍为 $19/10 = 1.9$。

所以这个算法最坏竞争比 1.9（≤2）。

为什么不能更好？ 设你第 t 次买。对手有两种报复方式：

1. 你买完后只让你再滑极少次：离线是”一直租”$t-1$ 元，你花了 $10+(t-1)$ 元租买，比值 $(t+9)/(t-1)$。t 越大，这个比值越接近 1（越不亏）。
2. 你一直拖着不买（t 很大）：对手让你滑很多次，离线直接买 10 元，你一直租花了 k 元，比值 $k/10$ 爆炸。

两种报复权衡：t 越大第一种越好、第二种越糟。取折中 $t = 10$，最坏比值 1.9。这是该问题的最优竞争比——再聪明也突破不了 2。

2.2 竞争比 vs 平均情况

竞争比是**最坏情况**保证，偏保守。LRU 在真实缓存里 miss 率往往只比 Belady 高一点点，但理论竞争比是 k（页框数）。两者不矛盾：竞争比防的是”对手刻意构造序列”，真实请求序列通常没有那么 adversarial。

2.3 分页问题（paging）

内存有 k 个页框，外存很多页。访问序列一页：在内存（hit）免费；不在（miss）要换入，必须踢一页出去。

- **Belady 最优（OPT）**：踢掉”未来最远才会用到的页”。这是离线最优，需要未来信息。
- **LRU（least recently used）**：踢掉最久没用到的。

对 k 个页框，LRU 的竞争比是 k——最坏 miss 数不超过 OPT 的 k 倍。且这是确定性算法的下界：没有确定性算法能比 k 更小。随机化（如 LRU 的随机版）能把竞争比降到 $O(\log k)$，因为对手无法预测你随机踢哪页。

2.4 在线学习与 regret

在线算法的现代变体是**在线学习**：每轮预测、遭受损失、与”事后最好的固定策略”比较。regret = 你累计损失 − 最优固定策略累计损失。目标是让 regret 随轮次 T 亚线性增长（如 $O(\sqrt{T})$），即平均到每轮的劣势趋于零。指数加权（Hedge）是经典算法。

与 AI/大模型的联系

在线学习是大模型推理时的 KV-cache 驱逐策略：请求一个一个来，不知道后面是什么，要决定驱逐哪个 KV 页。这和分页问题同构——竞争比告诉你 LRU 最坏比最优差 k 倍。PagedAttention 就是在线分页策略在 GPU 显存上的应用。推荐、定价、库存这些实时决策，本质都是在线问题。

3. 直觉理解

直觉：在线算法像个盲人摸象——你不能回头，只能根据已经摸到的部分决定下一步。竞争比是在问：“不管大象长什么样，你最坏能比那个提前看过大象的人差多少倍?”滑雪问题里，答案约 2 倍——你不会因为赌错滑几天而破产，但也不会省到极致。这个”别太差”的保证比”找最优”更现实，因为未来本来就不可知。

4. 常见误区

误区一：以为在线算法就是贪心算法。贪心每步选局部最优，但不保证竞争比；在线算法要的是对所有可能输入的最坏情况比值。

误区二：觉得”既然不知道未来，干脆随机吧”。随机化只有在能证明竞争比改善时才有用，否则就是赌博。

误区三：把竞争比当实际性能。竞争比是对抗性最坏情况，真实系统平均可能好得多；但设计时不能赌”对手不针对我”。

5. 代码实现

模拟出租问题，比较不同在线策略的竞争比。

```python
def ski_rental_alg(days, buy_at):
    """ 前 buy_at-1 次租，第 buy_at 次买，之后免费。"""
    cost = 0
    for d in range(days):
        if d < buy_at - 1:
            cost += 1          # 租
        elif d == buy_at - 1:
            cost += 10         # 买
        # 买了之后再滑不要钱
    return cost
```

```
def opt_offline(days, buy_price=10):
    """ 离线最优：知道 days 后，选 min(一直租，直接买)。"""
    return min(days, buy_price)

# 扫描 days 1..30，比较不同 buy_at 的最坏竞争比
for buy_at in [5, 10, 15]:
    ratios = []
    for days in range(1, 31):
        c = ski_rental_alg(days, buy_at)
        o = opt_offline(days)
        ratios.append(c / o)
    print(f"buy_at={buy_at}: 最坏竞争比 = {max(ratios):.2f}")
```

预期：buy_at=10 时最坏比值 1.9（滑超过 10 天时，算法花 19、最优花 10）；buy_at=5 和 15 的最坏比值都更差，验证 $t = 10$ 最优。

再模拟 LRU 与 OPT 在一个对抗序列上的 miss 数。

```
def paging_miss(pages, cache_size, policy="LRU"):
    cache = []; misses = 0
    for p in pages:
        if p in cache:
            continue
        misses += 1
        if len(cache) < cache_size:
            cache.append(p)
        else:
            if policy == "LRU":
                cache.pop(0)          # 踢最久未用（队首）
            cache.append(p)
    return misses

seq = [1,2,3,1,4,1,2,5,2,1,4,5]
print(" 序列:", seq)
print("LRU miss (k=3):", paging_miss(seq, 3, "LRU"))
print(" 若 Belady：踢未来最远用的页，miss 更少")
```

6. 小结

1. 在线问题：输入逐次到达，决策不可反悔。
2. 竞争比 = 算法代价 / 离线最优代价的最坏情况比值。
3. 滑雪问题最优竞争比 ≈ 2；分页 LRU 竞争比 $= k$。
4. 随机化在线算法能突破确定性下界；在线学习用 regret 框架。

局限与延伸

- 竞争比是最坏保证，可能过于悲观；实践中常结合平均情形分析。
- 延伸阅读：Borodin–El-Yaniv《Online Computation》；滑雪/分页经典 competitive analysis；在线学习见 Cesa-Bianchi–Lugosi《Prediction, Learning, and Games》。

7. 思考题

基础档

1. 如果买价从 10 改成 20，最优 buy_at 应该是多少？为什么？
2. 为什么 LRU 在实践中比竞争比 k 表现好？竞争比是哪种极端输入下的结论？

进阶档

3. 设计随机化滑雪算法（以一定概率每轮决定买），说明为什么它能突破确定性的 2 竞争比。
4. 写出 regret 框架下，你每轮预测一个价格、遭受 $(\quad - \quad)^2$ 损失时，指数加权算法的直觉。

开放档

5. 大模型推理的 KV-cache 驱逐是在线分页问题。真实请求序列有时间局部性（刚用过的还会用），LRU 竞争比 k 这个悲观结论在这种局部性下还有意义吗?
6. 当”对手”不是刻意的、只是自然分布偏移时，竞争比和鲁棒优化（第 12 讲）哪个更合适?

第 12 讲 鲁棒优化：对不确定性做最坏打算

> **本讲问题**：面包房老板按第 1 讲的最优解烤了 92 炉面包，结果今天客流比预期少 30%，剩下的面包只能打折处理。期望利润算得再好，一次意外就吃掉半年利润。鲁棒优化（Robust Optimization，RO）的思路不是”赌期望”，而是”对最坏情况做打算”——不管不确定性在什么范围内实现，我的解都能行得通。

1. 数学工具/预备知识

- **不确定参数**：$\tilde{a}$、$\tilde{b}$ 属于某个**不确定集** $\mathcal{U}$。
- **鲁棒解**：对所有 $\xi \in \mathcal{U}$ 都可行的解 x。
- **鲁棒优化问题**：

$$\max_{x} \min_{\xi \in \mathcal{U}} f(x,\xi) \quad \text{s.t.} \quad g(x,\xi) \le 0,\ \forall \xi \in \mathcal{U}.$$

- **盒子不确定集**：$\{|a_j - \bar{a}_j| \le u_j\}$，每个参数独立偏离。
- **椭球不确定集**：$\{(a-\bar{a})^\top \Sigma^{-1}(a-\bar{a}) \le \Omega^2\}$。
- **基数约束不确定集**（Bertsimas–Sim）：$\sum_j |\xi_j| \le \Gamma,\ |\xi_j| \le 1$。
- **随机规划**：假设参数有已知分布，优化期望或 CVaR——与鲁棒优化对照。

为什么需要”最坏情况”

期望优化只保证平均好，但风险厌恶的决策者不能接受”小概率破产”。鲁棒优化把不确定集内**每一种**实现都当成必须满足的约束，牺牲一点平均性能，换掉”出事”的可能。

2. 完整推导

2.1 从 LP 到鲁棒 LP

标准 LP:

$$\max c^\top x \quad \text{s.t.} \quad Ax \le b,\ x \ge 0.$$

假设 A 的元素有不确定性，实际 $A(\xi) = \bar{A} + \Delta A \cdot \mathrm{diag}(\xi)$，其中 $\xi \in [-1,1]^m$。鲁棒约束要求：对所有 $\xi \in [-1,1]^m$,

$$(\bar{A} + \Delta A \cdot \mathrm{diag}(\xi))x \le b.$$

2.2 盒子不确定集的鲁棒对偶

对第 i 条约束，最坏情况是什么? ξ 怎么选会让左边最大?

$$\max_{\xi \in [-1,1]^m} \sum_j (\bar{a}_{ij} + \Delta a_{ij}\xi_j)x_j = \sum_j \bar{a}_{ij}x_j + \max_{\xi} \sum_j \Delta a_{ij}\xi_j x_j.$$

第二项里 $\xi_j x_j$ 的最大值在 $\xi_j = \text{sign}(x_j)$ 时取到（$\xi_j \in [-1,1]$），贡献 $\Delta a_{ij}|x_j|$。所以最坏左边是

$$\sum_j \bar{a}_{ij} x_j + \sum_j \Delta a_{ij} |x_j| \le b_i.$$

若 $x_j \ge 0$（面包房例子），$|x_j| = x_j$，则每条约束都用”最坏的系数”：

$$\sum_j (\bar{a}_{ij} + \Delta a_{ij}) x_j \le b_i.$$

这相当于把所有不确定项同时放大到边界——最保守的鲁棒解。

2.3 为什么盒子鲁棒太保守

一条约束若有 10 个系数，每个都拉到最坏，等于假设所有系数**同时**最坏。这个联合发生的概率极低，所以可行域被切得过小、目标值很差。盒子鲁棒是”防御过当”。

2.4 基数约束不确定集（Bertsimas–Sim）

改进：不是所有系数都能同时偏离最坏，最多 Γ 个。即 ξ 满足 $\sum_j |\xi_j| \le \Gamma$，$|\xi_j| \le 1$。最坏情况是选 Γ 个影响最大的系数拉满、其余不动。这通过 LP（辅助变量）可解，且 Γ 是可调旋钮：

- $\Gamma = 0$：原确定 LP；
- $\Gamma = m$：退回盒子鲁棒；
- Γ 中等：折中，且能保证”破坏概率随 Γ 增加而平滑上升”。

2.5 椭球不确定集

$$\mathcal{U} = \{a = \bar{a} + Pu : \|u\|_2 \le \Omega\}.$$

约束 $a^\top x \le b$ 鲁棒化为：

$$\bar{a}^\top x + \Omega \|P^\top x\|_2 \le b.$$

这是一个二**阶锥约束（SOCP）**，凸可解。椭球对应”参数服从高斯分布、置信椭球”，Ω 是置信度（$\Omega = 2$ 约 95% 置信）。它比盒子不保守，因为允许参数相关地偏离。

2.6 鲁棒 vs 随机规划

- **鲁棒优化**：不确定集外的参数不管，只求集内全可行。不要求概率分布。
- **随机规划**：假设参数有已知分布，优化期望目标或 CVaR（条件风险价值，即”最坏 5% 情形的平均损失”）。

鲁棒适合”分布不知道、但不能出事”的场景；随机规划适合”分布能估、要平均好”的场景。CVaR 是两者的桥梁：它既像鲁棒（盯住尾部），又用分布加权。

与 AI/大模型的联系

鲁棒优化直接对应 AI 的分布外泛化（OOD）：训练分布内损失低，不代表测试分布偏移时还稳。鲁棒优化的”不确定集”就是 OOD 风险集——在这个集合内所有可能偏移下，模型性能都不能崩。大模型对齐（RLHF + 红队测试）本质是人为构造一个不确定集，验证模型在这个集合内不输出有害内容。

3. 直觉理解

直觉：鲁棒优化像出门带伞——不管天气预报说下雨概率 10% 还是 90%，你都带伞。代价是可能多背个伞；收益是真下雨时你不狼狈。不确定集 $\mathcal{U}$ 定义”什么样的天气算可能”，你把所有可能天气都考虑到。盒子不确定集是”任何天气都带伞”，太保守；基数约束是”只防最可能的几种坏天气”，更实用。

4. 常见误区

误区一：以为鲁棒解一定比期望解差。事实上，鲁棒解在最坏情况下保住可行，而期望解可能在小概率事件下直接崩掉。工程上看下行风险，不是看平均利润。

误区二：把鲁棒优化等同于”在最坏点优化”。它要的是**所有**不确定点下都可行，不是挑一个最坏点。

误区三：不确定集拍脑袋定。集太大 → 过度保守；集太小 → 漏防真实风险。集的形状（盒子/椭球/基数）要结合参数的真实统计关联来选。

5. 代码实现

对比确定 LP、盒子鲁棒解，并检验确定解在最坏系数下是否可行。

```
import numpy as np
from scipy.optimize import linprog

# 面包房例子：面粉约束系数 2 有 ±0.2 不确定性（最坏 2.2）
# 确定 LP
r1 = linprog([-30,-50], A_ub=[[2,1],[1,2]], b_ub=[200,120],
             bounds=[(0,None),(0,None)], method="highs")
print(" 确定 LP: x =", np.round(r1.x,3), " 利润 =", round(-r1.fun,2))

# 盒子鲁棒：面粉系数最坏按 2.2 算，约束变 2.2 x1 + x2 <= 200
r2 = linprog([-30,-50], A_ub=[[2.2,1],[1,2]], b_ub=[200,120],
             bounds=[(0,None),(0,None)], method="highs")
print(" 盒子鲁棒: x =", np.round(r2.x,3), " 利润 =", round(-r2.fun,2))

# 检验：确定 LP 解在实际系数 2.2 下是否还满足面粉约束
x_det = r1.x
used = 2.2*x_det[0] + x_det[1]
print(f" 确定解在系数 2.2 下: 2.2*{x_det[0]:.2f}+{x_det[1]:.2f} = {used:.2f}")
print(" 是否 <= 200? ", used <= 200)
```

预期：确定 LP 解 $(93.33, 13.33)$ 在最坏系数 2.2 下消耗 $2.2 \times 93.33 + 13.33 \approx 218.7 > 200$，**不可行**（违反面粉约束）；盒子鲁棒解则主动把系数按 2.2 设计，保证可行，但利润更低。这演示了”期望解在最坏情况出事”、“鲁棒解牺牲利润换可行”。

6. 小结

1. 鲁棒优化要求解在不确定集内所有参数下都可行。
2. 盒子不确定集最简单但最保守；基数约束、椭球是折中。
3. 鲁棒对偶把”对所有 ξ“的半无限约束转化为凸可解形式（LP / SOCP）。
4. 鲁棒 vs 随机规划：不依赖分布 vs 利用分布；CVaR 是桥梁。

局限与延伸

- 鲁棒解可能过度保守，且不确定集外的真实风险一旦越界就无保护；需要事后情景检验。
- 延伸阅读：Ben-Tal–El Ghaoui–Nemirovski《Robust Optimization》；Bertsimas–Sim (2004) 基数约束原始论文；CVaR 见 Rockafellar–Uryasev。

7. 思考题

基础档

1. 如果面包需求不确定性在 ±20%，用盒子鲁棒该怎么改约束?
2. 基数约束参数 Γ 取 0、m、$m/2$ 分别对应什么策略?

进阶档

3. 为什么椭球不确定集比盒子不保守？从”参数是否允许相关偏离”角度解释。
4. 鲁棒优化和”压力测试”有什么异同?

开放档

5. 大模型红队测试是在构造”有害输入”的不确定集。传统鲁棒优化的不确定集是连续凸集，但”有害 prompt”是离散文本——这两者的不确定集刻画有何本质不同?
6. 当无法确定分布、也无法确定不确定集边界时，该用鲁棒还是随机规划？为什么这是一个建模哲学问题而非纯技术问题?

第 13 讲 学习增广算法：用数据替代人工规则

> **本讲问题**：运筹求解器靠人写分支规则、割平面、启发式——专家写一条规则调半年。但过去十年，机器学习开始接管这些决策：神经网络看着问题实例，直接预测”该从哪个变量分支”“该加哪条割”“初始解长什么样”。这就是 Learning to Optimize（L2O），把求解器里的人工经验换成学习模型。它不是要取代单纯形法，而是在求解器每个”需要人做决策”的节点上，放一个训练好的策略网络。

1. 数学工具/预备知识

1.1 Learning to Optimize (L2O) 的定位

传统运筹求解器（Gurobi、CPLEX）内部有很多”需要人做决策”的节点：分支变量选哪个、割平面加哪条、初始解怎么构造。这些决策过去由专家写规则实现（pseudocost 分支、best-bound 节点选择）。L2O 用机器学习从数据中学这些决策——神经网络看着当前问题实例的特征，直接预测”该选哪个变量分支”。

1.2 神经组合优化（NCO）

NCO 是 L2O 的极端形式：不用传统求解器，神经网络直接输出一个完整解。例如输入 TSP 城市坐标，输出一条访问顺序排列。优点是推理快（一次前向传播），缺点是没有最优性保证——输出可能是次优的。

1.3 Pointer Network

Pointer Network（Vinyals 等 2015）是一种特殊的 seq2seq 模型：decoder 不生成固定词表的 token，而是用 attention 从输入序列中”指针”式地选一个位置。对 TSP, encoder 把每个城市编码为向量 h_i, decoder 每步输出一个 attention 分布在所有城市上，选中的城市就是下一站。输出是一个排列 π。

1.4 REINFORCE 算法

因为 TSP 没有标签解（最优解要跑几小时求解器），不能用监督学习。REINFORCE 是 policy gradient 的最简形式:

$$\nabla_\theta J = \frac{1}{N}\sum_{i=1}^{N}(R_i - b)\nabla_\theta \log p_\theta(\pi_i|x).$$

$R_i = -L(\pi_i|x)$ 是第 i 条 rollout 的 reward（路线长度取负, 越短 reward 越大）。b 是基线（通常是 greedy rollout 的长度），用来降方差。

1.5 热启动（warm start）

ML 预测一个初始解，交给精确求解器从这个起点开始 branch-and-bound。好的初始解能让 B&B 少探索很多节点——因为初始解给了一个上界，很多分支可以直接剪掉。

为什么不能用监督学习

组合优化”没有标准答案”——求一个实例的最优解本身就要跑求解器几小时，不可能收集海量标签。所以训练信号只能来自”解出来之后它好不好”（reward），这天然是强化学习范式，不是监督学习。

2. 完整推导

2.1 L2O 的三种位置

1. **在求解器外围**：ML 预测初始解、热启动点，再交给精确求解器改进。最成熟——不改求解器内部，只给好起点。
2. **在求解器内部**：ML 预测分支变量、选割、定价策略（branching rule、cut selection）。传统 B&B 用 pseudocost 选分支变量，ML 学”在这种 LP 松弛解下，哪个变量分支最能剪枝”。
3. **端到端输出解**：ML 直接输出完整解，求解器做后处理。快但没有最优性保证。

2.2 Pointer Network 解 TSP

输入 n 个城市坐标 $x_1, \dots, x_n \in \mathbb{R}^2$。Encoder 把每个城市编码为向量 h_i。Decoder 从起点开始，每步输出一个”指针”指向下一个访问的城市：

$$a_t = \text{softmax}\big(\text{score}(h_t, H)\big).$$

score 函数通常是点积或 additive attention。输出是一个排列 $\pi = (\pi_1, \dots, \pi_n)$，代表访问顺序。

训练：TSP 没有标签”正确解”，但有评价指标——路线长度 $L(\pi|x) = \sum_i d(\pi_i, \pi_{i+1})$。把它当 reward，用 REINFORCE：

$$\nabla J(\theta) = \mathbb{E}_{x\sim P}\Big[\big(L(\pi_\theta|x) - b(x)\big)\nabla_\theta \log p_\theta(\pi_\theta|x)\Big].$$

（最小化长度，所以 reward 取 $-L$，这里写成 $L - b$ 是损失视角。）$b(x)$ 是基线（greedy rollout 长度），作用：当前 episode 比基线好就强化该策略，比基线差就弱化。没有基线时，所有 episode 同向更新，方差巨大；减去基线相当于只保留”相对好坏”的信号。

为什么用 REINFORCE 而不是更高级的 PPO? 因为组合优化的 reward 是确定性的（给定实例和解，路线长度是确定的），没有随机环境，REINFORCE 够用。PPO 的 clip 机制主要对付连续控制里的策略震荡，在组合优化里收益不大。

2.3 为什么 L2O 有用

- 精确求解器在实例分布固定时，重复做同样的搜索工作。学习模型记住”这种分布下的好解模式”，能省大量分支。
- 同一类城市布局（如欧洲小镇 TSP）最优解结构相似，ML 记住结构就不用每次从零搜索。
- 但 L2O 不替代精确求解——输出的解通常要经 2-opt 改进，或作为 MIP 热启动。

核心经济逻辑：精确求解器的瓶颈是”每个新实例都从零开始搜索”。如果你的业务要每天解 10000 个相似实例（如物流路径规划），每个实例都跑 1 小时 branch-and-cut 是不可接受的。ML 学一次”这种分布下的好解长什么样”，之后每个实例推理 10 毫秒就出解——省下来的时间就是 L2O 的价值。

2.4 典型工业 pipeline

ML 预测初始解 → 局部搜索改进 → MIP 求解器从这个起点 branch-and-bound → 输出最优解 + gap。ML 负责”省搜索时间”，MIP 负责”保证最优性”。这是目前落地最稳的范式：**用学习替代搜索的前半截，用求解器兜底后半截**。

为什么这个 pipeline 最稳？因为它把 ML 的不确定性限制在”初始解质量”上——即使 ML 给的初始解很差，MIP 也能从差起点开始搜索，只是慢一点。如果是端到端 ML 直接输出解，ML 错了就是错了，没有兜底。工业界最怕”静默失败”——所以混合 pipeline 比纯 ML 更受欢迎。

2.5 局限

- **泛化**：训练 20 城市、测试 50 城市性能骤降（位置泛化）。attention 对顺序敏感，城市数变了要重做位置编码。缓解方法：用 GNN 代替 RNN（GNN 对节点数不敏感），或做数据增强。
- **没有最优性保证**：输出可能不满足约束。工程上要用求解器后处理修复不可行解。
- **数据成本**：要生成几十万合成实例。生成 TSP 实例很便宜（随机撒点就行），但如果要学一个工业级调度问题，生成数据本身就是工程。
- **推理成本**：每个实例跑一次 GNN，比贪心慢。小规模问题上，ML 推理开销可能超过它省的搜索时间。

3. 直觉理解

> **直觉**：传统求解器像个新手顾问，每个新实例都从零分析。L2O 是个熟手——见过成千上万类似实例，一眼看出”这种结构该往哪切一刀”。神经网络把专家写规则的过程，变成从数据总结规律。但熟手也会翻车，所以工程上是”ML 给初值 + 求解器严谨求优”的混合——像老司机靠直觉并道，靠导航算最优路线。

这个混合范式的哲学：ML 擅长模式识别（这种实例大概长什么样），OR 擅长精确推理（这个具体实例的最优解是什么）。把两者拼起来——ML 给方向，OR 给精度——比单独用任何一个都强。

4. 常见误区

> **误区一：以为 L2O 会取代 MIP 求解器**。它只是加速组件，不能替代最优性证明。
>
> **误区二：把 L2O 当监督学习**。组合优化没有标签解，主要靠 RL 训练——收集”最优解”当标签的成本等于求解本身。
>
> **误区三：以为 ML 输出一定比人工启发式好**。小数据上，最近邻、savings 等人工规则往往更强；ML 优势在大规模、分布稳定的重复场景。

5. 代码实现

演示”ML 预测初始解 → 局部搜索改进”的接口位置：用贪心最近邻当初始解（模拟 ML 热启动），再用 2-opt 改进。

```
import numpy as np

def tour_length(tour, coords):
    """ 计算 TSP 路线总长度（欧氏距离）。"""
    n = len(tour)
    return sum(np.linalg.norm(coords[tour[i]] - coords[tour[(i+1)%n]])
               for i in range(n))

def greedy_tour(coords):
    """ 最近邻贪心：模拟一个（粗糙的）ML 热启动解。
    真实 L2O 里，这个" 初始解" 是 GNN 预测出来的；
    这里用最近邻代替，因为它足够简单且能说明接口位置。"""
    n = len(coords); visited = [0]; cur = 0
    for _ in range(n-1):
        # 从未访问城市中选距离当前最近的
```

```
        nxt = min((j for j in range(n) if j not in visited),
                  key=lambda j: np.linalg.norm(coords[cur]-coords[j]))
        visited.append(nxt); cur = nxt
    return visited

def improve(tour, coords):
    """ 确定性 2-opt：反复扫描直到无改进（模拟求解器对热启动解的精修）。
    每轮尝试所有 O(n^2) 个 2-opt 反转，选改进最大的执行，直到无改进。"""
    best = tour[:]; best_len = tour_length(best, coords)
    n = len(best); improved = True
    while improved:
        improved = False
        for i in range(1, n-1):
            for j in range(i+1, n):
                new = best[:i] + best[i:j+1][::-1] + best[j+1:]
                L = tour_length(new, coords)
                if L < best_len - 1e-9:  # 有实质改进才更新
                    best, best_len = new, L; improved = True
    return best, best_len

np.random.seed(0)
coords = np.random.rand(10, 2) * 100    # 10 个随机城市
tour = greedy_tour(coords)               # ML 热启动：贪心最近邻
print(" 热启动（贪心）长度 =", round(tour_length(tour, coords), 3))
opt, L = improve(tour, coords)           # 求解器精修：2-opt
print("2-opt 精修后长度 =", round(L, 3))
```

预期 2-opt 精修后长度明显小于贪心热启动——这正是”ML 热启动 + 求解器精修”省时间的效果。

6. 小结

1. L2O 用 ML 替代求解器的人工决策组件（外围、内部、端到端三种位置）。
2. 神经组合优化用指针网络 + REINFORCE 端到端学解，基线降方差。
3. L2O 输出要经精确求解器后处理，不能单独交付。
4. 泛化性与最优性保证是核心挑战。

局限与延伸

- 学习增广的可靠性依赖实例分布稳定；分布漂移时模型可能给出更差热启动。
- 延伸阅读：Bello 等《Neural Combinatorial Optimization with Reinforcement Learning》(2016)；Khalil 等学习 branch variable；warm-start 与 MIP 结合的 Gasse 等 (2019)。

补充一个实操建议：L2O 项目从”ML 热启动 + MIP 精修”这个最稳的 pipeline 开始，不要一上来就做端到端。热启动只需要 ML 输出一个可行解，容错率高——解差一点没关系，MIP 会从起点继续优化。端到端要求 ML 直接输出高质量解，容错率低，调试也难。

7. 思考题

基础档

1. 为什么 TSP 不能直接用监督学习（没有标准答案）？
2. REINFORCE 里的基线 $b(x)$ 起什么作用？去掉它会怎样？

进阶档

3. 训练集都是 20 城市、测试 50 城市，性能为何下降？怎么缓解（提示：排列不变性、平移不变性）？
4. “ML 热启动 + MIP 精修”相比纯 2-opt，优势在哪？在什么规模下这个优势最明显？

开放档

5. 当实例分布未知且不断漂移时，固定训练的 L2O 热启动会不会反而比贪心还差？该怎么在线适应？

6. 如果求解器本身很快（小 LP），L2O 热启动还有意义吗？什么规模下”学习省时间”的收益才超过”推理开销”？

第 14 讲 强化学习求解组合优化：从构造到改进

本讲问题：上一讲 Pointer Network 像”一步一步构造解”——先选第一站、再选第二站……适合 TSP。但如果初始解已经不错，想”改进它”而不是从头构造呢？强化学习也能做这件事：把每一次局部改动当一个动作，reward 是改完后目标值的改善。本讲看 RL 在组合优化里的两种姿势——构造式和改进式。

1. 数学工具/预备知识

1.1 构造式 RL 与改进式 RL

- **构造式 RL**：策略 $\pi(a_t|s_t)$ 逐步加决策，从空解到完整解。状态是”已选了哪些城市”，动作是”下一个选哪个”。输出是一条完整路线。
- **改进式 RL**：给定初始解，RL agent 在邻域里选动作（2-opt 交换、移动一个任务），改到不能改。状态是”当前解长什么样”，动作是”做哪种邻域变换”。输出是改进后的解。

1.2 策略梯度（policy gradient）

策略梯度方法直接参数化策略 $\pi_\theta(a|s)$，然后最大化期望 reward：

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\big[R(\tau)\big], \quad \nabla_\theta J = \mathbb{E}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)\cdot A_t\Big].$$

其中 $\tau = (s_1, a_1, s_2, a_2, \dots)$ 是一条 trajectory，$R(\tau)$ 是这条轨迹的总 reward，A_t 是 advantage（第 t 步动作的相对好坏）。直觉：让”导致好结果的动作”概率增大，“导致坏结果的动作”概率减小。

1.3 Actor-Critic 与优势函数

Actor-Critic 架构有两个网络：

- **Actor**：$\pi_\theta(a|s)$，输出动作概率。
- **Critic**：$V_\phi(s)$，估计状态价值（从 s 出发的期望总 reward）。

优势函数 $A_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$，衡量”实际走了 a_t 比平均水平好多少”。如果 $A_t > 0$，说明这个动作比平均好，增大其概率；$A_t < 0$ 则减小。γ 是折扣因子，权衡短期和长期 reward。

1.4 基线（baseline）的作用

基线 $b(s)$ 是一个不依赖动作的基准值，从 reward 中减掉它：

$$\nabla J \approx \mathbb{E}\big[(R - b(s))\nabla_\theta \log \pi_\theta(a|s)\big].$$

减 b 不改变梯度的期望（因为 $\mathbb{E}[\nabla_\theta \log \pi] = 0$，所以 $\mathbb{E}[b\cdot\nabla_\theta\log\pi] = b\cdot 0 = 0$），但大幅降低方差。直觉：没有基线时，所有 reward 都是正数（路线长度取负后），梯度总是同方向更新，不管这次 rollout 是好是坏。减基线后，比基线好的 rollout 才正向强化，比基线差的负向弱化——信号清晰了。

从贝尔曼方程到 policy gradient

第 6 讲的贝尔曼方程 $V(s) = \min_a\{c(s,a) + \gamma V(s')\}$ 是”精确 DP”。当状态太大、存不下整张表时，用参数化策略 $\pi_\theta(a|s)$ 直接输出动作，再用 policy gradient 调 θ。梯度用 reward 的”相对好坏”加权 $\nabla\log\pi_\theta$——这就是组合优化 RL 的根基。

2. 完整推导

2.1 构造式：RL 当解码器

状态 s_t = 已选序列 $\pi_{1:t}$ + 问题实例 x。动作 a_t = 选下一个城市。转移 $\pi_{1:t+1} = \pi_{1:t} \cup \{a_t\}$。Reward 在最后一步：$R = -L(\pi|x)$（路线长度取负）。

Policy gradient：

$$\nabla_\theta J = \mathbb{E}_{x\sim P,\ \pi\sim\pi_\theta}\big[-L(\pi|x)\nabla_\theta \log \pi_\theta(\pi|x)\big].$$

直觉：这次 rollout 路线越短（$-L$ 越大），越增大导致该路线的动作概率。但单次 rollout 噪声大，加基线 $b(x)$：

$$\nabla_\theta J \approx \frac{1}{N}\sum_{i=1}^{N}\big[-L(\pi_i|x) - b(x)\big]\nabla_\theta \log \pi_\theta(\pi_i|x).$$

$b(x)$ 通常是 greedy rollout 长度或历史平均。它不改变梯度的期望（因为 $\mathbb{E}[\nabla \log \pi] = 0$，减 b 不引入偏差），但大幅降低方差。

为什么减 b 不引入偏差?数学上:$\mathbb{E}[\nabla_\theta \log \pi_\theta(a|s)] = 0$(因为 $\sum_a \pi(a|s) = 1$,对 θ 求导得 $\sum_a \nabla_\theta \pi(a|s) = 0$，而 $\nabla_\theta \log \pi = \nabla_\theta \pi/\pi$，所以 $\mathbb{E}[\nabla_\theta \log \pi] = \sum_a \pi(a|s)\cdot\nabla_\theta \pi(a|s)/\pi(a|s) = \sum_a \nabla_\theta \pi(a|s) = 0$)。因此 $\mathbb{E}[b\cdot\nabla_\theta \log \pi] = b\cdot 0 = 0$，减 b 不改变梯度期望。

2.2 改进式：RL 当局部搜索引导

状态 s_t = 当前解 x_t。动作 a_t = 邻域变换（选一对城市做 2-opt）。转移 $x_{t+1} = T(x_t, a_t)$。Reward：$r_t = f(x_t) - f(x_{t+1})$（改进量）。

这种方式把”局部搜索何时接受什么改动”交给 RL——它学会识别”哪种改动在哪个实例上有用”。欧氏 TSP 上，反转相邻城市段通常比交换远距离城市有效；RL 会学到优先选相邻对。改进式 RL 也可看成”学习一个更好的 SA 调度”——SA 是固定温度表，RL 按状态自适应决定要不要接受差解。

2.3 为什么 RL 比纯启发式好

纯启发式（2-opt）“改进就接受”，不会跳过暂时变差的步。RL 学到”接受一个略差的步，后面能省更多”——SA 也在做，但 RL 从数据学这个权衡，不手调温度。更深层：RL 能根据实例特征（城市密度、边权分布）调整策略，而纯启发式对所有实例一视同仁。

具体来说，2-opt 是贪心的：只有 $\Delta > 0$ 才接受。这意味着它一旦走到局部最优就出不来。模拟退火（SA）以一定概率接受 $\Delta < 0$ 的步，跳出局部最优，但接受概率由温度表决定——温度是手调的。RL 的优势：它从大量实例中学到”在什么状态下、接受多大的退步是值得的”，这个策略是数据驱动的，不用手调温度。

2.4 RL 训练的难点

- **稀疏 reward**：TSP reward 只在最后一步给出（路线总长度），中间每步没有反馈。要 baseline 降方差——没有 baseline 时，policy gradient 的方差正比于 reward 波动幅度。
- **实例分布固定**：换分布（20 城市变 50 城市）策略要重训。因为策略网络见过的是 20 城的布局模式，50 城的布局模式不同。
- **仿真成本**：每个 episode 构造/改进一条解，慢。构造式 RL 解一个 20 城 TSP 要 20 步解码，10 万 episodes 就是 200 万步。
- **信用分配**：最后一步 reward 怎么分配到前面每步动作? policy gradient 靠 $\nabla \log \pi$ 自动分配，但方差大。Actor-Critic 用 critic 估计每步价值，缓解这个问题。

2.5 两种方式对比

维度	构造式 RL	改进式 RL
输入	只有问题实例	初始解 + 实例
输出	完整解	改进后的解
样本效率	低（每 episode 一条解）	中（已有解上迭代）
适用	分布固定、批量出解	有好初始解、想微调

3. 直觉理解

直觉：构造式 RL 像导航——一步一步决定走哪条路，到终点看总里程打分。改进式 RL 像装修——房子已经有了，agent 决定先拆哪面墙、再补哪面墙，每步改完看体验提升多少。前者从零开始，后者在已有解上微调，都靠”最终目标值”当奖励。改进式优势是不需要从零学，利用贪心解当起点，收敛更快。

一个关键区别：构造式 RL 的每一步都是不可逆的——选了城市 3 就不能再改。改进式 RL 的每一步是可逆的——2-opt 换错了可以再换回来。可逆性让改进式 RL 的搜索空间更大，但也更难训练（动作空间是 $O(n^2)$ 的邻域变换）。

4. 常见误区

误区一：把 RL 当”更聪明的启发式”。RL 弱点是样本效率——要训几万 episodes 才收敛，而 SA 一次就能跑。RL 适合”实例分布固定、要解成千上万个相似实例”。

误区二：以为 RL 解 TSP 比精确求解器好。RL 输出是近似解，100 城市 TSP 常比 LKH（Lin–Kernighan 启发式）差。RL 价值在大规模、求解器算不动的场景。

误区三：把改进式 RL 等同于”学习局部搜索”。局部搜索是固定规则，RL 学习何时用什么邻域、是否接受差解，策略本身数据驱动。

5. 代码实现

演示”改进式 RL”的接口：用一个打分函数（此处用真实改进量当 oracle，演示动作选择逻辑）引导 2-opt。

```
import numpy as np

def tour_length(tour, C):
    """ 计算路线长度：C[i,j] 是城市 i 到 j 的距离矩阵。"""
    return sum(C[tour[i], tour[(i+1)%len(tour)]] for i in range(len(tour)))

def improve_oracle(C, init_tour):
    """ 改进式搜索的演示：每步枚举所有 2-opt 动作，选改进最大的。
    真实 RL 中，'选哪个动作'由策略网络打分代替枚举——
    枚举 O(n^2) 个动作每步，n=100 就有 5000 个候选，RL 网络一次前向就选出 top-k。
    """
    tour = init_tour[:]; n = len(tour); cur = tour_length(tour, C)
    while True:
        best_delta, best_move = 0, None
        for i in range(1, n-1):
            for j in range(i+1, n):
                # 2-opt：反转 tour[i+1..j] 这段
                new = tour[:i+1] + tour[i+1:j+1][::-1] + tour[j+1:]
                delta = cur - tour_length(new, C)   # 改进量当 reward
                if delta > best_delta:
                    best_delta, best_move = delta, new
        if best_move is None:
            break                               # 没有动作能改进，停在局部最优
        tour, cur = best_move, tour_length(best_move, C)
    return tour, cur
```

```
np.random.seed(1)
coords = np.random.rand(10,2)*100       # 10 个随机城市坐标
C = np.linalg.norm(coords[:,None,:]-coords[None,:,:], axis=-1)  # 欧氏距离矩阵
init = list(range(10))                   # 初始路线: 0->1->2->...->9
print(" 初始随机路线长度 =", round(tour_length(init, C), 2))
opt, L = improve_oracle(C, init)
print(" 改进式搜索后长度 =", round(L, 2))
```

预期改进后长度远小于初始随机路线。真实 RL 把”枚举所有动作选最大改进”换成”策略网络预测哪个动作好”，以换取大规模下的速度。

6. 小结

1. 构造式 RL：逐步解码完整解，从空到满。
2. 改进式 RL：在邻域里选局部动作，改进已有解。
3. Policy gradient + baseline 是主流训练方式，降方差。
4. RL 适合固定分布下的大规模重复求解，不适合一次性问题。

局限与延伸

- RL 解组合优化的样本效率与泛化仍是开放问题；工业上更多把 RL 当”启发式引导”而非独立求解器。
- 延伸阅读：Khalil 等《Learning to Construct》(2017)；Sutton–Barto 关于 policy gradient；MADQL、DRL for routing 的综述。

补充：改进式 RL 的一个实操难点是动作空间太大。2-opt 有 $O(n^2)$ 个动作，$n = 1000$ 就是 50 万个——策略网络不可能输出 50 万维的 softmax。解决方法：不枚举所有动作，而是用 GNN 给每条边打分，选 top-k 候选做 2-opt。这把”选动作”从分类问题变成了”给边打分”问题，动作空间从 $O(n^2)$ 降到 $O(n)$。

7. 思考题

基础档

1. 构造式 RL 和 Pointer Network 是什么关系?
2. 改进式 RL 的 reward 是每步改进量，这会带来什么偏差（提示：短视)?

进阶档

3. 为什么 policy gradient 要加基线? 从方差公式角度说明减 b 不引入偏差。
4. 改进式 RL 相比纯 2-opt，理论优势是什么？工程优势又是什么?

开放档

5. 如果把初始解换成 LKH 的输出，RL 还能改进多少？这说明什么?
6. 当 reward 稀疏（只在最终）且实例巨大时，怎么用课程学习（从易到难）提高样本效率?

第 15 讲 端到端可微优化：把 LP 嵌入神经网络

> **本讲问题**：传统流程是”先预测需求、再拿预测值跑 LP 求库存”——预测误差通过 LP 传播成决策误差。但预测和优化是两个独立模型，没人告诉 LP”你的输入是不准的”。端到端可微优化把 LP 变成一个神经网络层，让梯度从 LP 反传回来，联合训练预测和决策。这一讲看这个”可微层”怎么构造。

1. 数学工具/预备知识

1.1 可微优化层的概念

普通神经网络层是固定公式（如卷积、激活函数），可微优化层把一个优化问题当成层：输入参数 θ，输出优化问题的最优解 $z^*(\theta) = \arg\min_x f(x, \theta)$。关键是让 z^* 对 θ 可微——这样 loss $L(z^*)$ 的梯度才能穿过这层反传回 θ。

1.2 KKT 条件

对凸优化问题 $\min_x f(x)$ s.t. $g_i(x) \le 0, h_j(x) = 0$，最优解 x^* 必须满足：

1. **平稳性**：$\nabla f(x^*) + \sum_i \lambda_i \nabla g_i(x^*) + \sum_j \mu_j \nabla h_j(x^*) = 0$。
2. **互补松弛**：$\lambda_i g_i(x^*) = 0$。如果约束 i 不活跃（$g_i(x^*) < 0$），则 $\lambda_i = 0$。
3. **原可行性**：$g_i(x^*) \le 0$，$h_j(x^*) = 0$。
4. **对偶可行性**：$\lambda_i \ge 0$。

这四个条件是凸优化最优解的充分必要条件（Slater 条件下）。在活跃约束集固定时，KKT 退化为一个线性方程组——这就是隐函数求导的基础。

1.3 隐函数定理

设 KKT 条件写成 $F(x^*, \lambda^*, \theta) = 0$，其中 θ 是外部参数。隐函数定理说：如果 F 对 (x, λ) 的雅可比矩阵可逆，则

$$\frac{\partial (x^*, \lambda^*)}{\partial \theta} = -J_{(x,\lambda)} F^{-1} \cdot J_\theta F.$$

这给出 $dz^*/d\theta$ 的解析表达式。工程上不需要手推——cvxpylayers 等库自动做这件事。

1.4 子梯度（subgradient）

当优化问题在边界点不可微时（如玩具 QP 的 $\theta = 0$），用子梯度代替梯度。子梯度是一个区间：在 $\theta = 0$ 处，$x^* = \max(\theta, 0)$ 的子梯度是 $[0, 1]$ 区间里的任意值。训练时从中随机取一个，或取区间中点。

KKT 条件的角色

回忆第 2 讲的对偶。对凸优化，最优解同时满足三条件：梯度为零（平稳性）、不等式约束乘子非负、互补松弛。在活跃约束集（active set）固定时，这组条件是线性方程，可解出 x^* 对参数 θ 的导数。

2. 完整推导

2.1 问题设定

给定参数 θ（如预测的需求），解一个 LP：

$$z^*(\theta) = \arg\min_x c(\theta)^\top x \quad \text{s.t.} \quad A(\theta) x \le b(\theta),\ x \ge 0.$$

要把 $z^*(\theta)$ 当网络层，让 loss $L(z^*(\theta))$ 的梯度 $\partial L / \partial \theta$ 能反传，即要算 $\partial z^* / \partial \theta$。

2.2 KKT 条件

对凸优化问题，最优解 x^* 满足：

$$\begin{aligned}
&\nabla_x f(x^*) + \sum_i \lambda_i \nabla g_i(x^*) + \sum_j \mu_j \nabla h_j(x^*) = 0, \\
&\lambda_i g_i(x^*) = 0,\ \lambda_i \ge 0, \\
&g_i(x^*) \le 0,\ h_j(x^*) = 0.
\end{aligned}$$

对 LP，在活跃集（当前紧约束）上，这变成一组线性方程。活跃集内约束取等号，KKT 退化为线性系统。

逐条解释这三个条件：第一行是”目标函数的梯度被约束梯度平衡”——最优处不能往任何方向改进了。第二行是互补松弛——如果约束 i 不紧（$g_i < 0$），则乘子 $\lambda_i = 0$，这个约束不影响最优解。第三行是原问题可行性——最优解必须满足所有约束。

2.3 隐函数求导

设 KKT 条件写成 $F(x^*, \lambda^*, \theta) = 0$。由隐函数定理:

$$\frac{\partial(x^*, \lambda^*)}{\partial \theta} = -J_{(x,\lambda)} F^{-1} \cdot J_\theta F.$$

这给出 $dz^*/d\theta$ 的解析表达式。工程上用 cvxpylayers 等库——你写出凸优化问题，库自动构造可微层，前向解一次、反向解一个线性系统。

2.4 为什么端到端比 predict-then-optimize 好

Predict-then-optimize 里，预测模型优化”预测准”（MSE），但”预测准”不等于”决策好”。报童例子：需求预测误差小 1%，库存成本可能只降一点；但预测偏 5%，库存成本可能涨很多。决策损失对预测误差的敏感度非线性。

端到端直接优化决策损失:

$$\min_\theta \mathbb{E}_{data}\big[L(z^*(\theta))\big],$$

梯度穿过 LP 层反传，让预测模型朝”让决策更好”调，而不是”让预测 MSE 更小”。预测模型不再是独立预测器，而是决策系统的一部分。

举个具体例子：预测模型输出需求均值 $\hat{D}$，库存策略按 $Q = \hat{D} + z\sigma$ 订货。predict-then-optimize 训练预测模型最小化 $(\hat{D} - D)^2$（MSE）。端到端则直接最小化库存成本 $c_u \mathbb{E}[(D-Q)^+] + c_o \mathbb{E}[(Q-D)^+]$。如果缺货代价 c_u 很大，端到端会让 $\hat{D}$ 稍微偏高——因为多备一点比少备一点划算。这个偏置在 MSE 训练里是”错误”，但在决策里是”正确”。

2.5 数值算例：玩具 QP 的梯度

最简单的 QP：$\min_x (x-\theta)^2$，约束 $x \geq 0$。解析解 $x^* = \max(\theta, 0)$。

- $\theta = 2$：$x^* = 2$，梯度 $dx^*/d\theta = 1$（约束非活跃）。
- $\theta = -1$：$x^* = 0$，梯度 $= 0$（约束激活，解固定在边界）。
- $\theta = 0$：解在边界上，梯度不存在（子梯度在 $[0, 1]$）。

这个玩具例子展示可微层核心：非活跃区梯度光滑（=1），活跃区梯度为 0（不随 θ 变），边界点不可微。工程上用平滑（ReLU 换 softplus）避开不可微点。

2.6 局限

- 前向解一次 LP、反向解一个线性系统，比普通网络层慢。
- LP 在 active set 切换处不可微，要子梯度或扰动。
- 只适用凸问题：MIP 有离散决策，KKT 不连续，不能直接反传。

3. 直觉理解

> **直觉**：传统是”预报员报天气、农民按预报决定种地”——预报员只关心报得准，不关心农民种错地的损失。端到端把预报员和农民绑在一起训练，让预报员知道”你的预报偏差怎么变成农民的损失”，从而往对决策好的方向预报。可微优化层就是打通这个分工的数学工具。

这个比喻的关键：MSE 训练的预报员会把均值报准，但不关心农民是要”保守一点”还是”激进一点”。端到端训练的预报员会知道——如果农民是种草莓（缺货代价大），预报员会主动把温度报高一点（多备防寒）；如果农民是种小麦（折价代价小），预报员会报保守一点。

4. 常见误区

误区一：以为端到端可微优化会取代 MIP 求解器。它只在凸优化（LP/QP/SOCP）上可微，MIP 离散决策没法直接反传。

误区二：把”可微”等同于”精确梯度”。 LP 在 active set 切换处不可微，要用子梯度或平滑化。

误区三：以为端到端一定比 predict-then-optimize 好。若预测模型容量小、数据少，端到端可能过拟合决策损失，反而不如先学好预测再优化。

5. 代码实现

用玩具 QP $\min(x-\theta)^2,\ x \geq 0$ 演示解析解与数值梯度。

```
import numpy as np

# 玩具 QP: min_x (x-theta)^2  s.t. x >= 0
# 解析解: x* = max(theta, 0)
# 这个例子展示可微优化层的核心现象:
#   theta > 0 时约束不活跃, x* = theta, 梯度 = 1
#   theta < 0 时约束活跃, x* = 0, 梯度 = 0（解被边界钉住）
#   theta = 0 时不可微, 子梯度在 [0,1]
def solve_qp(theta):
    return max(theta, 0.0)

# 展示解析解随 theta 的变化
thetas = np.linspace(-2, 2, 9)
xs = [solve_qp(t) for t in thetas]
print("theta :", np.round(thetas, 1))
print("x*(t) :", xs)

# 用数值微分验证梯度（中心差分）
# 中心差分 (f(x+h)-f(x-h))/(2h) 比前向差分更精确
eps = 1e-6
for t in [-1.0, 0.5, 2.0]:
    grad = (solve_qp(t+eps) - solve_qp(t-eps))/(2*eps)
    region = " 非活跃区 (应 =1)" if t > 0 else " 活跃区 (应 =0)"
    print(f"theta={t:+.1f}: 数值梯度 ={grad:.3f}  [{region}]")
```

预期：$\theta > 0$ 时梯度约 1（解随 θ 同步变化）；$\theta < 0$ 时梯度约 0（解被边界钉在 0）。这正是”约束激活与否决定梯度”。

6. 小结

1. 可微优化层把 LP/QP 写成网络层，梯度可反传。
2. KKT 条件 + 隐函数定理给出解析梯度。
3. 端到端训练优化决策损失，而非预测 MSE。
4. 局限：仅凸问题、active set 切换处不可微、训练慢。

局限与延伸

- 端到端要求凸问题；含离散决策的 MIP 需用黑盒梯度估计（如 REINFORCE）而非解析反传。
- 延伸阅读：Agrawal 等《Differentiable Optimization Layers》(cvxpylayers)；Wilder 等《Melding Planning and Learning》；Prolog：predict-then-optimize 综述。

补充：实际工程中，cvxpylayers 的前向时间主要花在解凸优化上——一个 LP 可能要毫秒到秒级。如果你的网络要做 100 次前向，就是秒级开销。缓解方法：只在 loss 计算时做端到端反传，预测时用快速近似。

7. 思考题

基础档

1. 为什么 MIP 不能直接做可微层？从 KKT 不连续解释。
2. 玩具 QP 中 $\theta = 0$ 时梯度为什么不存在？子梯度是什么？

进阶档

3. predict-then-optimize 与端到端的损失函数有何不同？为什么后者对数据量要求更高？
4. LP 在 active set 切换处不可微，训练时怎么用平滑化处理？

开放档

5. 大模型推理调度里”预测请求到达率 → 解 batch 配置”能否端到端？哪些环节是凸的、哪些是离散的？
6. 当优化层前向很慢时，如何用代理模型（surrogate）加速反传？这会引入什么偏差？

第 16 讲 神经网络的形式化验证：用运筹方法证明安全性

> **本讲问题**：自动驾驶的神经网络看到停止标志，输出”刹车”。但旁边多一个光斑、标志稍微偏一点，网络会不会突然输出”加速”？我们不能靠测试覆盖所有情形——必须**证明**：在输入的某个邻域内，网络输出不会变。这就是神经网络形式化验证，它本质是一个 LP 或 MILP 问题。运筹学的对偶、割平面、分支定界，在这里全都能用。

1. 数学工具/预备知识

1.1 验证问题的形式化定义

给定一个训练好的神经网络 $f : \mathbb{R}^n \to \mathbb{R}^m$，我们想证明它在某个输入区域 X 上满足某个性质 P。形式化地：

$$\text{求证：} \quad \forall x \in X, \quad P(f(x)) \text{ 成立.}$$

举例：自动驾驶场景中，$f(x)$ 是对图像 x 的分类输出（如”停止标志”“限速标志”“行人”）。性质 P 可能是”在停止标志图像 x_0 的 ℓ_∞ 邻域 $\|x - x_0\|_\infty \le \epsilon$ 内，f 始终输出’停止标志’“。

关键区别：**测试**是从 X 中抽有限个点看是否满足 P；**验证**是要证明 X 中所有点都满足 P。测试给统计概率（“我测了 1000 张都对”），验证给数学保证（“邻域内不可能出错”）。

1.2 ReLU 激活函数的分段线性性

ReLU 定义为 $z = \max(0, y)$，其中 $y = ax + b$ 是前一层的线性输出。它有两段：

- 当 $y > 0$ 时，$z = y$（激活/开启，active）；
- 当 $y \le 0$ 时，$z = 0$（关闭/inactive）。

在每一段内，z 是 y 的线性函数（要么恒等映射要么归零）。一个 ReLU 网络有 k 个 ReLU 单元，每个有两种状态（开/关），总共 2^k 种组合。每种组合对应一个”线性段”，在该段内整个网络退化为一个线性函数 $f(x) = Wx + b$。

1.3 大 M 法（big-M encoding）

要把 $z = \max(0, y)$ 编码成线性约束，需要引入一个 0-1 变量 α 和一个足够大的常数 M：

$$z \ge 0, \quad z \ge y, \quad z \le y + M(1 - \alpha), \quad z \le M\alpha, \quad \alpha \in \{0, 1\}.$$

- $\alpha = 0$ 时：$z \le M \cdot 0 = 0$，结合 $z \ge 0$ 得 $z = 0$（关闭）。同时 $z \le y + M$ 自动满足（因为 M 很大）。
- $\alpha = 1$ 时：$z \le M$（上界松），$z \le y + M(0) = y$，结合 $z \ge y$ 得 $z = y$（开启）。

M 的选取：要足够大以包含所有可能的 y 值，但太大又会导致数值不稳定。实践中用神经元的已知上界（通过区间传播算出）来定 M。

1.4 对偶方法与抽象解释

- **LP 对偶**：原问题求 $\min_{x\in X} f(x)$ 给一个上界（找到一个可行解）；对偶问题求 max 给一个下界。若对偶下界 > 0，则原问题最小值 > 0，性质成立。对偶法不求具体对抗样本，只求”最坏情况也坏不到哪去”的下界。
- **抽象解释（abstract interpretation）**：不精确跟踪每个神经元的值，而是用一个集合（区间 $[l, u]$、zonotope、多面体）来近似”神经元可能取的值范围”。逐层传播这些集合，最后看输出集合是否满足性质。这给**保守保证**：抽象后性质还成立，则原网络一定成立；抽象后不成立，原网络可能成立也可能不成立（需要更精细验证）。

为什么 ReLU 网络好验证

ReLU 是分段线性的。一个 ReLU 要么”开”（$z = y$）要么”关”（$z = 0$），开关状态枚举出来，网络就退化成一个线性函数。把所有开关组合枚举——再用分支定界剪枝——就是把网络验证变成第 3 讲的 MIP。

对比其他激活函数：sigmoid、tanh 是光滑非线性的，不能拆成有限个线性段，没法编码成线性约束。GELU、Swish 同理。这就是为什么形式化验证目前主要针对 ReLU 网络——分段线性是它能变成优化问题的前提。一个有 k 个 ReLU 的网络有 2^k 种开关组合，全枚举是指数爆炸，但分支定界（第 3 讲）可以剪枝：一旦某个分支的对偶下界已经证明性质成立，就不用再枚举那个子树。

2. 完整推导

2.1 把 ReLU 编码为约束

ReLU 单元 $z = \max(0, y)$，$y = ax + b$。它等价于（大 M 法）：

$$z \geq 0, \quad z \geq y, \quad z \leq y + M(1 - \alpha), \quad z \leq M\alpha,$$

其中 $\alpha \in \{0, 1\}$、M 是上界。$\alpha = 0$ 时 $z = 0$（关闭），$\alpha = 1$ 时 $z = y$（开启）。整个网络所有 ReLU 这样编码，就变成一个 MILP：变量数 = 权重数 + ReLU 开关数。

为什么大 M 法有效？逐条约束验证两种情形：

- 若 $\alpha = 0$：第四条 $z \leq M \cdot 0 = 0$，结合第一条 $z \geq 0$，强制 $z = 0$。第三条 $z \leq y + M$ 因为 M 很大而自动满足。关闭态正确编码为 $z = 0$。
- 若 $\alpha = 1$：第四条 $z \leq M$（上界松，只要 M 大于 y 的所有可能值就不限制）。第三条 $z \leq y + M(0) = y$，结合第二条 $z \geq y$，强制 $z = y$。开启态正确编码为 $z = y$。

M 不能取无穷大——那样 LP 松弛会很差，分支定界要枚举几乎所有组合。实践中用前向区间传播算出每个神经元的取值上界，把 M 设为那个上界，既保证正确性又让松弛紧。

2.2 验证问题形式化

要验证”对所有 $\|x - x_0\| \leq \epsilon$，分类器输出类 1 而非类 2”，即求最坏情况下两类的差距：

$$\min_{x:\ \|x - x_0\| \leq \epsilon} \big(f_1(x) - f_2(x)\big).$$

若这个最小值 > 0，则邻域内类 1 总比类 2 大，验证通过；否则那个最小化点 x 就是对抗样本。这是一个 MILP：目标 $f_1 - f_2$，约束是所有 ReLU 编码 + 输入邻域。

为什么求最小值就能判定？因为 $f_1(x) - f_2(x) > 0$ 等价于”网络预测类 1”。如果在整个输入邻域上 $f_1 - f_2$ 的最小值都大于 0，那邻域内任何扰动都不会让分类翻转——安全性得证。反之，如果最小值 ≤ 0，那个取得最小值的 x^* 就是一个对抗样本：在 x^* 处网络会误分类。这把”证明安全”和”找对抗样本”统一成同一个优化问题的两面：最小值 > 0 是安全，最小值 ≤ 0 时最优点就是攻击。

2.3 对偶方法：效率更高

直接解 MILP 在大网络上太慢。换思路：求 $f_1(x) - f_2(x)$ 的下界。网络分段线性，写成 LP 后，其对偶给出一个可证明的下界。若对偶下界 > 0，就证明了安全性；否则可能不成立，要找反例。对偶法不求具体对抗样本，只求”最坏也坏不到哪去”的下界，因此快。

为什么对偶能加速？回忆第 2 讲的对偶理论：原 LP $\min c^\top x$ s.t. $Ax \le b$ 的对偶是 $\max b^\top y$ s.t. $A^\top y = c, y \ge 0$。对神经网络验证问题，原问题是”找最坏输入使输出差距最小”，对偶问题是”在对偶空间里求一个可证明的下界”。如果对偶下界 > 0，原问题最小值一定 > 0，验证通过——不需要枚举 ReLU 开关组合，只解一个 LP。Wong–Kolter (2018) 的方法就是把每一层 ReLU 都对偶化，逐层传播对偶界，一次前向传播就算出整个网络的输出下界。

2.4 抽象解释

对百万参数大网络，完整 MILP 算不动。实用做法：把每个神经元取值范围用区间（或 zonotope、多面体）近似，逐层传播。这给**保守保证**：抽象后性质还成立，则原网络一定成立；抽象后不成立，原网络可能成立也可能不成立，需更精细验证。

区间传播最简单：若第 i 层输入 $x \in [l_i, u_i]$，经过 ReLU 后输出 $z \in [\max(0, l_i), \max(0, u_i)]$。逐层算区间，最后得到输出的区间 $[L, U]$。如果输出区间 $[L, U]$ 完全满足性质（如 $L > 0$），则验证通过。缺点是区间传播很保守——它假设每个神经元独立取端点，实际它们之间有耦合，导致区间过宽，很多本可验证的案例被误判为”不确定”。

2.5 为什么这是运筹问题

形式化验证把”神经网络安全性”这个看似 AI 的问题，转化成”带 ReLU 约束的优化问题”。运筹学过去 60 年的对偶、割平面、分支定界全都能用——这也是运筹学家在 AI 安全领域有发言权的原因：手里的工具正好对症。

具体来说，验证问题的 MILP 结构和第 3 讲的整数规划完全同构：ReLU 开关变量 α 就是 0-1 变量，大 M 约束就是线性约束，目标函数就是 $f_1 - f_2$。求解时用的分支定界策略（在哪个 ReLU 上分支、先探哪个分支）、割平面生成（切掉那些导致误分类的开关组合）、热启动（先用区间传播给一个初始下界），全都是运筹学的标准技术。差别只在于问题规模：一个小网络可能有几千个 ReLU 变量，比传统 MIP 问题大得多，所以需要专门的剪枝技巧。

3. 直觉理解

> **直觉**：验证像安检——不能只检查几个乘客，要证明”整个包里没有危险品”。把每层 ReLU 开关状态列出来，就是枚举所有”网络反应模式”。对偶法不求每种模式的具体反应，而用一个”最坏情况”的下界证明安全性。抽象解释是更粗的安检：X 光扫一遍快速筛可疑包，但精度有限。

这个比喻里，“全枚举”是逐个翻包检查（最准但最慢），“对偶法”是用 X 光机扫一遍看有没有金属（快但可能漏小物件），“抽象解释”是只看包的重量和外形（最快但最粗）。三种方法对应不同的精度-速度 trade-off。

4. 常见误区

> **误区一：以为测试准确率 99% 就等于安全**。测试只覆盖训练分布，对抗扰动在分布外。形式化验证给数学保证，不是统计概率。
>
> **误区二：把验证当成可解问题**。大网络完整验证目前算不动，常用抽象近似给保守保证。
>
> **误区三：以为验证只对安全关键系统有用**。它在模型压缩、量化鲁棒性、可解释性上都有应用。

5. 代码实现

演示一个极简 ReLU 网络的 LP 验证。

```
import numpy as np
from scipy.optimize import linprog

# 玩具网络: f(x) = ReLU(x1) + x2
```

```
# 验证目标：在 x1, x2 ∈ [-1, 1] 这个输入框内，f(x) >= -1 是否恒成立？
# 手算分析：x1=-1 时 ReLU(x1)=0，x2=-1 时 f=-1。所以下界就是 -1，紧成立。
#
# 为什么这是一个 LP？因为在固定 ReLU 开关状态下，网络退化为线性函数。
# 这里 ReLU(x1)：当 x1 >= 0 时 ReLU=x1（开），当 x1 < 0 时 ReLU=0（关）。
# 我们要求 f 的全局最小值，所以需要考虑两种开关状态。
# 下面的 LP 编码了" 关闭" 状态（z=0），因为求最小值时 x1 倾向取负。
#
# 变量顺序 [x1, x2, y, z]，其中 y = x1（ReLU 输入），z = ReLU(y) = max(0,y)
# 约束：
#   y = x1             (等式约束 A_eq)
#   z >= y             (不等式 y - z <= 0)
#   z >= 0             (bounds 里 z 的下界)
# 注意：这里没有编码 z <= y + M(1-alpha) 等开关约束，因为玩具例子只取关闭态。

c = [0, 1, 0, 1]              # 目标 min f = z + x2，所以系数是 [0, 1, 0, 1]
A_eq = [[-1, 0, 1, 0]]        # -x1 + y = 0  => y = x1（等式约束）
b_eq = [0]
A_ub = [[0, 0, 1, -1]]        # y - z <= 0    => z >= y（不等式约束）
b_ub = [0]
# 变量边界：x1,x2 ∈ [-1,1]；y=x1 ∈ [-10,10]（宽松上界不影响最优解）；z >= 0
bounds = [(-1,1), (-1,1), (-10,10), (0,10)]

# scipy 的 linprog 做的是 min c^T x s.t. A_ub x <= b_ub, A_eq x = b_eq
# method="highs" 是 HiGHS 求解器，默认且数值稳定
r = linprog(c, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
            bounds=bounds, method="highs")
print("f 的最小值 =", round(r.fun, 3), " 在 (x1,x2) =", round(r.x[0],2), round(r.x[1],2))
print(" 性质 f>=-1 是否成立？", bool(r.fun >= -1))
```

预期最小 $f = -1$（在 $x_1 = x_2 = -1$），性质成立。若把阈值改成 $f \geq -0.9$，则该性质被同一个反例 $(-1, -1)$ 违反——这就是”求最小值、看是否过阈值”的验证逻辑。

这个玩具例子展示了验证的核心操作：求函数在输入域上的全局最小值，然后看最小值是否过阈值。对大网络，这个最小值要用 MILP 求解器算——不是因为问题难，而是因为 ReLU 的分段线性性让”全局最小”变成了组合搜索问题。

6. 小结

1. 神经网络形式化验证 = 带 ReLU 约束的 MILP。
2. 验证目标：证明在输入邻域内输出不变；求输出在邻域上的最坏值。
3. 对偶方法给可证明下界，比直接解 MILP 快。
4. 大网络验证仍困难，常用抽象近似给保守保证。

局限与延伸

- 完整验证对大网络仍是指数难；当前只能验证中小网络或用抽象近似。
- 延伸阅读：Reluplex（Katz 等 2017）；abstract interpretation 见 Ehlers (2017)；对偶验证见 Wong–Kolter (2018)。

补充：工业实践中，形式化验证不是”全有或全无”。安全关键系统（自动驾驶、医疗设备）做完整验证；一般 AI 产品用”抽象解释快速筛 + 红队测试补充”。验证精度和计算成本是 trade-off——验证越精确越慢，太粗又漏风险。

7. 思考题

基础档

1. 为什么 ReLU 比 sigmoid 更容易编码成 MILP？

2. 抽象解释给出”保守保证”，“保守”是什么意思?

进阶档

3. 1000 个 ReLU 全枚举开关是 2^{1000} 种，怎么用分支定界剪枝避免?
4. 验证和对抗训练（adversarial training）是什么关系？前者证明、后者只是经验。

开放档

5. 大模型有几千亿参数、激活函数不是 ReLU（GELU/SwiGLU）。形式化验证还能用吗？哪里失效?
6. 当验证代价远超训练收益时，工业界如何权衡”证明安全”与”红队测试”?

第 17 讲 LLM 辅助建模：从自然语言到优化模型

> **本讲问题**：业务经理说”我们要在三个仓库之间调货，每个仓库不能断货，运费最便宜”。这句话要变成一个 LP，过去需要懂运筹的分析师花一天。现在 LLM 能直接把这段话翻译成数学模型。但翻译经常错——漏约束、错系数、变量名冲突。这一讲讲怎么用 LLM 辅助建模，以及怎么检测它的错误。

1. 数学工具/预备知识

1.1 建模流水线的六个环节

把业务问题变成可求解的优化模型，标准流程是：

1. **抽取实体**：从自然语言中识别哪些是资源、哪些是活动、哪些是约束。“仓库”是供应点（有供应量上限），“门店”是需求点（有需求量），“运费”是目标系数。
2. **列决策变量**：每个可调的量设一个变量。运输问题里 x_{ij} = 从仓库 i 运到门店 j 的吨数。
3. **写目标函数**：业务要优化什么？最小成本 $\min \sum c_{ij}x_{ij}$ 还是最大利润?
4. **写约束**：资源上限（仓库不能发超过库存的货）、需求下限（门店必须收到要的货）、平衡（运出 = 运入）。
5. **给数据**：系数表（供应量、需求量、运费矩阵的具体数值）。
6. **求解 + 解释**：把求解器输出的 x_{ij} 矩阵翻译回业务语言——“仓库 1 给门店 2 运 15 吨”。

1.2 LLM 建模错误的五种类型

LLM 把业务描述翻成模型时，常见错误：

1. **漏约束**：业务说了 5 条限制，LLM 只写了 4 条。最危险，因为求解器不知道缺了什么。
2. **错变量**：把连续变量建成整数，或反过来。
3. **单位错**：把”元/吨公里”当成”元/吨”，漏乘距离。
4. **约束方向反**：把 $\leq$ 写成 $\geq$，或把等式写成不等式。
5. **系数错位**：变量下标 x_{ij} 顺序写反，运费矩阵行列对调。

1.3 建模验证方法

- **可行性测试**：随便赋一组值，看约束是否满足。随便赋的值都违反约束，模型肯定错。
- **维度检查**：每条约束两边量纲是否一致。左边是吨、右边也是吨，不能一边吨一边元。
- **对偶检验**：解原问题和对偶问题，最优值应相等（强对偶，第 2 讲）。不等说明约束写错。
- **边界测试**：极端数据下模型输出是否合理。需求全 0 时总运费应为 0。

为什么 LLM 会建模错

LLM 见过大量”问题描述—模型”配对，能学到模式，但它不理解业务语义。它擅长”像个 LP”，不擅长”这个 LP 对不对”。所以定位是”出草稿”，不是”给答案”。

2. 完整推导

2.1 LLM 建模的标准流程

1. **抽取实体**：从业务描述识别资源、活动、约束。“仓库”是供应点、“门店”是需求点、“运费”是目标系数。

2. **列决策变量**：每个可调的量是一个变量。x_{ij} = 仓库 i 到门店 j 的运量。
3. **写目标**：业务想要什么（最小成本/最大利润）。min $\sum c_{ij}x_{ij}$。
4. **写约束**：资源上限、需求下限、平衡。供应约束 $\sum_j x_{ij} = s_i$、需求约束 $\sum_i x_{ij} = d_j$。
5. **给数据**：系数表（供应量、需求量、运费矩阵）。
6. **求解 + 解释**：把解翻译回业务语言。

2.2 典型错误案例

业务："3 个仓库、4 个门店，仓库各 20/30/25 吨，门店各要 10/25/20/20 吨，运费见矩阵，怎么调最便宜?"

LLM 可能犯的错：

- 把供应约束写成 $\sum_j x_{ij} \geq s_i$（应是 $=$），导致多运。
- 漏掉需求约束 $\sum_i x_{ij} = d_j$。
- 把"元/吨公里"当成"元/吨"，漏乘距离。
- 变量下标 x_{ij} 顺序写反，系数矩阵错位。
- 把"至少运 10 吨"理解成"最多运 10 吨"。

2.3 怎么检测错误

- **可行性测试**：随便赋一组值，看约束是否满足。随便赋的值都违反约束，模型肯定错。
- **维度检查**：每条约束两边量纲是否一致。
- **对偶检验**：解一次原问题、一次对偶，最优值应相等（强对偶）。不等说明约束写错。
- **业务反推**：把解代入，看业务上是否合理（运量、运费不能为负）。
- **边界测试**：需求 = 0 时总运费应为 0；某仓库断货时模型是否报警。

2.4 LLM 不能替代的部分

- **数据准备**：LLM 不知道你的真实运费表。它能写 c_{ij} 的符号，但具体数字要人填。
- **约束取舍**：哪些是硬约束（必须满足）、哪些是软约束（尽量满足），要业务方确认。LLM 默认把所有约束写成硬约束。
- **异常处理**：模型解不可行时，问"哪条约束过紧"需要人判断。LLM 能给出 IIS（最小不可行子集）的候选，但哪个该放松要业务决策。
- **模型选择**：连续用 LP、整数用 MIP、随机用鲁棒——这个判断 LLM 经常选错。例如它会把带"至少/最多"约束的问题自动建成整数规划，但其实连续 LP 就够。

2.5 一个完整的 LLM 建模交互示例

业务描述："我们有 3 个仓库，各有 20、30、25 吨货。4 个门店分别要 10、25、20、20 吨。运费见下表。怎么调货运费最少?"

Prompt 给 LLM 时，要求它：1. 先列决策变量定义；2. 再写目标函数；3. 再逐条列约束（标明供应约束和需求约束的区别）；4. 最后说"请检查：供应约束是 $\leq$ 还是 $=$? 需求约束是 $\leq$ 还是 $=$?"

这个自检 prompt 能显著减少 LLM 把约束方向写反的概率。

3. 直觉理解

> **直觉**：LLM 像个刚毕业的建模助理——能快速把业务描述翻成数学，但对业务细节不熟，容易漏。你要做的不是全盘接收，而是把它当草稿：让它出第一版，你审、改、测。真正值钱的是"知道哪里可能错"的审计能力。LLM 把建模时间从一天压到一小时，但审计那半小时省不掉。

这个助理的能力边界：它见过大量 LP 例子，所以"像个 LP"没问题。但它不知道你的业务里"至少 10 吨"是硬约束还是软约束，不知道运费里是否包含保险——这些业务知识不在训练数据里。

4. 常见误区

误区一：把 LLM 输出直接交给求解器跑。 LLM 常写出语法对但逻辑错的模型，求解器会高高兴兴解一个错问题。

误区二：以为 LLM 能自动补数据。它能写模型结构，系数必须人填。

误区三：以为模型越复杂越好。复杂模型约束多、求解慢、易错。先简单跑通，再按需加约束。

5. 代码实现

演示”LLM 生成的运输问题模型”的可行性与边界测试。

```
import numpy as np
from scipy.optimize import linprog

# LLM 生成的模型：3 仓库 4 门店的运输问题
# s[i] = 仓库 i 的供应量（吨），d[j] = 门店 j 的需求量（吨）
s = [20, 30, 25]
d = [10, 25, 20, 20]
# C[i,j] = 从仓库 i 运到门店 j 的单位运费（元/吨）
C = np.array([[2,3,1,4],[5,4,2,3],[3,2,5,2]], dtype=float)

m, n = 3, 4
c_flat = C.flatten()  # linprog 要求一维目标系数向量
# 供应约束用不等式 <=（仓库可留货不发完），需求约束用等式 =（门店必须收到足够货）
A_ub = []; b_ub = []
for i in range(m):
    row = [0.0]*(m*n)
    for j in range(n): row[i*n+j] = 1  # x_{i,j} 的系数 = 1
    A_ub.append(row); b_ub.append(s[i])  # sum_j x_{i,j} <= s_i
A_eq = []; b_eq = []
for j in range(n):
    row = [0.0]*(m*n)
    for i in range(m): row[i*n+j] = 1  # x_{i,j} 的系数 = 1
    A_eq.append(row); b_eq.append(d[j])  # sum_i x_{i,j} = d_j

# linprog 解 min c^T x s.t. A_ub x <= b_ub, A_eq x = b_eq, x >= 0
r = linprog(c_flat, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
            bounds=[(0,None)]*(m*n), method="highs")
print(" 求解状态:", r.message)
print(" 总运费 =", round(r.fun, 2))
X = r.x.reshape(m,n)  # 恢复成 3x4 矩阵看调运方案
print(" 调运方案:\n", np.round(X, 2))

# 审计 1：行和 <= 供应（仓库没超发），列和 = 需求（门店都满足了）
print(" 行和（应 <= 供应):", np.round(X.sum(axis=1),2), " 上限 =", s)
print(" 列和（应 = 需求）  :", np.round(X.sum(axis=0),2), " 应 =", d)

# 审计 2：边界测试——需求全 0 时，应一件不运、总运费 = 0
b_eq_zero = [0,0,0,0]
r2 = linprog(c_flat, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq_zero,
             bounds=[(0,None)]*(m*n), method="highs")
print(" 边界测试（需求 =0）总运费 =", round(r2.fun, 2), "(应 =0)")
```

如果 LLM 误把供应约束写成 $\geq$，行和会大于供应、总运费偏低——行和审计和边界测试能立刻发现。

这个例子展示了 LLM 建模的典型工作流：生成 → 求解 → 审计 → 修错。审计是不可省略的环节——LLM 输出的模型语法上正确，但逻辑上可能错。求解器会高高兴兴地解一个错问题，给你一个”看起来很合理”的错答案。只有通过审计（行和检查、边界测试、对偶检验），你才能确信模型是对的。

6. 小结

1. LLM 能快速把业务描述翻成 LP 草稿。
2. 常见错误：漏约束、错系数、量纲错、方向反。
3. 审计方法：可行性测试、对偶检验、业务反推、边界测试。
4. LLM 出草稿，人做审计和数据。

局限与延伸

- LLM 建模的可靠性依赖 prompt 与验证闭环；没有自动审计时，错模型会被当成真结果。
- 延伸阅读：LLM for OR 的综述（“LLM for optimization modeling”）；对偶检验（第 2 讲）；模型审计的软件实践。

补充：一个正在出现的方向是”LLM + 求解器”自动闭环——LLM 生成模型，求解器跑，不可行时 LLM 读求解器日志自动修约束。但这个闭环目前还不可靠，核心瓶颈是 LLM 看不懂 IIS（最小不可行子集）的诊断信息。

7. 思考题

基础档

1. 让 LLM 把”外卖骑手路径规划”翻成 TSP，它会漏哪些约束（时间窗、载重）?
2. 对偶检验为什么能发现”约束写错”?

进阶档

3. 怎么用自然语言 prompt 让 LLM 输出更不容易错的模型（提示：要求它列假设、自检）?
4. 如果 LLM 生成的模型无解，怎么定位是哪条约束过紧（提示：最小不可行子集 IIS）?

开放档

5. 当 LLM 既能建模又能调求解器时，人在建模流水线里的增值点是什么？会消失吗?
6. 如何设计一个”LLM 建模 + 自动验证”的闭环，让错模型在交付前被拦住?

第 18 讲 需求预测与库存联动：从预测误差到决策损失

> **本讲问题**：预测模型 RMSE 降 10%，库存成本能降多少？很多人以为是 10%，其实答案是——不一定。预测误差对库存成本的传导是**非线性**的：偏差方向（高估/低估）、方差大小、服务水平都决定最终成本。本讲把这个传导链算清楚。

1. 数学工具/预备知识

本讲的核心是”预测误差怎么变成库存成本”。要把这条传导链算清楚，需要先把几个概念的定义、计算方式和各自的角色拆开讲。

1.1 预测误差：偏差与方差

设某期真实需求为 D（随机变量），预测模型输出 $\hat{D}$（通常是对 D 的点估计）。定义预测误差

$$\epsilon = D - \hat{D}.$$

误差不是一个数，而是一个分布——每次预测都有一个误差值，多次预测累积成一个分布。这个分布有两个独立特征：

- **均值偏差** $\mu_\epsilon = \mathbb{E}[\epsilon] = \mathbb{E}[D] - \hat{D}$。它衡量预测”系统性偏高还是偏低”。若 $\mu_\epsilon > 0$，说明真实需求平均比预测大——预测**低估**了；若 $\mu_\epsilon < 0$，说明预测**高估**了。$\mu_\epsilon = 0$ 叫无偏预测。
- **方差** $\sigma_\epsilon^2 = \mathrm{Var}(\epsilon) = \mathrm{Var}(D - \hat{D})$。若预测本身是确定性的（不随随机源变化），则 $\sigma_\epsilon^2 = \mathrm{Var}(D)$，即误差方差等于需求方差。若预测模型本身也有随机性（如贝叶斯预测输出一个分布），则 σ_ϵ^2 还包含预测不确定性。

为什么要把偏差和方差分开？因为它们对库存成本的作用路径完全不同：偏差直接平移订货量，方差通过安全库存起作用。混在一起算（比如只报一个 RMSE）会掩盖问题。RMSE $= \sqrt{\mu_\epsilon^2 + \sigma_\epsilon^2}$ 把两者揉成一个数，无法区分”你预测偏了”和”你预测抖动大”——而这两种情况的补救措施完全不同（纠偏 vs 加安全库存）。

1.2 库存成本：缺货与持有

库存成本有两个对称但不等价的部分：

- **缺货成本** c_u（underage cost）：少订了一件，本来能赚的利润没赚到。例如售价 15 元、成本 10 元，少卖一件损失利润 $c_u = 5$ 元。在零售场景，缺货还可能导致客户流失，实际 c_u 比单笔利润高。
- **持有/折价成本** c_o（overage cost）：多订了一件，卖不掉要折价处理。例如成本 10 元、折价后残值 3 元，多订一件损失 $c_o = 7$ 元。

注意 c_u 和 c_o 一般不相等。对大多数快消品，$c_u > c_o$（缺货丢客户比折价清货更痛），这就是后面”低估比高估更痛”的根源。

1.3 报童公式与分位数函数

报童问题的最优订货量是一个分位数：

$$Q^* = F^{-1}\left(\frac{c_u}{c_u + c_o}\right),$$

其中 F 是需求 D 的累积分布函数。如果需求是正态分布 $D \sim \mathcal{N}(\mu, \sigma^2)$，记标准正态分布的累积分布函数为 Φ、其反函数（分位数函数）为 Φ^{-1}，则

$$Q^* = \mu + z\sigma, \qquad z = \Phi^{-1}\left(\frac{c_u}{c_u + c_o}\right).$$

这里 z 是**临界分位数**对应的标准正态偏离值：

- 若 $c_u/(c_u + c_o) = 0.95$（缺货代价远大于折价），则 $z = \Phi^{-1}(0.95) \approx 1.645$。
- 若 $c_u = c_o$（对称损失），则 $z = \Phi^{-1}(0.5) = 0$，订货量就等于均值。

Φ^{-1} 叫**分位数函数**（quantile function / percent-point function），它回答”要累积概率达到 p，需要偏离均值几个标准差”。在代码里对应 `scipy.stats.norm.ppf(p)`。

1.4 服务水平

服务水平（service level）= 不缺货的概率 $P(D \le Q)$。在报童最优解处，

$$P(D \le Q^*) = F(Q^*) = \frac{c_u}{c_u + c_o}.$$

所以服务水平完全由 c_u, c_o 的比值决定。c_u 越大（缺货越痛），服务水平要求越高，z 越大，安全库存越多。常见的 95%、99% 服务水平，对应的就是 $z = 1.645$ 和 $z = 2.33$。

两条传导路径

预测误差有两条独立路径影响成本：**均值偏差** μ_ϵ 直接平移订货量；**标准差** σ_ϵ 通过安全库存 $z\sigma$ 影响持有成本。二者方向、代价不对称，必须分开算。

2. 完整推导

2.1 从预测到订货

报童问题里，若预测均值 $\hat{D}$、预测标准差 σ_ϵ，最优订货量：

$$Q^* = \hat{D} + z\sigma_\epsilon, \qquad z = \Phi^{-1}\left(\frac{c_u}{c_u + c_o}\right).$$

订货量 = 预测均值 + z 倍预测标准差。均值偏差直接改变 Q；方差通过 $z\sigma$ 项影响安全库存。

2.2 偏差 μ_ϵ 的代价

若预测系统性高估（$\hat{D} > D$，$\mu_\epsilon = \mathbb{E}[D] - \hat{D} < 0$ 按定义），订货量比真实最优多订了 $|\mu_\epsilon|$，多出来的按折价处理，每单位损失 c_o。若低估（$\hat{D} < D$，$\mu_\epsilon > 0$），少订了，每单位缺货损失 c_u。
为什么不对称？把期望库存成本写成订货量 Q 的函数（假设需求正态）：

$$C(Q) = c_u\,\mathbb{E}[(D-Q)^+] + c_o\,\mathbb{E}[(Q-D)^+].$$

这里 $(x)^+ = \max(x, 0)$ 表示取正部。第一项是缺货期望损失，第二项是折价期望损失。对 Q 求导并令导数为零，就得到报童公式 $F(Q^*) = c_u/(c_u + c_o)$。

现在假设预测均值偏了 δ，即 $\hat{D} = \mu + \delta$。按预测订货 $Q = \mu + \delta + z\sigma$。和真实最优 $Q^* = \mu + z\sigma$ 相比，多订或少订了 δ 件。多订 δ 件的边际成本是 c_o，少订 δ 件的边际成本是 c_u。

因为通常 $c_u > c_o$（缺货丢客户比折价痛），同样大小的偏差，低估造成的损失是 $c_u|\delta|$，高估是 $c_o|\delta|$，比值为 $c_u/c_o > 1$。**低估比高估更痛**，痛多少倍取决于 c_u/c_o。这就是库存策略”宁可多备不可少备”的来由。

2.3 方差 σ_ϵ^2 的代价

安全库存 = $z\sigma_\epsilon$，持有成本 $h \cdot z\sigma_\epsilon$。方差越大，安全库存越多，持有成本越高。注意它和 σ 是线性关系：σ 降一半，安全库存、持有成本都降一半。

为什么是线性而不是二次？从报童公式看，$Q^* = \mu + z\sigma$，安全库存 $= Q^* - \mu = z\sigma$。持有成本正比于平均库存（近似等于安全库存），所以持有成本 $\propto z\sigma$。这里没有 σ^2 项——方差的平方影响的是”极端需求出现的概率”，但期望持有成本只通过标准差起作用。这也是为什么库存管理里报”标准差”比报”方差”更直观：安全库存直接和标准差挂钩。

举个例子：$z = 1.645$（95% 服务水平），σ 从 10 降到 5，安全库存从 16.45 件降到 8.23 件，持有成本同步减半。

2.4 预测精度提升的边际收益递减

把 σ_ϵ 降 10%，安全库存降 10%，但总成本只降一小部分——因为总成本里还有采购成本、固定成本等大头。举例：总成本 = 采购 100 万 + 持有 10 万 + 缺货 5 万。预测改进让持有成本降 10%（1 万），总成本只降约 0.95%。但乘以几十家门店、上百 SKU，就是大钱。

这就是”边际收益递减”的数学表达：库存成本对 σ 的敏感度，取决于库存成本在总成本中的占比。占比越小，预测改进的绝对收益越小。工程上的含义是——不要在已经很准的预测模型上继续堆复杂度，先看库存成本占比再决定投入。

2.5 预测-决策联合优化

不要分别优化预测模型和库存策略。端到端：训练预测模型时，loss 不是 MSE，而是”用这个预测去订货，最终库存成本是多少”。即第 15 讲的可微优化思想：把报童公式 $Q = \hat{D} + z\sigma$ 当可微层，输入预测均值方差，输出订货量，再用真实需求模拟库存成本，反传梯度调预测模型。

3. 直觉理解

> **直觉**：预测像天气预报，库存像带伞。预报均值准但方差大（今天可能晴可能暴雨），你还是要带把大伞——这就是安全库存。预报均值偏乐观（总说晴天），你会忘带伞被淋——这就是偏差代价。预测改进的价值不在 RMSE 数字本身，而在它怎么变成”少带几把伞、少淋几次雨”。

4. 常见误区

误区一：把 RMSE 当预测价值指标。 RMSE 相同的两个模型，一个偏差 0、一个偏差大，库存成本天差地别。

误区二：以为预测越准越好。预测模型复杂度上升有数据、算力成本，而库存成本边际收益递减——过拟合预测反而不划算。

误区三：以为预测均值准了就够。方差估计对安全库存同样重要；很多预测模型只输出均值不输出方差，导致库存拍脑袋。

5. 代码实现

用蒙特卡洛精确比较不同预测精度下的库存成本（避免手算偏差）。

```
import numpy as np
from scipy.stats import norm

# cu = underage cost（少订一件损失的利润），co = overage cost（多订一件的折价损失）
# 这里取 cu=15, co=7，模拟" 缺货比折价痛" 的典型零售场景
cu, co = 15, 7
# 报童临界分位数：服务水平 P(D<=Q) = cu/(cu+co) = 15/22 ≈ 0.682
# norm.ppf(p) 就是 Phi^{-1}(p)，即标准正态累积分布的反函数
z = norm.ppf(cu/(cu+co))            # ≈ 0.474，对应 68.2% 服务水平

def inventory_cost(mu_hat, sigma_hat, true_mu, true_sigma, n=200000):
    """ 给定预测均值/标准差，按报童公式订货 Q=mu_hat+z*sigma_hat，
    用真实需求蒙特卡洛算期望库存成本。

    为什么用蒙特卡洛而不是解析公式?
    解析公式 C(Q) = cu*E[(D-Q)^+] + co*E[(Q-D)^+] 对正态有闭式，
    但蒙特卡洛更直观、且换分布（如泊松、负二项）时不用重推公式。
    n=200000 次采样让蒙特卡洛标准误 ~0.1%，足够对比各场景差异。
    """
    Q = mu_hat + z*sigma_hat         # 按预测的报童最优订货量
    np.random.seed(0)                # 固定种子保证可复现
    D = np.random.normal(true_mu, true_sigma, n)  # 真实需求分布
    lost_sales = np.maximum(D-Q, 0)  # D>Q 时缺货量；否则为 0
    overstock  = np.maximum(Q-D, 0)  # Q>D 时积压量；否则为 0
    return cu*lost_sales.mean() + co*overstock.mean()

# 真实需求：均值 30，标准差 5
true_mu, true_sigma = 30, 5
# 场景 1：完美预测——预测均值 = 真实均值、预测标准差 = 真实标准差，成本最低
print(" 完美预测         :", round(inventory_cost(30, 5, true_mu, true_sigma), 2))
# 场景 2：均值高估 2——多订 2 件，每件折价损失 co=7，额外损失 ~14
print(" 均值高估 2       :", round(inventory_cost(32, 5, true_mu, true_sigma), 2))
# 场景 3：均值低估 2——少订 2 件，每件缺货损失 cu=15，额外损失 ~30（> 高估）
print(" 均值低估 2       :", round(inventory_cost(28, 5, true_mu, true_sigma), 2))
# 场景 4：标准差高估（5->8）——安全库存 z*sigma 从 2.37 涨到 3.79，多持库存
print(" 标准差高估 (5->8):", round(inventory_cost(30, 8, true_mu, true_sigma), 2))
```

预期：完美预测成本最低；低估 2 的成本 > 高估 2 的成本（因为 $c_u = 15 > c_o = 7$，缺货更痛）；标准差高估导致多备安全库存、成本上升。这定量印证了”偏差方向不对称、方差影响安全库存”。

6. 小结

1. 预测误差通过均值偏差和方差两条路径影响库存成本。
2. 最优订货量 = 预测均值 + z × 预测标准差。
3. 因为 $c_u > c_o$，低估比高估代价更大；预测改进边际收益递减。

4. 端到端优化决策损失，而非 MSE。

局限与延伸

- 这里假设需求正态、报童单期；多期、有提前期、有相关性时要用更完整的库存模型。
- 延伸阅读：报童与预测联动的 literature；Croston 方法（间歇性需求预测）；第 15 讲可微优化。

7. 思考题

基础档

1. 偏差 μ_ϵ 和方差 σ_ϵ 哪个对库存成本影响更大？为什么不对称？
2. 服务水平从 90% 提到 99%，z 从 1.28 涨到 2.33，安全库存涨多少？

进阶档

3. 为什么低估预测比高估代价更大？用 c_u、c_o 的相对大小解释。
4. 怎么设计一个”预测-库存”联合训练的 loss（提示：把报童公式当可微层）？

开放档

5. 大模型推理里”预测请求到达率 → 配 GPU”是同样的联动。低估到达率会怎样？这里的 c_u、c_o 对应什么？
6. 当需求是间歇性（长周期零需求）时，普通均值预测会高估还是低估？该用什么预测方法？

第 19 讲 供应链网络优化：从单仓到多级网络

> **本讲问题**：一个商品从工厂 → 中心仓 → 区域仓 → 门店，每一级都要备货。每一级单独优化库存，整体可能过剩——因为各级各自加安全库存，叠加起来总库存爆炸。这就是**牛鞭效应**：终端一点小波动，传到上游被放大成大波动。本讲看多级库存网络怎么建模，以及牛鞭效应怎么破解。

1. 数学工具/预备知识

1.1 多级库存系统

一个典型的多级供应链从上游到下游依次是：原材料供应商 → 工厂 → 中心仓（RDC）→ 区域仓 → 门店（零售商）。每一级都持有库存，向上游订货、向下游供货。和单仓报童的区别在于：每一级面对的”需求”不是终端顾客需求，而是**下一级的订货量**。这个订货量被下一级的库存策略扭曲过——它不反映终端真实需求。

1.2 牛鞭效应的定义

牛鞭效应（bullwhip effect）：终端需求的小幅波动，沿供应链向上游传播时被逐级放大。数学上定义为：沿供应链每上升一级，订单方差与需求方差之比增大。如果终端需求方差是 σ_D^2，零售商订单方差是 $\sigma_{O_r}^2$，批发商订单方差是 $\sigma_{O_w}^2$，则牛鞭效应指 $\sigma_{O_w}^2 > \sigma_{O_r}^2 > \sigma_D^2$。

1.3 级库存（echelon stock）

级库存 = 某一级的现有库存 + 在途库存 + 该级所有下游的库存之和。例如中心仓的级库存 = 中心仓自己的库存 + 运输在途 + 所有区域仓和门店的库存。用级库存做决策时，中心仓看到下游还有很多货，就不会向工厂多订货——从全局视角抑制盲目补货。对比”本地库存”（只看自己仓库），级库存把下游库存算进”我已经有的货”，避免重复备货。

1.4 order-up-to 策略

order-up-to(订货到水平)策略:每个周期盘点库存 I_t,把库存补到目标水平 S,即订货量 $O_t = \max(S - I_t, 0)$。S 的设定取决于提前期 L、预测需求 $\hat{D}$、安全库存系数 z:

$$S = \hat{D}(L+1) + z\hat{\sigma}\sqrt{L+1}.$$

其中 $\hat{D}(L+1)$ 是提前期加回顾期内的预测总需求，$z\hat{\sigma}\sqrt{L+1}$ 是安全库存。这个公式来自第 8 讲的报童扩展：提前期 L 内需求不确定，要备够 $L+1$ 期的量。

1.5 Clark–Scarf 模型

Clark–Scarf (1960) 是多级库存的经典动态规划解。核心思想：用级库存分解，把 n 级系统拆成 n 个单级问题依次求解。最下游（门店）面对真实需求，解报童问题；上一级（区域仓）面对的是门店的订单，但门店订单被门店策略扭曲过——Clark–Scarf 证明了在一定条件下，最优策略仍然是 order-up-to，只是目标水平要用级库存来算。

为什么多级比单仓难

单仓报童只面对一个需求分布。多级系统里，每一级的”需求”其实是下游的**订货**，而订货被下游的库存策略扭曲过——你看到的不是真实终端需求，而是被加工过的信号。所以多级优化的核心是：别被扭曲信号骗了。

举个具体例子：终端顾客每天买 100 件，但零售商用 order-up-to 策略，看到某天卖了 120 件就向仓库订 150 件（因为要补安全库存）。仓库看到订单 150，以为需求涨到了 150，向工厂订 200 件。工厂看到 200，开足马力。但终端真实需求只有 100——工厂多生产的 100 件变成了库存。这就是”你看到的需求不是真需求”。

2. 完整推导

2.1 牛鞭效应的四个成因（Lee 等 1997）

1. **需求信号处理**：每级把下游订单当需求预测，用移动平均 + 安全库存外推，层层加边际。终端需求涨了 10%，零售商预测涨了 10%，但 order-up-to 策略让它多订了 $(L+1)$ 倍的增量。
2. **批量订货**：凑整车才发，订单脉冲式，比真实需求抖。终端每天卖 100 件，零售商凑一车 500 件订一次——订单从”每天 100”变成”每 5 天 500”，方差暴涨。
3. **价格波动**：促销导致囤货，订单锯齿形。降价时零售商大量囤货，促销结束后订单骤降——订单波动远超真实消耗波动。
4. **短缺博弈**：缺货时零售商多下单抢货，工厂看到虚高订单。工厂扩产之后，零售商发现货够了就取消多余订单——工厂产能闲置。

2.2 order-up-to 策略为何放大方差

设提前期为 L、回顾期 1 期。零售商用最近 N 期平均预测需求 $\hat{D}$、样本标准差 $\hat{\sigma}$，把库存补到

$$S = \hat{D}(L+1) + z\,\hat{\sigma}\sqrt{L+1}.$$

订单 $O_t = S - I_t$。问题出在 $\hat{D}$ 是**滑动窗口估计**：当终端需求突然上升，预测滞后地追上来，订单 O_t 必须一次性补上预测增量加上安全库存增量——这个”一次性补”的动作让订单方差超过需求方差。提前期 L 越长，$(L+1)$ 倍放大越狠。这就是牛鞭效应的数学根源：**正提前期 + 滑动预测 + order-up-to，三者叠加必然放大**。

具体推导：设第 t 期需求突然比之前高了 Δ。滑动窗口预测 $\hat{D}_t$ 慢慢追上（每期只更新一个窗口位置），但目标库存 $S_t = \hat{D}_t(L+1) + \cdots$ 一下子要补 $(L+1)\Delta$ 的预测增量。订单 $O_t = S_t - I_t$ 就要一次性下这个增量。终端需求只涨了 Δ，但订单涨了约 $(L+1)\Delta$——这就是方差放大的数学机制。L 越大，乘数 $(L+1)$ 越大，放大越狠。

2.3 级库存策略（Clark–Scarf）

把”级库存”定义为本级库存 + 在途 + 所有下游库存之和。策略：当级库存降到某点就订货。好处：把下游库存算进本级”账面库存”，避免本级盲目补货。中心仓看到门店还有很多货，就不向工厂订货——从源头掐断放大。

对比本地库存策略：零售商报童只看门店自己的库存。级库存策略下，中心仓做决策时把所有门店的库存也算进去——如果门店库存总量还很高，中心仓就不向工厂补货。这消除了”中心仓不知道下游还有多少货，所以多备”的问题。Clark–Scarf 证明了在一定条件下（同级仓库独立、需求平稳），级库存策略是最优的。

2.4 破解办法

- **信息共享**：让工厂直接看终端 POS 数据，不通过订单层层传话。这直接消除了”需求信号处理”这个成因——工厂看到的是真实需求，不是被零售商策略扭曲过的订单。
- **CPFR（协同计划预测补货）**：上下游联合预测、联合补货。零售商和工厂共享预测模型，用同一个预测值，消除各自加安全库存的叠加效应。
- **VMI（供应商管理库存）**：供应商直接管下游库存，消除下游订货博弈。零售商不再自己下单，工厂根据实际销售数据主动补货——短缺博弈消失，因为零售商不能虚报订单。
- **缩短提前期、减小批量**：直接减小放大系数 $(L+1)$。提前期从 2 周缩到 2 天，牛鞭比从 3 降到 1.1。批量从一车 500 件降到每天 100 件，订单脉冲变平滑。

2.5 多级网络优化是 MIP

实际供应链是网络：多工厂、多中心仓、多区域仓、多门店。优化目标 = 运输 + 库存 + 缺货总成本最小。决策变量包括：开哪些仓（0-1）、每级备多少库存（连续）、运输量多少（连续）。这是第 3 讲的混合整数规划，用 branch-and-cut 求解。

具体建模：设 $y_i \in \{0,1\}$ 表示是否开设仓库 i，x_{ij} 表示从仓库 i 到门店 j 的运输量。目标 $\min \sum_i f_i y_i + \sum_{ij} c_{ij} x_{ij}$ + 库存成本，约束包括：每个门店需求必须被满足 $\sum_i x_{ij} = d_j$，每个仓库的运输量不超过其处理能力 $\sum_j x_{ij} \le C_i y_i$（若不开仓则 $y_i = 0$，运输量也为 0）。这就是一个典型的设施选址 + 运输联合 MIP。

这个 MIP 的规模：100 个仓库候选 + 1000 个门店，变量数 = 100（开仓 0-1）+ 100000（运输量连续）。约束数 = 1000（需求）+ 100（容量）。现代求解器（Gurobi、CPLEX）能解这种规模，但要分钟级。如果加上库存决策变量，规模再翻一倍。

3. 直觉理解

> **直觉**：牛鞭效应像甩鞭子——手腕轻轻抖一下，鞭梢抖得厉害。零售商看到终端卖了 100 件，觉得”可能要涨”，向仓库订 120 件；仓库看到订单 120，向工厂订 150 件；工厂看到 150，开足马力生产 200。信息每经一级被放大。破解办法是信息共享——让工厂直接看到终端需求，别通过订单层层传话。

另一个直觉：牛鞭效应不是”大家预测不准”造成的——每一级的预测其实都是无偏的（滑动窗口均值收敛到真实均值）。问题是 order-up-to 策略把”预测更新”放大成了”订单脉冲”。即使每级预测都准，只要提前期 > 0、策略是 order-up-to，放大就必然发生。这是结构性的，不是人的问题。

4. 常见误区

> **误区一：以为库存越多越安全**。多级网络里库存分布错了（上游堆太多、下游不够）反而更危险。
>
> **误区二：把单仓报童直接套到每一级**。多级要考虑级库存和上下游联动。
>
> **误区三：以为牛鞭是”信息延迟”造成的**。主要是信息扭曲——每级在订单里加了自己的猜测，不是单纯慢。

5. 代码实现

用 order-up-to 策略模拟牛鞭效应，并对比信息共享。

```
import numpy as np

np.random.seed(0)
T = 500                                    # 模拟 500 个周期
mu = 100                                   # 终端需求均值 100/期
```

```
demand = mu + np.random.normal(0, 10, T) # 终端需求，标准差 10（平稳正态）

# 零售商 order-up-to 策略：
# - 提前期 L=2：订单发出后 2 期才到货
# - z=1.0：安全库存系数（约 84% 服务水平）
# - N=5：滑动窗口长度，用最近 5 期数据预测均值和标准差
L, z, N = 2, 1.0, 5
inv = 100.0                                  # 初始库存
orders = []
for t in range(N, T):
    # 滑动窗口预测：用最近 N 期需求的均值和标准差
    fc = demand[t-N:t].mean()
    sd = demand[t-N:t].std()
    # 目标库存水平 S = 预测需求 *(L+1) + z* 标准差 *sqrt(L+1)
    # (L+1) 的来源：要覆盖" 提前期 L 期 + 当前期 1 期" 的总需求
    S = fc*(L+1) + z*sd*np.sqrt(L+1)
    order = max(S - inv, 0)                  # 订货量 = 目标 - 当前库存（不能负订）
    inv += order - demand[t]                 # 库存更新：到货 + 订货量，消耗 - 当期需求
    orders.append(order)
orders = np.array(orders)

# 对比：终端需求方差 vs 零售商订单方差
d_var = demand[N:].var()                     # 终端需求方差
o_var = orders.var()                         # 零售商订单方差
print(" 终端需求方差:", round(d_var, 2))
print(" 零售商订单方差:", round(o_var, 2))
print(" 牛鞭比 (订单方差/需求方差):", round(o_var/d_var, 2), " >1 即放大")

# 信息共享场景：工厂直接看终端 POS 数据，不看零售商的扭曲订单
# 此时工厂的" 订单" 就是真实需求本身，牛鞭比应 = 1
shared = demand[N:]
print(" 信息共享后工厂订单方差:", round(shared.var(), 2),
      " 牛鞭比:", round(shared.var()/d_var, 2))
```

2.6 牛鞭效应的量化对比

在上面的代码模拟中，我们期望看到：- 终端需求方差约为 100（标准差 10）。- 零售商订单方差 > 终端需求方差（牛鞭比 > 1）。- 信息共享后，工厂直接看终端需求，牛鞭比 ≈ 1。

这个模拟展示了”正提前期 + 滑动预测 + order-up-to”三者叠加的放大效应。如果把提前期 L 从 2 改成 0（即期补货），放大效应会消失——因为 $L = 0$ 时目标库存 $S = \hat{D} + z\hat{\sigma}$，不需要补 $(L + 1)$ 倍的预测增量。这也解释了为什么”缩短提前期”是破解牛鞭的最直接手段。

6. 小结

1. 多级库存要考虑级库存，不能单仓独立优化。
2. 牛鞭效应：正提前期 + 滑动预测 + order-up-to 叠加，使订单方差 > 需求方差。
3. 破解：信息共享、CPFR、VMI、缩短提前期。
4. 多级网络优化是 MIP（选仓 + 库存 + 运输联合）。

局限与延伸

- 本讲模拟假设需求平稳正态；季节性、趋势、缺货博弈会让放大更复杂。
- 延伸阅读：Lee–Padmanabhan–Whang (1997) 牛鞭效应四成因；Clark–Scarf (1960)；Sterman 啤酒游戏。

补充：啤酒游戏（Beer Game）是 MIT Sloan 的经典课堂模拟——4 个角色（零售商、批发商、分销商、工厂）各做库存决策，看不到终端需求只看下一级订单。几乎所有班级都会玩出牛鞭效应：订单波动逐级放大，最后工厂产能过剩或缺货。这个游戏直观展示了”信息扭曲”比”信息延迟”更致命。

7. 思考题

基础档

1. 为什么提前期 L 越长，牛鞭比越大？从目标库存公式 $S = \hat{D}(L+1) + \cdots$ 解释。
2. 信息共享怎么把工厂这级的牛鞭比降到 1？

进阶档

3. 批量订货（凑整车）为什么会放大牛鞭？它和滑动预测放大的机制有何不同？
4. 级库存和本地库存的区别是什么？为什么用级库存能抑制盲目补货？

开放档

5. 大模型推理服务里，各 GPU 队列相当于”多级库存”。集中式调度器（全局看请求）相当于哪种牛鞭破解手段？
6. 如果上下游无法共享数据（竞争、隐私），有什么替代手段抑制牛鞭？

第 20 讲 定价与收益管理：动态价格的最优结构

> **本讲问题**：航班机票为什么越临期越贵？酒店为什么周末涨价？这不是”宰客”，而是收益管理——在固定容量下，动态定价让每一单位容量卖给出价最高的人。本讲看动态定价的数学结构，以及为什么它是运筹问题而非纯市场行为。

1. 数学工具/预备知识

- **需求函数**：$D(p)$，价格越高需求越低，$D'(p) < 0$。
- **价格弹性**：$\epsilon = \dfrac{p}{D}\dfrac{dD}{dp}$，需求对价格的相对敏感程度。
- **动态定价**：随时间/库存调整价格 p_t。
- **收益管理（revenue management）**：固定容量下，通过价格和配额最大化收益。
- **残值为零**：机票飞了空座就没价值——这是收益管理成立的前提。

弹性的含义

$|\epsilon| > 1$ 是弹性需求（奢侈品、有替代品），降价能增收；$|\epsilon| < 1$ 是刚需（处方药），提价能增收。最优加价率和弹性成反比——这是本讲第一个核心公式。

2. 完整推导

2.1 单期定价

给定需求 $D(p)$、单位成本 c，利润 $\pi(p) = (p-c)D(p)$。对 p 求导令零：

$$\pi'(p) = D(p) + (p-c)D'(p) = 0.$$

即

$$p^* = c - \frac{D(p^*)}{D'(p^*)}.$$

含义：最优售价 = 成本 + 加成，加成由需求曲线形状 D/D' 决定。

2.2 弹性定价

代入弹性定义 $\epsilon = \frac{p}{D}D'$，从最优条件 $D + (p-c)D' = 0$：

$$(p-c)\frac{D'}{D} = -1 \;\Rightarrow\; \frac{p-c}{p}\cdot\frac{pD'}{D} = -1 \;\Rightarrow\; \frac{p-c}{p} = \frac{1}{|\epsilon|}.$$

最优加价率 $(p-c)/p = 1/|\epsilon|$。弹性越大（有替代品），加价率越低；弹性越小（刚需），加价率越高。

2.3 动态定价的 Bellman 结构

多期、库存有限 I。每期定价 p_t，需求 $D_t(p_t)$，卖一单位库存减一。价值函数：

$$V_t(I) = \max_{p_t}\Big\{p_t\min(D_t(p_t), I) + V_{t+1}\big(I - \min(D_t(p_t), I)\big)\Big\}.$$

含义：当期收入 $p_t \cdot \min(\text{需求}, \text{库存})$，加上剩余库存的未来价值。由它推出结构：库存多、距期末远 → 降价引流；库存紧、临期 → 抬价收割高支付意愿者。这解释了酒店周末（需求高、库存紧）涨价。

2.4 Littlewood 折扣配额法则

航班：商务旅客临期高价 p_h、休闲旅客提前低价 p_l。设全价舱需求为随机变量 D_h，要给折扣舱留多少配额。边际逻辑：把最后一个座位给折扣舱，稳赚 p_l；留给全价舱，期望赚 $p_h \cdot P(\text{全价舱卖到这个座})$。两者相等时最优：

$$p_l = p_h \cdot P(D_h > q^*) \;\Rightarrow\; P(D_h > q^*) = \frac{p_l}{p_h}.$$

若折扣票 200、全价 1000，则 $P(D_h > q^*) = 0.2$——只要还有 20% 概率全价舱能卖到这个座，就该留给全价；否则提前低价卖。

2.5 动态定价 vs 静态定价

动态定价的收益提升来自**价格歧视**：对支付意愿高的人收高价、对低的人收低价。前提是产能固定、残值为零。生鲜临期反而降价，是因为它残值为负（会变质）——方向相反，因为不降价的代价是扔掉，而非空飞。

3. 直觉理解

直觉：定价像调节阀——库存多、时间多，就降价引流；库存少、临期，就抬价收割高支付意愿的人。收益管理在”卖得掉”和”卖得贵”之间找最优。航空复杂在同时有多个票价等级、要在它们之间分配座位。这不是宰客，而是把固定产能的残值最大化——空座飞掉就是零，不如低价卖出去。

4. 常见误区

误区一：把成本加成定价当最优。成本加成忽略需求弹性，可能定价过高卖不动、或过低亏了。

误区二：以为机票临期贵、生鲜临期便宜是矛盾。不矛盾：机票残值为零（空座 =0），生鲜残值为负（变质 = 倒贴）。最优方向由残值符号决定。

误区三：以为动态定价只适用航空酒店。网约车、电商大促、演唱会门票都是——只要产能固定、需求随时间波动。

5. 代码实现

数值求解单期最优定价，并验证弹性加价公式。

```
import numpy as np
from scipy.optimize import minimize_scalar

# 需求 D(p)=100-2p，成本 c=10
def neg_profit(p):
    return -(p - 10) * (100 - 2*p)

r = minimize_scalar(neg_profit, bounds=(10, 50), method="bounded")
p_star = r.x
print(" 最优价格 =", round(p_star, 2), " 最大利润 =", round(-r.fun, 2))

# 解析验证：100-2p+(p-10)(-2)=0 -> 120-4p=0 -> p=30
D_star = 100 - 2*p_star
elasticity = p_star/D_star * (-2)
print(" 解析最优价格 = 30")
print(" 在 p* 处需求量 =", round(D_star, 1))
print(" 弹性 epsilon =", round(elasticity, 3))
print(" 加价率 (p-c)/p =", round((p_star-10)/p_star, 3),
      " 1/|epsilon| =", round(1/abs(elasticity), 3), "(应相等)")
```

预期 $p^* = 30$、利润 800、弹性 -1.5、加价率 $2/3 \approx 0.667 = 1/1.5$，公式自洽。

再演示 Littlewood 法则：全价需求服从正态，求配额 q^*。

```
from scipy.stats import norm
p_l, p_h = 200, 1000
# 全价舱需求 D_h ~ N(mu=80, sigma=30)
mu_h, sig_h = 80, 30
target = p_l / p_h                        # P(D_h > q*) = p_l/p_h = 0.2
# P(D_h > q*) = 1 - Phi((q-mu)/sig) = target  => Phi((q-mu)/sig) = 1-target
q_star = mu_h + sig_h * norm.ppf(1 - target)
print(f"Littlewood 配额 q* = {q_star:.1f}（全价舱保留这么多座）")
```

预期 $q^* \approx 80 + 30 \times \Phi^{-1}(0.8) \approx 80 + 30 \times 0.84 \approx 105$。

6. 小结

1. 单期最优定价由边际收益 = 边际成本决定。
2. 加价率 = 1/| 弹性 |。
3. 动态定价随库存和时间调整，Bellman 方程给出结构。
4. Littlewood 法则给折扣舱配额 $P(D_h > q^*) = p_l/p_h$。

局限与延伸

- 动态定价涉及公平性、用户反感；实际系统要加价格边界、防歧视。
- 延伸阅读：Talluri–van Ryzin《The Theory and Practice of Revenue Management》；Littlewood (1972)；动态定价在网约车的文献。

7. 思考题

基础档

1. 弹性 $\epsilon = -2$ 时最优加价率是多少?
2. 为什么机票越临期越贵、生鲜越临期越便宜？用残值符号解释。

进阶档

3. 需求 $D(p) = 200 - 4p$、成本 10，求最优价格。为什么它和 $D = 100 - 2p$ 的最优价格一样?

4. Littlewood 法则中，若全价需求方差变大，配额 q^* 怎么变?

开放档

5. 大模型推理算力是固定产能、残值为零（闲时不产生收入）。能不能对不同请求动态定价？公平性约束怎么加?
6. 动态定价和”大数据杀熟”的边界在哪？运筹模型如何被约束到合规区间?

第 21 讲 用运筹框架读论文

> **本讲问题**：拿到一篇运筹/优化论文，几十页公式，从哪读？大多数人从头读到尾，三页就放弃。其实任何运筹论文都可以拆成四个零件：**决策变量、目标、约束、不确定性**。把这四个零件抽出来，论文骨架就清楚了。本讲给你一张”四要素阅读卡”，以后读论文按图索骥。

1. 数学工具/预备知识

- **四要素**：决策变量（做什么决策）、目标（优化什么）、约束（不能违反什么）、不确定性（哪里有随机/鲁棒）。
- **拆解顺序**：先看问题设定，再看数学模型，最后看算法。
- **论文骨架**：问题 → 模型 → 算法 → 实验。
- **复杂度声明**：论文会标注多项式 / NP-hard / 近似比，这决定算法定位。

为什么是这四要素

运筹学的全部问题都可以写成一句话：“在某种资源限制和不确定性下，选一组决策让目标最好。”四要素恰好对应这句话的四个名词——决策（选什么）、目标（让什么最好）、约束（限制）、不确定性（哪里随机）。抓不住这四点，就被公式细节淹没。

2. 完整推导

2.1 四要素阅读法

读任何优化论文，先回答四个问题：

1. **决策变量**：要优化的量是什么？连续（库存水平）、整数（选址开关）、还是组合（路径排列）？变量类型决定用 LP、MIP 还是组合算法。
2. **目标函数**：最小化成本？最大化收益？多目标？单目标还是多目标，决定问题难度。
3. **约束**：资源上限、需求满足、平衡约束？哪些硬、哪些软？约束紧密度决定可行域大小。
4. **不确定性**：需求确定还是随机？随机规划、鲁棒优化、还是机会约束？处理方式决定方法论分支。

2.2 拆解示范：经典报童

- 决策变量：订货量 Q（单变量连续）。
- 目标：最大化期望利润 $E[b\min(D,Q)-c_o(Q-D)^+]$。
- 约束：$Q\geq 0$。
- 不确定性：需求 D 随机，分布 F 已知。

四要素一列，立刻知道这是”单变量、单约束、随机需求”问题，核心是求临界分位数 $F(Q^*)=c_u/(c_u+c_o)$（第 8 讲）。

2.3 拆解示范：TSP 论文

- 决策变量：$x_{ij}\in\{0,1\}$，是否走 $i\to j$。
- 目标：最小化总距离 $\sum c_{ij}x_{ij}$。
- 约束：出度 = 1、入度 = 1、子环消除。
- 不确定性：无（确定型）。

这是纯组合优化问题，核心是消子环、找哈密顿回路（第 9 讲）。

2.4 拆解示范：鲁棒库存论文

- 决策变量：订货量 Q。
- 目标：最坏情况下的期望利润（max-min，不是期望）。
- 约束：$Q \geq 0$，需求落在不确定集 $\mathcal{U}$ 内。
- 不确定性：需求 $\tilde{D} \in \mathcal{U}$，鲁棒优化（第 12 讲）。

和报童的区别：目标从”期望”变”最坏情况”，不确定性从”已知分布”变”已知不确定集”。这就是放松了”分布已知”假设。

2.5 阅读顺序建议

1. 先读 abstract + introduction，知道”解决什么问题”。
2. 跳到模型章节，抽四要素。
3. 看算法章节，知道”怎么解”。
4. 看实验，知道”效果好不好”。
5. 最后读相关工作和结论。

不要从引言逐字读——引言常写得宏大，模型才是核心。

2.6 读完四要素后，怎么判断值不值得深读

四要素抽完后，问三个问题决定要不要精读：

1. **问题重要吗？** 是你关心的领域、有真实应用？还是玩具问题？
2. **方法你能复现吗？** 是经典 LP/MIP（你能跑），还是依赖你没有的专有求解器/数据？
3. **贡献在第几格？** 如果只在”算法更快”一格动，且和你方向无关，可以略读。

只有三问都过，才值得逐行读证明。

2.7 四要素对比表：快速定位贡献

拿到三篇报童论文做对比：

论文	决策变量	目标	约束	不确定性
经典报童	Q	期望利润最大	$Q \geq 0$	分布已知
鲁棒报童	Q	最坏情况利润最大	$Q \geq 0$	不确定集
多产品报童	Q_1, Q_2	期望利润最大	预算约束	分布已知

一眼看出：第三篇改了约束（加预算），前两篇改了不确定性。论文之间的区别，往往就在四要素的某一格。

与 AI/大模型的联系

四要素框架也适用于读 AI 论文：决策变量是”模型参数”、目标是”损失函数”、约束是”正则化/显存”、不确定性是”数据分布偏移”。用它读 Transformer、RLHF、检索增强生成，能快速抓住每篇论文在四要素里改了哪个零件。

3. 直觉理解

> **直觉**：四要素像论文的 DNA 序列。不管包装成”深度学习 + 优化”还是”大模型 + 决策”，底层都是这四个零件。抽出零件，就能判断论文到底在改什么——换了目标？加了约束？还是处理了新的不确定性？这比逐字读快十倍。

4. 常见误区

误区一：从引言开始逐字读。引言常写得宏大，模型才是核心。

误区二：跳过模型直接看实验。不知道模型，就看不懂实验为什么这么设计。

误区三：只看算法不看问题。算法再花哨，问题不重要，论文价值也有限。读论文先判断问题值不值得解。

5. 代码实现

本讲无代码，但给一张可复用的”四要素阅读卡”模板。下次拿到论文，先填这张卡再决定深读与否。

```
# 论文四要素阅读卡

## 题目：
## 决策变量：（连续 / 整数 / 组合？维度多大？）
## 目标函数：（min 还是 max？单目标还是多目标？）
## 约束：（资源上限 / 平衡 / 硬约束还是软约束？）
## 不确定性：（确定 / 随机规划 / 鲁棒 / 机会约束？）
## 算法：（精确 / 近似 / 启发式 / 学习？）
## 复杂度与保证：（多项式 / NP-hard / 近似比？）
## 实验亮点：（标准实例 / 真实数据 / 消融？）
## 这篇论文改了四要素里的哪一格？
## 我的疑问：
```

6. 小结

1. 任何优化论文都可拆成决策变量、目标、约束、不确定性四要素。
2. 阅读顺序：摘要 → 模型 → 算法 → 实验。
3. 四要素对比表帮你快速定位论文贡献在第几格。
4. 不确定类型（随机/鲁棒/机会约束）决定方法论分支。

局限与延伸

- 四要素框架抓骨架，但读证明细节仍需逐行；框架用于筛”要不要深读”。
- 延伸阅读：读 OR 论文的方法论文章；运筹顶会（INFORMS、Math Programming）的 problem formulation 章节写法。

7. 思考题

基础档

1. 拿一篇你最近读的优化论文，填一张四要素卡。
2. 一篇论文把”确定需求”改成”随机需求”，它改了四要素里的哪个？

进阶档

3. “多目标优化”在四要素里是改目标还是改约束？为什么？
4. 怎么判断一篇论文的核心贡献是算法还是模型？（提示：看它在四要素哪一格动得最多。）

开放档

5. 用四要素拆解一篇你读过的 Transformer/RLHF 论文，它的”约束”和”不确定性”对应什么？
6. 当一篇论文四要素都没动、只换了算法求解器，这种贡献为什么容易被审稿人质疑？

第 22 讲 找创新点的方法论：从缺口到论文

本讲问题：研究生最愁的不是写论文，而是”不知道写什么”。创新点不是拍脑袋想出来的，而是有方法论的——从已有论文里找”它没说的地方”。本讲给四类常见缺口，以及怎么把缺口变成可写的论文。读完你会发现：创新点不是灵光一现，而是系统搜索的结果。

1. 数学工具/预备知识

找创新点不是灵感，而是系统搜索。下面把本讲用到的分析框架展开讲清楚。

1.1 四类缺口的精确含义

1. **模型缺口**：现有模型假设太强，放松它就长出新问题。例如报童模型假设需求分布已知——放松到”分布未知”就引出鲁棒报童；假设单期——放松到多期就引出动态库存。判断标准：放松假设后，原模型的结论是否不再成立？如果不再成立，就需要新方法。
2. **算法缺口**：现有算法在某个指标（时间、内存、最优性 gap）上不够好。找瓶颈：求解器在哪一步最慢？是 LP 松弛太慢还是分支定界节点太多？瓶颈在哪，改进就在哪。例如 TSP 的 branch-and-cut 在 1000 城实例上耗时 1 小时，瓶颈在分支变量选择——用 ML 预测好的分支变量就能加速。
3. **数据缺口**：现有论文都用合成数据，没人用真实数据验证。找一个公开但被忽略的数据集，用现有模型跑一遍，分析新现象。这种论文的价值在实证发现，不在新算法。
4. **场景缺口**：现有模型在 A 场景好用，没在 B 场景试过。把成熟方法迁移到新场景，关键是识别新场景的特殊约束——这些约束让原模型不能直接套。

1.2 可证伪猜想

一个科学猜想必须能被实验证明错。波普尔的证伪主义：不能被证伪的命题不是科学。好的猜想形如”当 X 条件满足时，Y 量会变化 Z%“——你可以做实验看 Y 是否真的变了 Z%。差的猜想形如”大概会变好”——变好多少？怎么算好？无法证伪。

1.3 假设清单法

读 10 篇同主题论文，把每篇的 5 个核心假设列成表。统计每个假设出现的次数。出现次数 ≤ 2 的假设就是”重叠度最低”的方向——大多数论文都没碰它，说明它可能是缺口。但要注意：没人做可能是因为太难或没价值，要先判断可解性。

为什么”放松假设”是最高产的创新

成熟问题（报童、TSP、库存）已经被解透了，从零发明新问题极难。但每篇论文都在某个假设上做了简化（需求分布已知、单期、离线、确定）。放松一个假设，旧模型不再适用、需要新方法——这就自然长出一篇论文。这条路风险低、文献基础厚，是研究生最稳的创新路径。

为什么风险低？因为你站在已有文献的肩膀上：你知道原模型怎么解、它的瓶颈在哪、它的实验怎么设计。放松一个假设后，原模型变成你的 baseline——你不用从零开始建实验，只需要在原实验上加一个”放松条件”。相比之下，从零开一个新问题，你要自己定义问题、自己建 baseline、自己设计评价指标——成本高十倍。

为什么高产？因为一个成熟问题的假设空间是有限的。报童模型的假设清单：需求分布已知、单期、单产品、无提前期、无容量约束、无预算约束……每个假设放松一次，就是一篇潜在论文。这个”假设空间”是一棵决策树，每放松一个假设就长出一个新分支。

2. 完整推导

2.1 模型缺口

例子：报童假设需求分布已知。缺口：分布未知时怎么办？引出鲁棒报童、分布鲁棒优化（第 8、12 讲）。放松方向：

- 确定需求 → 随机需求；
- 已知分布 → 未知分布（鲁棒）；
- 单期 → 多期；
- 单产品 → 多产品。

为什么放松假设是”最高产”的？因为每个假设都是一篇潜在论文的入口。成熟问题（报童、TSP、最短路径）已经被解透了，但它们的假设集合是有限的——每个假设放松一次，就长出一个子问题。这个子问题有文献基础（你知道原模型怎么做的），有明确的对比基线（原模型就是 baseline），有可衡量的改进（放松后成本降低多少）。风险低、路径清晰。

2.2 算法缺口

例子：TSP branch-and-cut 解 1000 城市要 1 小时。缺口：能不能用 ML 预测初始解加速？引出 Learning to Branch（第 13 讲）。动作：找现有算法的瓶颈（时间、内存、最优性 gap），瓶颈在哪改进就在哪。

怎么找瓶颈？profile 你的算法：把时间分解成各阶段（LP 松弛、分支定界节点数、割平面生成），看哪步占 80% 时间。如果 LP 松弛占大头，加速方向是用更松但更快的松弛；如果分支定界节点数太多，加速方向是更好的分支策略或更强的割平面。不要在只占 5% 时间的地方优化——那是浪费。

2.3 数据缺口

例子：所有人都用合成 TSP 实例。缺口：真实城市 TSP（带山路、单行道）没人算过。动作：找一个公开但被忽略的数据集，用现有模型跑一遍，分析新现象。这种论文易发，因为填补了实证空白。

数据缺口的论文不一定需要新算法——它的贡献在”发现新现象”。例如：在真实物流数据上跑经典 VRP 模型，发现最优解结构和合成数据上完全不同（因为真实路网有很多非对称边），由此提出修正模型。这类论文的关键是：你跑出来的新现象必须是别人没注意到的，而不是”我在新数据上复现了老结论”。

2.4 场景缺口

例子：库存模型都是线下零售。缺口：共享充电宝的库存？共享单车调度？动作：把成熟方法迁移到新场景，写”场景适配”。关键是新场景有什么特殊约束——充电宝要考虑归还随机性，不能直接套报童。

场景缺口和”应用论文”的区别：应用论文是”把 A 方法用到 B 场景”，场景缺口是”B 场景的特殊约束让 A 方法不适用，需要修改”。前者是迁移，后者是改造。改造才是创新——迁移别人一眼就能看到，改造需要你识别出原模型哪个假设不成立、怎么放松。

2.5 从直觉到可证伪猜想

看到缺口后，先写一个**猜想**：“我觉得放松这个假设后，最优解会变成 X”。猜想要可证伪——不能是”大概会变好”。好的猜想形如：“当需求分布偏离假设超过 ϵ 时，鲁棒报童的最优订货量比经典报童低 $z\sigma$，且库存成本降低 $\alpha\%$。”如果实证发现耦合效应不显著，猜想就被证伪——这同样是有价值的科学结论。

怎么写好一个可证伪猜想？三步：

1. **指明自变量**：什么条件变了？（如”需求分布的偏度从 0 变成 1”）
2. **指明因变量**：什么量会变？（如”最优订货量”或”期望成本”）
3. **指明方向性和量级**：变大还是变小？大概多少？（如”降低 5%~10% “）

缺任何一步都不算可证伪。“性能会变好”不可证伪——变好多少？在什么条件下？“在偏度 > 0.5 时，鲁棒报童的期望成本比经典报童低 8%±2%”才可证伪——你可以跑实验看是不是 8%。

2.6 缺口识别清单

读论文时问：

- 它的假设是什么？现实里成立吗？
- 实验在哪类数据上做的？换数据还成立吗？
- 算法时间复杂度？瓶颈在哪？
- 问题是静态还是动态？动态扩展过吗？
- 不确定性怎么处理？分布已知还是鲁棒？

每个问题对应一类缺口。第一问找模型缺口（假设不现实就放松），第二问找数据缺口（换数据集验证），第三问找算法缺口（瓶颈处加速），第四问找动态扩展缺口（静态变动态），第五问找不确定性处理缺口（已知分布变鲁棒）。把这五个问题当读论文的 checklist，每篇论文读完都过一遍。

2.7 完整示范：从假设清单到选题

读 10 篇 TSP 论文，列假设：10 篇都是欧氏距离（三角不等式成立）、9 篇静态需求、8 篇单商品、7 篇离线求解。重叠度最低的是”离线求解”——只有 3 篇做过在线 TSP。选题：“在线 TSP 中，新城市动态出现时，怎么用竞争比（第 11 讲）分析给出路由策略？”这就是一个明确方向。

这个示范的关键步骤：

1. **选领域**：TSP——成熟问题，文献厚，放松一个假设就能写。
2. **列假设表**：每篇论文 5 个假设，10 篇就是 50 个假设条目。
3. **数频率**：欧氏距离 10/10（太常见，不是缺口），离线求解 3/10（少见，可能是缺口）。
4. **验证可解性**：在线 TSP 已有竞争比理论（第 11 讲），说明问题是可分析的，不是无底洞。
5. **写出猜想**："当新城市以泊松过程到达时，贪心策略的竞争比不超过 2。"

与 AI/大模型的联系

找 AI 论文创新点同理：列 10 篇大模型推理优化论文的假设，找重叠度最低的。比如"假设请求长度固定"是 8 篇都做的假设，只有 2 篇处理变长请求——这就是缺口方向。

和传统运筹不同的是，AI 领域的"假设"更多在工程层面："假设单 GPU""假设 batch 内请求同长""假设无 SLA"。这些假设放松后，问题往往从凸变非凸、从确定变随机，需要新的算法和理论工具。

实操建议：找 AI 方向的缺口时，别只盯 NeurIPS 论文——也看工业界的技术博客（如 Google、Meta 的 ML 系统博客）。工业界遇到的真实约束（成本、延迟、GPU 显存）往往是学术界论文忽略的，而这些忽略的约束就是缺口。

2.5 一次完整的缺口搜索流程

把前面的步骤串起来：

1. 选一个你熟悉的领域（如库存管理）。
2. 找最近 2 年 10 篇顶会论文。
3. 每篇列 5 个假设，列成表。
4. 统计每个假设出现次数，找出现 ≤ 2 的。
5. 判断那个假设放松后问题是否可解、是否有价值。
6. 写一句可证伪猜想。

这个流程的关键是第 4 步——不要找"完全没人做过"的方向，要找"有人做过但做得少"的方向。完全没人做过的方向可能是因为太难或没价值；做得少的方向说明有空间但不至于无从下手。

> **直觉**：找创新点像考古——不是挖新坑，而是在已有坑边上找"还没挖透"的边角。每篇论文都在某个假设上做了简化，那个简化就是下一篇论文的入口。读 10 篇同主题论文，把假设列出来，重叠度最低的那个，就是你最可能的创新方向。这是系统搜索，不是灵感。

这个比喻的关键：考古学家不是随便找个地方就挖——他们先看已有挖掘点的分布图，找"已挖掘区域的边缘"。创新点也一样：不是在空白领域里碰运气，而是在已有研究的边缘找"还没覆盖到"的地方。

4. 常见误区

> **误区一：以为创新要"从零发明新问题"**。大多数好论文是在已有问题上放松一个假设，不是开新领域。
>
> **误区二：追热点**。 AI+ 优化热点多，但能发论文的是有 rigor 的那篇，不是蹭热度那篇。
>
> **误区三：以为缺口就是"没人做过"**。没人做过的方向可能因太难或没价值。要找"有人做过但做不深"的方向。

5. 代码实现

本讲无代码，给一张"缺口搜索清单"模板。

```
# 缺口搜索清单

## 第一步：选一个问题（如大模型推理调度）
## 第二步：找最近 2 年 10 篇顶会论文
## 第三步：列每篇的 5 个假设
1. 论文 A：假设请求长度固定
2. 论文 B：假设泊松到达
```

```
3. 论文 C：假设单 batch
4. 论文 D：假设无 SLA
...
## 第四步：找出出现次数 <= 2 的假设（重叠度最低）
## 第五步：判断放松它后问题是否可解、是否有价值
## 产出：一句可证伪猜想
```

使用说明：第三步的假设要具体到”如果不满足这个假设会怎样”。例如”假设请求长度固定”——如果请求长度可变，原来的 batch 策略还最优吗？如果不最优，就需要新策略。第五步的判断标准：放松后问题是否仍是多项式可解的？如果变成 NP-hard 了，你的贡献应该落在”近似算法”而非”精确解”。

6. 小结

1. 四类缺口：模型、算法、数据、场景。
2. 创新点 = 放松一个已有假设（风险最低的高产路径）。
3. 猜想要可证伪。
4. 读论文列假设清单，找重叠度最低的方向。

局限与延伸

- 放松假设可能让问题从可解变 NP-hard；要先评估可解性再投入。判断方法：先看放松后问题的结构是否保留——如果原问题的凸性消失了，可能需要近似算法；如果原问题的多项式算法还能用，那改进空间有限。
- 延伸阅读：Kuhn–Nash 关于问题选择；顶会（NeurIPS、INFORMS）的 problem-driven 范式；How to write a great paper 的方法论文章。

补充一个实操建议：找创新点时不要只看顶会最新论文，也要看经典论文的”future work”部分。经典论文的作者往往在结尾列出了”我们没做的方向”——那些方向经过 10 年可能已经有人做了，但也可能还没有。如果一个 future work 方向 10 年没人碰，要么是太难，要么是被遗忘了——判断清楚再决定。

另一个实操建议：和导师每周讨论一次”最近读了什么、想到什么缺口”。个人读文献容易陷入盲区，导师的经验能帮你判断”这个缺口是真的还是假的”。

7. 思考题

基础档

1. 选一篇你读过的论文，列它的 5 个假设，找一个能放松的。
2. “场景缺口”和普通”应用论文”有什么区别?

进阶档

3. 为什么”放松假设”比”提出全新问题”更容易发论文？从文献基础角度解释。
4. 怎么判断一个缺口是”真缺口”还是”做不了的假缺口”?

开放档

5. 放松一个假设后问题可能从多项式变 NP-hard，这时候你的创新该落在”新近似算法”还是”特殊结构下的精确解”?
6. 当一个方向被大模型自动发现（LLM 读文献找缺口），人的增值点在哪?

第 23 讲 实验诊断工具箱：从跑通到跑准

> **本讲问题**：算法跑通了，结果图也画了，但审稿人问”你的 baseline 公平吗？““实例生成器是不是挑过数据?”“随机种子控制了吗?”——实验不严谨，理论再好也被拒。本讲给一套标准化实验诊断流水线，让你的实验从”跑通”升级到”跑准”。

1. 数学工具/预备知识

1.1 实验设计三原则

1. **对照（control）**：必须有 baseline 做对比。baseline 必须是公认强方法，不能选弱的当衬托。解 TSP，baseline 不能只选最近邻，要选 LKH、Gurobi。更重要的是，baseline 也要调到最优——用默认参数的 baseline 不公平。
2. **随机（randomization）**：随机种子固定，结果可复现。每次实验记录种子，别人重跑能复现数字。如果算法是随机的（如 RL 训练），要跑多个种子报均值 ± 标准差。
3. **重复（replication）**：每个实验跑 5~10 次取平均。单次结果可能是运气——一个种子好不代表算法好。多次重复后看均值和标准差，才能判断性能。

1.2 最优性 gap

最优性 gap 衡量算法解离最优解有多远：

$$\text{gap} = \frac{f(\text{算法解}) - f(\text{最优解})}{f(\text{最优解})}.$$

gap = 0 说明找到最优解；gap = 5% 说明比最优差 5%。为什么要同时报 gap 和时间？因为”快 2 倍”如果是以 gap 从 0.1% 涨到 5% 换来的，那不是贡献。审稿人要的是”同等质量下快了多少”或”同等速度下质量好了多少”。

1.3 统计显著性

两个方法的均值不同，差异可能是真实的，也可能是随机波动。统计检验（t-test）回答：观察到的差异，有多大可能是纯随机？

- $p < 0.05$：差异由随机造成的概率 < 5%，通常认为显著。
- p 值小不代表效应大——极端大样本下，差 0.1% 也能 $p < 0.001$。所以还要看效应大小（两个均值差多少）。

1.4 实例生成器陷阱

自己造数据自己挑结果好看但不代表真实。常见陷阱：只生成对自己算法有利的参数范围、只报告成功实例、用标准实例时只选表现好的子集。破解：用文献标准实例集（TSPLIB、OR-Library），公开生成器和种子。

为什么 gap 比速度重要

“快 2 倍”如果是以质量换的（gap 5% vs 最优 0.1%），那不是贡献。审稿人最关心的是：你在**同等质量**下快了多少，或**同等速度**下质量好多少。所以实验必须同时报 gap 和时间，不能只报一个。

2. 完整推导

2.1 实验设计三原则

1. **对照**：baseline 必须是公认强方法，不能选弱的当衬托。解 TSP，baseline 不能只选最近邻，要选 LKH、Gurobi。
2. **随机**：随机种子固定，结果可复现。每次实验记录种子，别人重跑能复现数字。
3. **重复**：每个实验跑 5~10 次取均值 ± 标准差。单次结果可能是运气，多次平均才可信。

2.2 实例生成器陷阱

- **陷阱 1**：生成时刻意把参数调到对自己算法有利（只生成密集城市，不生成稀疏城市）。
- **陷阱 2**：只报告成功实例，失败的藏起来。
- **陷阱 3**：用公开标准实例时，只选自己表现好的子集。

破解：用文献标准实例集（TSPLIB、OR-Library），公开生成器和种子，报告所有实例（包括失败的）。

2.3 实验诊断流水线

1. **sanity check**：toy 实例上手算最优解，验证算法对。比如 4 城市 TSP 手算最优，跑算法看是否一致。如果算法在 4 城上都算不对，后面的实验不用做了。
2. **小规模对比**：小实例上和精确解比，算 gap。小实例 gap 都 >5%，算法肯定有 bug。
3. **可扩展性**：规模翻倍，时间怎么涨？线性、二次还是指数？画 log-log 图看斜率。
4. **鲁棒性**：换 10 个随机种子，结果标准差大不大。如果标准差比均值还大，结论不可信。
5. **消融**：去掉每个组件，看性能降多少，证明每个组件都有用。

2.4 常见审稿质疑回应

- “baseline 调参了吗?”——baseline 也要调到最优，否则不公平。回应：给出 baseline 的调参过程（如 grid search 范围）。
- “显著性检验做了吗?”——跑 5 次，做 t-test 看差异是否显著。
- “为什么选这个参数?”——给参数敏感性分析图，说明参数在什么范围内结论都成立。
- “实例怎么生成的?”——公开生成器代码和种子，让别人能复现。

2.5 一次不严谨实验的诊断示范

论文写”新算法在 100 城市 TSP 上比 Gurobi 快 2.3 倍”。审稿人三连问：

1. **Gurobi 调参了吗?** 你说”用默认参数”。默认是保守模式，应调时间限制和 gap。调完 Gurobi 只慢 1.2 倍。
2. **跑了几个实例?** 你说”5 个”。5 个方差大，应跑 50 个取平均。
3. **最优性 gap 多少?** 你没测。快 2.3 倍但 gap 5%，Gurobi gap 0.1%——你的”快”是以质量换的。

三问答完，“2.3 倍”就站不住了。

2.6 消融实验的正确做法

算法有三组件：A（GNN 预测）、B（2-opt 改进）、C（问题嵌入）。

- 完整算法：路线长度 540。
- 去掉 A（用随机分支）：620。组件 A 贡献 = 80。
- 去掉 B（不做局部改进）：580。组件 B 贡献 = 40。
- 去掉 C（用 one-hot 嵌入）：555。组件 C 贡献 = 15。

三组件各自贡献一目了然。只做”全去掉”，你只知道整个 pipeline 比 baseline 好，不知道哪个组件最重要。

3. 直觉理解

> **直觉**：实验像做药——要双盲、对照组、大样本。自己做药自己试不算数。算法实验也一样：公开数据、公平 baseline、多次重复、统计检验。跑通只是第一步，跑准才是论文。很多研究生算法很牛，却因实验不严谨被拒，就是跳过了诊断流水线。

这个比喻里，“sanity check”是先在培养皿里试药（看有没有毒性），“小规模对比”是在小鼠上试药（和现有药比效果），“可扩展性”是剂量加大（看药效是否随剂量线性增长），“消融”是去掉每个成分看哪个有效。

4. 常见误区

误区一：只跑一个实例就下结论。单次结果可能是运气。

误区二：把”快 10%“当卖点却不报 gap。快但差没意义。

误区三：消融只做一次。要每个组件单独去掉看贡献，不能只做一个”全去掉”。

5. 代码实现

演示标准化实验脚本：固定种子、多次重复、报均值 ± 标准差、t-test。

```python
import numpy as np
from scipy import stats

def run_nn(seed, n=20):
    """ 最近邻 TSP 启发式：固定种子，返回路线长度。
    作为我们的" 新算法"——它应该比随机排列好。"""
    np.random.seed(seed)                # 固定种子保证可复现
    coords = np.random.rand(n, 2)       # n 个随机城市坐标
    tour = [0]; unvisited = set(range(1, n)); cur = 0
    while unvisited:
        # 贪心选最近邻
        nxt = min(unvisited, key=lambda j: np.linalg.norm(coords[cur]-coords[j]))
        tour.append(nxt); unvisited.remove(nxt); cur = nxt
    tour.append(0)                      # 回到起点
    return sum(np.linalg.norm(coords[tour[i]]-coords[tour[i+1]]) for i in range(n))

def run_rand(seed, n=20):
    """ 随机排列 TSP：对照 baseline。
    随机排一条路线，应该比最近邻差很多。"""
    np.random.seed(seed)
    coords = np.random.rand(n, 2)
    tour = list(range(n)); np.random.shuffle(tour); tour.append(tour[0])
    return sum(np.linalg.norm(coords[tour[i]]-coords[tour[i+1]]) for i in range(n))

# 重复 10 次（不同种子），报均值 ± 标准差
# 单次结果可能是运气，10 次取均值才可信
res_nn   = [run_nn(s)   for s in range(10)]
res_rand = [run_rand(s) for s in range(10)]
print(f" 最近邻: {np.mean(res_nn):.3f} ± {np.std(res_nn):.3f}")
print(f" 随机  : {np.mean(res_rand):.3f} ± {np.std(res_rand):.3f}")

# 双样本 t-test：检验两组结果的均值是否有显著差异
# p < 0.05 说明差异不太可能是随机波动
t, p = stats.ttest_ind(res_nn, res_rand)
print(f"t-test: t={t:.3f}, p={p:.4f}, 显著差异? {p < 0.05}")
```

预期最近邻远优于随机排列，且 $p < 0.05$ 差异显著——这就是”重复 + 统计检验”给出的可信结论。

这个演示展示了标准化实验的最小闭环：固定种子、重复多次、报均值 ± 标准差、做统计检验。如果只跑一次，你可能碰巧最近邻那次特别差、随机那次特别好——单次结果完全不可信。跑 10 次后，均值的对比才稳定。

6. 小结

1. 实验三原则：对照、随机、重复。
2. 用标准实例集，避免自己挑数据；同时报 gap 和时间。
3. 诊断流水线：sanity → 小规模 → 扩展性 → 鲁棒 → 消融。
4. 做统计显著性检验（t-test / bootstrap）。

局限与延伸

- 统计显著不等于实际重要；还要看效应大小。极端大样本下微小差异也”显著”。
- 延伸阅读：统计学的实验设计（Fisher）；NeurIPS Reproducibility Checklist；运筹实验的 standard benchmark 实践。

补充：当算法单次运行代价极高（如训练一个大模型要几天），做不了 10 次重复。替代方案：用 bootstrap 置信区间——把一次运行的内部随机性（如 dropout、数据顺序）当重复采样源，估计性能的置信区间。或者用统计效率更高的实验设计（如 paired t-test，同一个 seed 跑两个方法，配对比较）。

7. 思考题

基础档

1. 为什么 baseline 要调到最优？直接用默认参数不行吗？
2. 消融实验是为了回答什么问题？

进阶档

3. 你的算法小规模比 baseline 差、大规模好，论文怎么写？（提示：报告 crossover point。）
4. 怎么判断一个结果是”算法好”还是”运气好”？从重复次数和 p 值角度回答。

开放档

5. 大模型论文经常被质疑”只在一个数据集上测”。这对应本讲哪个陷阱？怎么改？
6. 当算法是随机的、且单次运行代价极高（训练一个大模型），怎么做统计检验？

第 24 讲 论文写作的运筹范式：从算法到科学贡献

> **本讲问题**：算法写完了、实验跑完了，最后一关是写作。很多人算法好但论文被拒，因为写作没讲清楚”科学贡献”——不是”我做了一个更快的算法”，而是”我回答了一个什么科学问题”。本讲给一套摘要和论文的写作范式，让你的算法贡献被审稿人看见。

1. 数学工具/预备知识

1.1 摘要四要素

一篇论文的摘要必须在 200 字内回答四个问题：

1. **问题是什么**：领域里存在什么未解的困难。
2. **你做了什么**：方法叫什么、核心思想一句话。
3. **结果是什么**：在什么数据上、比 baseline 好多少。
4. **贡献是什么**：你留下了什么别人能用的东西。

四要素缺任何一个，审稿人就会困惑。缺问题——不知道你在解决什么；缺方法——不知道你怎么做的；缺结果——不知道你做成了没有；缺贡献——不知道这篇论文有什么留存价值。

1.2 定理-实验对应原则

论文里每个理论结论（定理、引理、推论）都必须在实验章节有对应的验证。反过来，每个实验图都必须对应一个理论主张。为什么？因为审稿人问的是”你的理论有什么用”。如果写了 5 个定理但实验只测了一个指标，审稿人会问”其他 4 个定理怎么验证”。

具体做法：在写定理时就想好”这个定理的推论是什么实验可测的”。如果一个定理没有可测推论，它可能太抽象，需要加一个推论把它落到实验上。

1.3 三类审稿质疑

审稿意见基本分三类：

- **Novelty（新意）**：和已有工作的区别在哪？回应方式：对比表，列出最近 3 篇，标出本文在哪格不同。
- **Rigor（严谨）**：证明有没有 gap？实验公不公平？回应方式：补证明细节、补 baseline 调参、补实验实例。
- **Significance（价值）**：这个结果为什么重要？回应方式：强调工业影响、加案例、说明对领域的推动。

1.4 贡献列表的格式

贡献列表 3~4 条，每条格式：“动词 + 具体结果 + 量化数字”。例如：“(1) 提出 GNN 分支策略，在 TSPLIB 1000 城上比 Gurobi 默认分支快 2.3 倍；(2) 证明预测分支与最优分支的 gap 上界为 5%；(3) 开源代码和数据。”少于 3 条显得贡献不够，多于 5 条说明你没抓住重点。

为什么写作本身是运筹问题

写论文像做优化：目标是”让审稿人 5 分钟看懂、30 分钟相信”，约束是页数、格式、领域惯例。每段话都是决策，都要服务目标——不贡献的段落删掉。这正是把前面 23 讲的”资源分配”思维用到写作上。

2. 完整推导

2.1 摘要四要素结构

一段好摘要四句话：

1. **问题**：“组合优化在大规模 TSP 上仍存在求解时间长的问题。”——要具体，不能说”优化问题很重要”。
2. **方法**：“本文提出 Learning to Branch，用 GNN 预测分支变量，嵌入 branch-and-cut。”——要给方法名和核心技术。
3. **结果**：“在 1000 城市 TSPLIB 上，求解时间比 Gurobi 快 2.3 倍，gap 保持在 1% 内。”——要给数据集、baseline、量化数字。
4. **贡献**：“首次证明 GNN 预测分支在欧氏 TSP 上可泛化到 2 倍规模。”——要给科学贡献，不是工程贡献。

四句话的顺序不能乱：先问题（为什么读）、再方法（你做了什么）、再结果（做成了吗）、最后贡献（为什么重要）。审稿人按这个顺序读，缺任何一环都会困惑。

2.2 定理-实验对应

- 论文里每个定理，要在实验章节有对应表/图验证。
- 每个实验图，正文要有一句话说明”这张图验证了定理 X 的推论 Y”。
- 反过来，每个实验结果要能回答”它支撑了哪个理论主张”。

不对应的后果：定理写一堆，实验只测一个指标，审稿人问”你的定理有什么用”。

2.3 三类审稿质疑回应

novelty 不够：明确指出和最近 X、Y 的区别——在哪个假设上放松了、用了什么新技术。写对比表：列最近 3 篇，每列假设/方法/结果，标出本文在哪格不同（呼应第 22 讲缺口）。

rigor 不够：补证明细节、补实验、补 baseline 调参。证明有 gap 补引理；实验不够补实例。具体来说：如果审稿人说”baseline 没调参”，你的回应是”已用 Gurobi 默认 gap=1e-4 重跑所有 baseline，表 3 更新”，而不是”我们觉得默认参数够了”。

significance 不足：强调对工业界/学术界的影响，加案例。如”本文方法已应用于某物流企业日常调度”。如果没有工业落地，强调理论意义：“本文首次证明了 X 类问题在 Y 条件下可近似到 Z 倍”。

2.4 论文结构模板

1. Introduction：问题 + 贡献列表（3~4 条）。前 3 段必须完成”宽 → 窄 → 点”的漏斗。
2. Related Work：分类对比（不是罗列）。把相关工作分成 2~3 类，每类说清楚它做了什么、局限在哪，最后引出本文属于哪类、补了哪个局限。
3. Problem Formulation：四要素（第 21 讲）。决策变量、目标、约束、数据假设，一一列清。
4. Algorithm：方法。从直觉到形式化，先讲思路再写公式。
5. Theoretical Analysis：定理。每个定理给证明梗概，附录补细节。
6. Experiments：诊断流水线（第 23 讲）。sanity → 小规模 → 扩展性 → 消融 → 鲁棒性。
7. Conclusion：总结 + 未来工作。不要重复摘要，要点出”本文打开了什么方向”。

2.5 摘要写作示范：从差到好

差的摘要：“本文研究了组合优化问题，提出一种新方法，实验表明效果良好。”——没具体问题、没方法名、没数字、没贡献。

好的摘要：“大规模 TSP 在 branch-and-cut 求解器上仍需数小时。本文提出 Learning to Branch，用图神经网络预测 LP 松弛后的分支变量选择。在 TSPLIB 1000 城市实例上，求解时间比 Gurobi 默认分支快 2.3

倍，gap 保持在 1% 内。贡献：(1) 首个 GNN 分支策略在欧氏 TSP 上可泛化到 2 倍规模；(2) 证明预测分支的剪枝效率与最优分支的 gap 上界为 5%。”

好摘要有问题、方法名、数字、贡献列表。

2.6 引言前三段的”漏斗”结构

审稿人通常在前 3 段决定粗读还是精读。这三段应是一个漏斗：

- **第 1 段（宽）**：领域与重要性。“组合优化是物流、芯片、调度的核心。”
- **第 2 段（窄）**：现有方法的具体瓶颈。“但 branch-and-cut 在 1000 城 TSP 上仍需数小时，瓶颈在分支变量选择。”
- **第 3 段（点）**：本文贡献列表。“本文 (1) 提出 GNN 预测分支；(2) 证明泛化上界；(3) 实证提速 2.3 倍。”

宽 → 窄 → 点，让审稿人在 5 分钟内看清”为什么是你、为什么是现在”。

为什么是漏斗？因为审稿人读论文的耐心是递减的。第 1 段如果没抓住注意力，他可能直接拒了。第 2 段必须说清楚”现有方法哪里不行”——不能只说”有改进空间”，要具体到”分支变量选择占 60% 时间”。第 3 段直接列贡献，不要铺垫。

2.7 审稿意见回应信的格式

回应信不是道歉，而是逐条对照表。格式：

审稿意见	我们的回应	修改位置
“baseline 没调参”	已用 Gurobi 默认 gap=1e-4 重跑，表 3 更新	Sec 5.2, 表 3
“t-test 只跑 5 次”	已增至 30 次，p 值从 0.03 降到 <1e-3	Sec 5.4

每条先”感谢”、再”具体改了什么”、最后指页码。原则：不与审稿人辩论，把分歧转成”已补充实验/证明”。

如果审稿人提的意见确实无法满足（如”请证明一个 NP-hard 问题的多项式算法”），不要硬刚——解释为什么这个方向不可行，并给出你做了什么替代实验来支撑论点。回应信的目标是让审稿人觉得”作者认真对待了我的意见”，不是”我赢了辩论”。

3. 直觉理解

> **直觉**：论文不是算法文档，而是一份”说服合同”——你和审稿人签订：“我承诺回答了 X 科学问题，证据在 Y、Z 章节。”写作目标是让审稿人 5 分钟看懂贡献、30 分钟相信证据。啰嗦的章节、不对应实验的定理，都是在撕毁合同。运筹思维告诉你：每段话都是决策，决策要服务目标（让审稿人接受）。

这个合同的核心条款：(1) 问题陈述清楚（我承诺解决 X）；(2) 方法可复现（我承诺把方法写得别人能重跑）；(3) 实验充分（我承诺给足够证据）；(4) 贡献明确（我承诺这篇论文留下了什么）。审稿人就是合同的验收方——他验收通过（接收），合同生效；验收不通过（拒稿），你回去补条款。

4. 常见误区

> **误区一：把摘要写成”本文研究了……“。**摘要要直接给结果和贡献，不要背景铺垫。
>
> **误区二：Introduction 写半页背景才到贡献。**贡献列表要在前 3 段出现。
>
> **误区三：相关工作写成罗列。**要分类对比，指出每类局限，引出本文位置。

5. 代码实现

本讲无代码，给一个可直接套用的摘要模板。

```
## 摘要模板

[问题一句话]。[现有方法的不足一句话]。本文提出[方法名]，[方法核心思想一句话]。
理论上，我们证明了[定理结论一句话]。实验上，在[数据集]上，我们的方法比
[baseline] [指标提升 X%]，且[额外性质]。本文贡献：(1) ...；(2) ...；(3) ...。
```

写贡献列表时自查：每条是否”动词 + 具体结果 + 量化数字”？三条是否层次不同（理论/方法/实证）？

自查清单：- 每条贡献能不能被一句话总结？不能说明太啰嗦。- 三条贡献是否在不同层面？（如：一条理论证明、一条新算法、一条实证发现——如果三条都是”我们做了 X 实验”，层次不够。）- 贡献列表和摘要里说的是否一致？不一致说明你写了两版。

6. 小结

1. 摘要四要素：问题、方法、结果、贡献。
2. 定理-实验一一对应。
3. 回应审稿质疑：novelty、rigor、significance。
4. 论文写作是说服，不是文档——把它当优化问题做资源分配。

局限与延伸

- 写作范式是通用骨架，但不同领域（OR vs ML）侧重不同：OR 重定理严谨，ML 重实证与 scale。
- 延伸阅读：《How to Write a Great Research Paper》；顶会论文范文（Operations Research、NeurIPS）；同行评审的伦理与回应信写作。

补充：投稿前自查清单——(1) 摘要四要素齐了吗？(2) 贡献列表 3~4 条、每条量化了吗？(3) 每个定理有对应实验验证吗？(4) 每个实验对应一个理论主张吗？(5) baseline 调到最优了吗？(6) 跑了几个种子、报了均值 ± 标准差吗？这六条全过了再投。

7. 思考题

基础档

1. 拿一篇你读过的论文，拆它的摘要四要素。
2. 为什么贡献列表要 3~4 条？少于 3 或多于 5 会怎样？

进阶档

3. 怎么在 Introduction 里 5 分钟让审稿人抓住贡献？（提示：前 3 段结构。）
4. 审稿人说”你的贡献和 X 论文太像”，怎么用第 21、22 讲的四要素对比表回应？

开放档

5. 大模型时代，写作辅助（AI 润色）能帮你写摘要，但不能帮你决定”科学问题”在哪。人的增值点是什么？
6. 一篇论文的定理很扎实但实验弱（或反之），在投稿选择上该怎么权衡？
